

Algoritmi per l'ingegneria

[Appunti di Piai Luca]

Academic Year 2023-2024

Indice

1	Problemi computazionali e algoritmi	6
1.1	Definizioni principali	6
1.2	Analisi di un algoritmo	7
2	Il paradigma Divide And Conquer	8
2.1	Proprietà di sotto-struttura	8
2.2	Albero delle chiamate	9
2.2.1	Il problema della mancanza di memoria	9
2.3	Caso di studio: determinare il massimo in un array di interi	10
2.3.1	Analisi di correttezza	11
2.3.2	Analisi di complessità	12
2.4	Induzione parametrica	14
2.4.1	Applicazione dell'induzione parametrica alla ricorrenza generale di MAX	15
2.5	Studio dell'albero della ricorrenza	16
2.5.1	L'albero delle chiamate di MAX	17
2.6	Formula generale di risoluzione di ricorrenze associate ad algoritmi Divide and Conquer	18
2.6.1	Le iterate di $f(n)$	19
2.6.2	Analisi dell'albero della ricorrenza	20
2.7	Il Master Theorem	22
2.7.1	Versione semplificata del MT	22
2.7.2	Prova del MT semplificato	23
2.7.3	Utilità del MT	26
2.7.4	Versione estesa del MT	26
2.7.5	Applicare il MT nel caso del MAX e del MERGESORT	27
2.7.6	Casi in cui la condizione di regolarità è semplice	27
3	Moltiplicazioni di matrici	28
3.1	Somma e sottrazione di matrici	28
3.2	Prodotto di matrici (versione iterativa)	29
3.3	Prodotto di matrici (algoritmo ricorsivo)	30
3.4	L'algoritmo di Strassen	33
3.4.1	L'idea	33
3.4.2	L'algoritmo nel dettaglio	34
3.4.3	Analisi dettagliata	36
3.5	Matrici rettangolari	36
3.6	Ibridazione	37
3.6.1	Analisi dettagliata	38

4	Polinomi e FFT	40
4.1	Polinomi e le loro rappresentazioni	40
4.1.1	Rappresentazione tramite coefficienti	40
4.1.2	Rappresentazione per punti	41
4.1.3	Somma nella rappresentazione per coefficienti	41
4.1.4	Prodotto nella rappresentazione per coefficienti	42
4.1.5	Somma nella rappresentazione per punti	44
4.1.6	Prodotto nella rappresentazione per punti	44
4.1.7	Conversione da coefficienti a punti: la valutazione	44
4.1.8	Conversione da punti a coefficienti: l'interpolazione	45
4.2	Discrete Fourier Transform e teorema di convoluzione lineare	46
4.3	La matrice di Fourier	48
4.4	Numeri complessi e radici n-esime dell'unità	48
4.4.1	Rappresentazione polare ed esponenziale	48
4.4.2	Proprietà delle radici n-esime dell'unità	49
4.5	Proprietà di sotto-struttura per il calcolo di DFT_n	52
4.5.1	Il caso base	52
4.5.2	Passo generico	52
4.6	La Fast Fourier Transform (FFT)	53
4.7	La trasformata discreta inversa di Fourier	54
4.7.1	La matrice F_n^{-1}	54
4.7.2	DFT^{-1} come valutazione su potenze negative di ω_n	55
4.8	Calcolo efficiente della convoluzione lineare	57
5	Programmazione dinamica	58
5.1	Il problema del D&C: i numeri di Fibonacci	58
5.2	Prime considerazioni sulla programmazione dinamica	61
5.3	La memoizzazione	61
5.4	Problemi di ottimizzazione e proprietà di sotto-struttura ottima	63
5.5	Paradigma DP	63
5.6	Problemi su stringhe	64
5.7	Longest Common Subsequence (LCS)	65
5.7.1	Proprietà di sotto-struttura ottima	65
5.7.2	Dimostrazione della proprietà di sotto-struttura ottima	66
5.7.3	Ricorrenza sui costi e informazione aggiuntiva	67
5.7.4	Calcolo dei costi e dell'informazione aggiuntiva (approccio bottom-up)	68
5.7.5	Codice bottom-up	69
5.7.6	Complessità al caso peggiore utilizzando un approccio D&C	70
5.7.7	Algoritmo memoizzato	71
5.8	Matrix Chain Multiplication (MCM)	72
5.8.1	Premesse	72
5.8.2	Descrizione del problema	73
5.8.3	Proprietà di sottostruttura ottima	74
5.8.4	Ricorrenza sui costi e informazione aggiuntiva	75
5.8.5	Approccio DP bottom-up	76
5.8.6	Algoritmo bottom-up con diverse scansioni della matrice	78
5.8.7	Algoritmo memoizzato	79
5.8.8	Stampa della parentesizzazione di costo minimo	79

6	Il paradigma Greedy	81
6.1	Critica alla programmazione dinamica	81
6.2	Aspetti generali dell'approccio Greedy	81
6.3	Analisi di un algoritmo Greedy	82
6.4	Il problema della selezione delle attività	82
6.4.1	Algoritmo Greedy	83
6.4.2	Correttezza	84
6.4.3	Scelte Greedy alternative	85
6.5	Compressione	86
6.5.1	Rappresentazione di un prefix code	86
6.5.2	Metrica di costo	87
6.5.3	Problema della codifica di carattere ottima	87
6.5.4	Proprietà di un trie ottimo	87
6.5.5	Codifica di Huffman: intuizione	88

Capitolo 1

Problemi computazionali e algoritmi

1.1 Definizioni principali

Definizione 1.1 (Algoritmo). *Un algoritmo è definito come una procedura computazionale finita e deterministica specificata come una sequenza di passi elementari (istruzioni) di un dato modello computazionale. Un algoritmo trasforma un dato in ingresso (input) in una e una sola uscita (output).*

Un algoritmo può essere visto come una funzione matematica che mappa gli ingressi nelle uscite.

$$A : I \rightarrow O$$

Diciamo che due algoritmi sono *equivalenti* se ogni ingresso viene mappato nella stessa uscita.

Definizione 1.2 (Problema computazionale). *Un problema computazionale Π è una specifica astratta della relazione esistente tra un insieme di istanze $\mathbb{I}$ e un insieme di soluzioni $\mathbb{S}$.*

$$\Pi \subseteq \mathbb{I} \times \mathbb{S} = \{(i_1, s_1), \dots, (i_n, s_n)\}$$

Proprietà del problema computazionale Π :

1. Deve essere una **relazione totale**, ovvero ogni istanza deve essere associata ad almeno una soluzione.

$$\forall i \in \mathbb{I}, \exists s \in \mathbb{S} \mid (i, s) \in \Pi$$

(Successivamente andremo ad utilizzare la notazione $i \Pi s$ che equivale a $(i, s) \in \Pi$).

2. La relazione Π **non è necessariamente univoca**, ovvero possono esistere soluzioni diverse per stesse istanze.

$$\exists s_1, s_2 \in \mathbb{S} \text{ con } s_1 \neq s_2 \mid \exists i \in \mathbb{I} \mid (i \Pi s_1) \wedge (i \Pi s_2)$$

Definizione 1.3 (Algoritmo che risolve un problema). *Dato un algoritmo $A_\pi : I \rightarrow O$, esso risolve un problema computazionale Π se*

$$I = \mathbb{I} \quad O = \mathbb{S} \quad \text{e} \quad \forall i \in \mathbb{I} : A_\pi(i) = s \in \mathbb{S} \Rightarrow i \Pi s$$

Osservazione 1.1. *Identificare l'insieme degli input e l'insieme degli output con le istanze e le soluzioni astratte è una semplificazione. In realtà, gli insiemi I e O sono una rappresentazione (codifica) di $\mathbb{I}$ e $\mathbb{S}$ compatibile con il modello computazionale adottato.*

Osservazione 1.2. *Nel caso in cui una istanza $i \in \mathbb{I}$ sia in relazione con più soluzioni in $\mathbb{S}$, l'algoritmo A_π associa ad i una ed una sola di queste. Dunque l'algoritmo estrae una relazione univoca (funzione) dalla relazione generale Π .*

1.2 Analisi di un algoritmo

Definizione 1.4 (Analisi della correttezza). *Provare la correttezza di un algoritmo A_π per un problema computazionale Π equivale a dimostrare la verità del seguente predicato:*

$$\forall i \in \mathbb{I} : (A_\pi(i) = s) \Rightarrow (i \Pi s)$$

Definizione 1.5 (Complessità). *L'analisi della complessità si basa su un modello di costo, che associa un dato costo non negativo a ogni istruzione presente nel corredo del modello computazionale.*

$$i_{istr} \rightarrow c_{istr} \in \mathbb{R}^+ \cup \{0\}$$

Dato un algoritmo A_π e una istanza/input $i \in \mathbb{I}$, la complessità dell'esecuzione $A_\pi(i)$ è la quantità

$$t_{A_\pi, i} = \sum_{\text{istr eseguite da } A_\pi(i)} c_{istr}$$

Essendo interessati ad analisi asintotiche, ignoreremo le costanti e assoceremo alle istruzioni un costo unitario oppure nullo, con l'idea di concentrarci sul computo del tipo di istruzioni che, si ritiene, dominino la complessità dell'algoritmo (in ordine di grandezza). A seconda del contesto, si potranno considerare diversi tipi di istruzioni a costo unitario.

L'analisi dei valori $t_{A_\pi, i}$ è un modo irragionevole di studiare la complessità di un algoritmo in quanto tale valore può cambiare per ogni $i \in \mathbb{I}$. Sono quindi necessarie **metriche di complessità riassuntive**. A tal fine, partizioneremo l'insieme delle istanze $\mathbb{I}$ in sottoinsiemi in base alla loro **taglia**.

Definizione 1.6 (Taglia). *La taglia di un'istanza $|i|$, con $|\cdot| : \mathbb{I} \rightarrow \mathbb{N}$, è una misura intera non negativa ragionevolmente rappresentativa della grandezza (della codifica) di un'istanza.*

Per $n \in \mathbb{N}$ si definisce $\mathbb{I}_n = \{i \in \mathbb{I} : |i| = n\}$. La complessità temporale di un algoritmo A_π verrà descritta da una funzione $T_{A_\pi}(n) : \mathbb{N} \rightarrow \mathbb{R}^+ \cup \{0\}$ che riassume con un solo valore la complessità di tutte le istanze in $\mathbb{I}_n$.

Definizione 1.7 (Complessità al caso peggiore).

$$T_{A_\pi}(n) = \sup_{\substack{i \in \mathbb{I} \\ |i| = n}} \{t_{A_\pi, i}\}$$

Definizione 1.8 (Complessità al caso migliore).

$$T_{A_\pi}(n) = \inf_{\substack{i \in \mathbb{I} \\ |i| = n}} \{t_{A_\pi, i}\}$$

Definizione 1.9 (Complessità al caso medio).

$$T_{A_\pi}(n) = E[t_{A_\pi, i}]$$

Capitolo 2

Il paradigma Divide And Conquer

Dato un problema $\Pi \subseteq \mathbb{I} \times \mathbb{S}$, per ottenere un approccio D&C bisogna individuare una proprietà di sotto-struttura, che metta in relazione istanze con sotto-istanze (*Divide*) e specifichi la ricombinazione da effettuare per ottenere la soluzione dell'istanza in funzione delle soluzioni delle sotto-istanze (*Conquer*).

2.1 Proprietà di sotto-struttura

Una proprietà di sotto-struttura di un problema $\Pi \subseteq \mathbb{I} \times \mathbb{S}$ è associata alla coppia di funzioni D (*Divide*) e C (*Conquer*) e a una taglia $n_0 \in \mathbb{N}$. La proprietà deve descrivere le soluzioni per le istanze i con $|i| \leq n_0$ (**istanze di base**). Mentre per le istanze con taglia $|i| > n_0$ (**istanze non di base**), la funzione *Divide* D prende un'istanza i e restituisce una sequenza di istanze:

$$D : \mathbb{I} \rightarrow \mathbb{I}^* \\ i \xrightarrow{D} \langle i_1, \dots, i_k \rangle$$

la funzione *Conquer* C prende una sequenza di soluzioni e restituisce una soluzione:

$$C : \mathbb{S}^* \rightarrow \mathbb{S}. \\ \langle s_1, \dots, s_k \rangle \xrightarrow{C} s$$

Le funzioni D e C soddisfano le seguenti proprietà:

- $\forall i, |i| > n_0$, se $D(i) = \langle i_1, \dots, i_k \rangle$ allora $|i_j| < |i| \ 1 \leq j \leq k$. (Ovvero le sotto-istanze hanno taglia inferiore).
- $\forall i, |i| > n_0$, se $D(i) = \langle i_1, \dots, i_k \rangle$ e $s_1, \dots, s_k$ sono tali che $i_1 \Pi s_1, \dots, i_k \Pi s_k$ allora, detta $s = C(\langle s_1, \dots, s_k \rangle)$, si ha che $i \Pi s$. In altre parole, la soluzione s dell'istanza "grande" (di partenza) i si ottiene "componendo" (tramite la funzione C) le soluzioni $s_1, \dots, s_k$ ottenute dalle sotto-istanze "più piccole" $i_1, \dots, i_k$ generate dalla funzione D .

Definita una proprietà di sotto-struttura del problema computazionale Π , è sempre possibile trovare un algoritmo D&C che risolve tale problema. Per ricavare l'algoritmo dobbiamo innanzitutto disporre di un **metodo risolutivo diretto** per le *istanze di base*, ovvero per le istanze $i \in \mathbb{I}$ che hanno taglia $|i| \leq n_0$. Ci rimane da implementare un algoritmo $A_D(i)$ che realizza la funzione $D(i) = \langle i_1, \dots, i_k \rangle$ e un algoritmo $A_C(\langle s_1, \dots, s_k \rangle)$ che realizza la funzione $C(\langle s_1, \dots, s_k \rangle)$. Poniamo di avere a disposizione questi due algoritmi, allora possiamo scrivere il *metacodice* generale di un algoritmo **ricorsivo** D&C che risolve Π :


```

1 D&C(i) { $i \in \mathbb{I}$ }
2   [B]: if ( $|i| \leq n_0$ ) then *risolvi direttamente*
3       // indichiamo la fase di risoluzione dei casi base con [B]
4
5   [D]:  $\langle i_1, \dots, i_k \rangle \leftarrow A_D(i)$ 
6       // fase di divide [D]
7
8   [R]: for  $j \leftarrow 1$  to  $k$  do  $s_j \leftarrow D\&C(i_j)$ 
9       // nella fase di recurse [R] otteniamo "gratuitamente" le soluzioni
10      // alle sotto-istanze tramite chiamate ricorsive
11
12   [C]: return  $A_C(s_1, s_2, \dots, s_k)$ 
13      // fase di conquer [C]

```

Abbiamo indicato con $D\&C(i)$ il nome del generico algoritmo, tra parentesi grafe abbiamo imposto un **vincolo statico** sulla variabile i .

In generale non è detto che le quattro sezioni *Base*, *Divide*, *Recurse* e *Conquer* siano tutte definite e ben distinte in ogni algoritmo: potrebbero intrecciarsi.

2.2 Albero delle chiamate

La generazione e risoluzione delle istanze associate ad un algoritmo D&C viene schematizzato utilizzando l'**albero delle chiamate** associato all'algoritmo e a una data istanza i . La radice dell'albero rappresenta l'istanza i (istanza principale). Ipotizzando che i non sia di base, da questa verranno generate da $A_D(i)$ k sotto-istanze $\langle i_1, \dots, i_k \rangle$, queste rappresentano i figli della radice. Analogamente da queste sotto-istanze (quelle che non rientrano in un caso base) verranno generate ulteriori sotto-istanze, e così via finché non si arriva alle foglie, le quali sono tutte associate a sotto-istanze di base. Nota che il numero di sotto-istanze generate da un nodo non è fisso, può variare, tuttavia la maggioranza degli algoritmi che studieremo in questo corso genererà un albero delle chiamate in cui i nodi interni hanno grado due.

La funzione A_D è associata alla generazione delle sotto-istanze, quindi di fatto espande l'albero dalla radice verso le foglie (modalità **top-down**). Viceversa la funzione A_C è associata alla produzione delle soluzioni delle istanze a partire dalle sotto-istanze, quindi di fatto contrae l'albero dalle foglie alla radice (modalità **bottom-up**). In questo modo arriva la soluzione finale della radice.

Osserviamo che nella pratica, dando in pasto l'algoritmo ad un risolutore, non viene generato interamente l'albero delle chiamate ma invece avviene un attraversamento in pre-ordine dell'albero. In questo può essere gestita l'occupazione in memoria.

2.2.1 Il problema della mancanza di memoria

Abbiamo visto che ad ogni sotto-istanza non di base è associato un sotto-albero. Nel momento che tale sotto-albero viene contratto e la soluzione della sotto-istanza viene calcolata, tutta l'informazione associata ai nodi di quel sotto-albero viene distrutta. Ciò è dovuto al fatto che l'approccio D&C non mantiene in memoria alcuna informazione sulle sotto-istanze già risolte, non ricorda le loro soluzioni. Questo comporta che, all'interno di uno stesso albero delle chiamate, la stessa istanza venga generata più volte e che quindi lo stesso sotto-albero venga più volte generato e contratto. Vederemo più avanti che questa "debolezza" verrà risolta dal successivo paradigma che andremo a studiare.

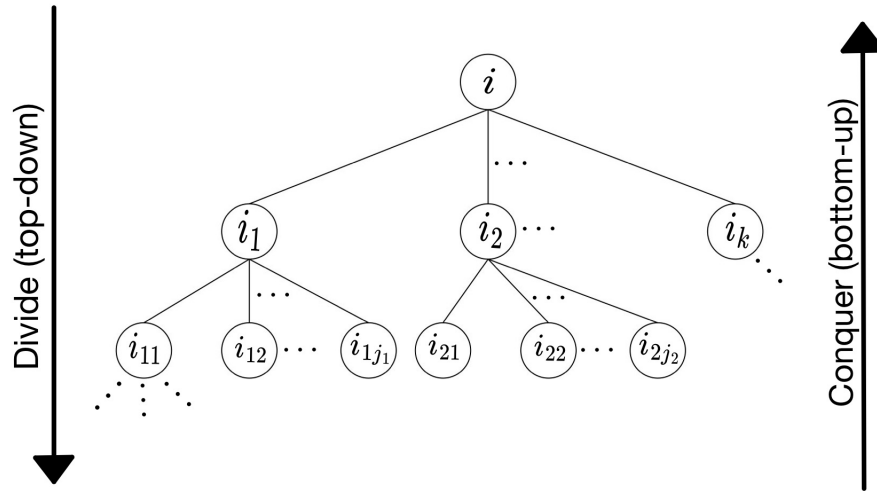


Figura 2.1: Albero delle chiamate di un algoritmo D&C.

2.3 Caso di studio: determinare il massimo in un array di interi

Problema: determinare il massimo in un array di interi $A[1..n]$ con $n > 0$ arbitrario. Definiamo la proprietà di sotto-struttura:

- $n = 1$: (Base) il massimo è $\text{MAX} = A[1]$;
- $n > 1$:
 - (Divide) divido il vettore in due parti:

$$D : \mathbb{I} \rightarrow \mathbb{I}^*$$

$$\underbrace{A[1..n]}_i \xrightarrow{D} \underbrace{A[1..\lceil n/2 \rceil]}_{i_1} \underbrace{A[\lceil n/2 \rceil + 1..n]}_{i_2}$$

- (Conquer) torna il massimo tra i due massimi:

$$C : \mathbb{S}^* \rightarrow \mathbb{S}$$

Se:

$$A[1..\lceil n/2 \rceil] \sqcap_{\max} m_1, \quad m_1 = s_1$$

$$A[\lceil n/2 \rceil + 1, \dots, n] \sqcap_{\max} m_2, \quad m_2 = s_2$$

Allora:

$$A[1..n] \sqcap_{\max} \underbrace{\max\{m_1, m_2\}}_{C(\langle s_1, s_2 \rangle)}$$

Dalla definizione della proprietà di sotto-struttura ricaviamo direttamente il pseudocodice:

```

1 MAX(A, i, j)
2 [B]: if (i = j) then return A[i]
3 [D]: k ← ⌊(i + j)/2⌋
    
```

```

4 [R]: m1 ← MAX(A, i, k)
5     m2 ← MAX(A, k + 1, j)
6 [C]: if (m1 ≤ m2) then return m1
7     else return m2

```

dove abbiamo utilizzato gli indici i, j per poter riferirci ad una sotto-istanza generica di una chiamata dell'algoritmo.

2.3.1 Analisi di correttezza

Il metodo principale per provare la correttezza di una proprietà di sotto-struttura è l'induzione, con parametro uguale alla taglia $n = |i|$ della generica istanza $i \in \mathbb{I}$. La dimostrazione per induzione della correttezza dell'algoritmo ottenuto dalla proprietà di sotto-struttura procede analogamente come segue:

- **Base [B]:** occorre provare la correttezza della risoluzione diretta delle istanze di base, ovvero quelle di taglia al più n_0 .
- **Ipotesi Induttiva [HP]:** si ipotizzano corrette le soluzioni, restituite dall'algoritmo, di sotto-istanze di taglia inferiore a un generico valore $n > n_0$.
- **Tesi induttiva [TH]:** data una istanza i di taglia $|i| = n$, occorre provare la correttezza di A_D , che deve produrre sotto-istanze di taglia strettamente minore di n e di A_C , che deve restituire una soluzione all'istanza principale.

Analizziamo la correttezza del caso di studio introdotto nella sezione precedente. In questo caso notiamo che il *parametro dell'induzione* n è funzione dei parametri i, j dell'algoritmo. Dunque definiamo:

$$n = j - i + 1$$

Dimostriamo la correttezza dell'algoritmo sfruttando l'induzione.

$$[B]: \quad \text{Se } (n = 1) \Rightarrow j - i + 1 = 1 \Rightarrow j = i.$$

allora MAX è corretto perché restituisce $A[i]$, unico elemento del sotto-vettore $A[i..i]$ che è anche suo massimo.

$$[HP]: \quad \text{Supponiamo che MAX sia corretto per tutte le taglie } l < n$$

[TH]: Correttezza del divide: per dimostrare la correttezza del divide dobbiamo dimostrare che le taglie delle sotto-istanze ottenute sono > 0 e $< n$.

Inoltre abbiamo che $j > i \geq 0$.

Le taglie delle sotto-istanze i_1, i_2 generate sono

$$\begin{cases} |i_1| = k - i + 1 \\ |i_2| = j - (k + 1) + 1 \end{cases} \quad k = \left\lfloor \frac{i+j}{2} \right\rfloor$$

$$|i_1| = \left\lfloor \frac{i+j}{2} \right\rfloor - i + 1 \leq \frac{i+j}{2} - i + 1 = \frac{j-i}{2} + 1 < j - i + 1 = n \Rightarrow |i_1| < n$$

$$|i_2| = j - \left\lfloor \frac{i+j}{2} \right\rfloor \leq j - \frac{i+j}{2} = \frac{j-i}{2} < j - i + 1 = n \Rightarrow |i_2| < n$$

$$|i_1| = \left\lfloor \frac{i+j}{2} \right\rfloor - i + 1 \geq \left\lfloor \frac{j}{2} \right\rfloor + 1 > 1 > 0 \Rightarrow |i_1| > 0$$

$$|i_2| = j - \left\lfloor \frac{i+j}{2} \right\rfloor \geq j - \frac{i+j}{2} = \frac{j-i}{2} > \frac{i-i}{2} = 0 \Rightarrow |i_2| > 0$$

[TH] : Correttezza del conquer: il conquer calcola il massimo per l'array $A[i..j]$ a partire dai massimi dei sotto-array $A[i..k]$ e $A[k+1..j]$.

Definiamo:

$$A = \{A[l] : i \leq l \leq j\}, \quad A_1 = \{A_1[l_1] : i \leq l_1 \leq k\}, \quad A_2 = \{A_2[l_2] : k+1 \leq l_2 \leq j\},$$

Per l'ipotesi induttiva HP sappiamo che MAX calcola correttamente i valori

$$m_1 = \max A_1$$

$$m_2 = \max A_2$$

Poiché $A = A_1 \cup A_2$, si ha che il valore tornato da MAX per l'istanza $A[i..j]$ è corretto in quanto MAX ritorna correttamente $\max\{m_1, m_2\} = \max A = m$, in quanto il massimo di una unione (finita) di insiemi (finiti) di interi è uguale al massimo dei massimi di tali insiemi.

In questo esempio abbiamo dimostrato sia la correttezza della proprietà di sotto-struttura che la correttezza dell'algoritmo. In futuro vedremo più spesso solo la dimostrazione di correttezza della proprietà di sotto-struttura e diremo che la correttezza dell'algoritmo deriva da questa.

2.3.2 Analisi di complessità

Il primo passo è quello di definire il modello di costo che si intende utilizzare. Nel caso del nostro esempio, definiamo

- Taglia: $n = |A[i..j]| = j - i + 1$ (dimensione del vettore).
- Costo istruzione: costo = 1 solo per confronti tra elementi del vettore (i confronti tra indici hanno costo 0).
- Complessità: consideriamo il caso peggiore.

Ora che abbiamo definito il modello di costo possiamo ricavare la complessità dell'algoritmo. Il modello di costo non è arbitrario: i parametri vengono fissati in base alla caratteristica che vogliamo studiare. Per esempio in questo caso abbiamo supposto nullo il costo dei confronti tra indici, per avere un'analisi più precisa potevamo anche considerare questi costi.

Nel caso di algoritmi D&C, per calcolare la complessità, si definisce una **relazione di ricorrenza** (o semplicemente **ricorrenza**). Una ricorrenza è una definizione implicita di $T(n)$ in funzione di valori assunti dalla stessa T su valori inferiori del parametro indipendente (ad esempio, $n-1$ o $n/2$). Solitamente la relazione di ricorrenza è definita a tratti, in particolare, con definizione esplicita per i casi base e definizione implicita per valori del parametro superiori.

Ricaviamo la ricorrenza del caso del MAX.

```

1 MAX(A, i, j)
2 [B]: if (i = j) then return A[i]
3       // T(1) = 0, no confronti con elementi del vettore
4
5 [D]: k ← ⌊(i + j)/2⌋
6       // TAd = 0, no confronti con elementi del vettore
7
8 [R]: m1 ← MAX(A, i, k)
9       m2 ← MAX(A, k + 1, j)
10      // TR = T(⌈n/2⌋) + T(⌊n/2⌋), MAX viene invocata due volte
11
12 [C]: if (m1 ≤ m2) then return m1 else return m2
13      // TAc = 1, un confronto tra elementi del vettore (m1 e m2)
    
```

Possiamo definire la relazione di ricorrenza:

$$T(n) = \begin{cases} T(n_0) & \text{se } n \leq n_0 \\ T_{Ad} + T_R + T_{Ac} & \text{se } n > n_0 \end{cases} = \begin{cases} 0 & \text{se } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + 1 & \text{se } n > 1 \end{cases}$$

L'obiettivo è quello di ricavare la formula esplicita. In particolare si vuole arrivare a una espressione analitica esatta (**analisi esatta**), oppure si ricava una maggiorazione/minorazione a meno di costanti (**analisi asintotica**). In alcuni casi l'analisi esatta è molto complessa e ci quindi ci si accontenta di quella asintotica.

Nota importante: la funzione di complessità $T : \mathbb{N} \rightarrow \mathbb{R}^+ \cup \{0\}$, cioè ha come dominio i numeri naturali e dunque bisogna sempre accertarsi che gli argomenti di T siano interi.

Per ricavare più facilmente la formula esplicita, in alcuni casi è utile **fare delle ipotesi restrittive sui valori ammissibili di n** . In questo modo andiamo a restringere l'insieme delle taglie delle istanze trattate dall'analisi e ciò può semplificare i calcoli. Nel nostro caso possiamo considerare come ipotesi restrittiva solo istanze che hanno taglia $n = 2^k$ (dove ricordiamo $k = \lfloor (i+j)/2 \rfloor$), in questo modo possiamo riscrivere la relazione di ricorrenza come segue:

$$T(n) = \begin{cases} 0 & \text{se } n = 1 \\ 2T(n/2) + 1 & \text{se } n > 1 \end{cases}$$

Questa nuova formulazione ci permette di ricavare informazioni sull'andamento asintotico, non ci permette di concludere l'analisi visto che questa va fatta per qualsiasi n .

Dunque per $n = 2^k$ si ricava la seguente formula chiusa:

$$\begin{aligned} T(n) &= 2T(n/2) + 1 \\ &= 2^2T(n/2^2) + 2 + 1 \\ &= 2^2(2T(n/2^3) + 1) + 2 + 1 \\ &= 2^3T(n/2^3) + \underbrace{2^2 + 2 + 1}_{\text{serie geometrica}} \\ &\dots \\ &= 2^i \cdot T(n/2^i) + \sum_{j=0}^{i-1} 2^j \end{aligned}$$

dove abbiamo utilizzato la **tecnica dell'unfolding** che in sostanza prevede l'applicazione ripetuta della definizione implicita, per esempio:

$$T(n/2) = 2T\left(\frac{n/2}{2}\right) + 1.$$

Quando si ferma il processo dell'unfolding? Quando $n/2^i$ diventa il caso base, dunque si tratta di determinare il valore di i quando $n/2^i = 1$.

$$\frac{n}{2^i} = 1 \Leftrightarrow n = 2^i \Leftrightarrow i = \log_2 n$$

Sostituendo $\log_2 n$ a i si ottiene:

$$\begin{aligned}
 T(n) &= 2^{\log_2 n} \cdot T(n/2^{\log_2 n}) + \sum_{j=0}^{\log_2 n - 1} 2^j \\
 &= n \cdot T(1) + \sum_{j=0}^{\log_2 n - 1} 2^j \\
 &= 0 + \sum_{j=0}^{\log_2 n - 1} 2^j \\
 &= n - 1
 \end{aligned}$$

dove abbiamo utilizzato la formula chiusa della serie geometrica troncata di ragione $a \in \mathbb{C} - \{0, 1\}$:

$$\sum_{j=0}^{k-1} a^j = \frac{a^k - 1}{a - 1} \quad (2.1)$$

Riassumendo: abbiamo trovato che $T(n) = n - 1$ per $n = 2^k$.

Verifica di una formula chiusa Ottenuta una formula chiusa con qualche metodo, possiamo verificare se il procedimento utilizzato è corretto sfruttando un processo di verifica. Vediamo l'applicazione di tale processo sulla ricorrenza semplificata associata a MAX.

$$T(n) = \begin{cases} 0 & \text{se } n = 1 \\ 2T(n/2) + 1 & \text{se } n > 1 \end{cases}$$

Dimostriamo induttivamente che $T(n) = n - 1$, limitandoci a valori del parametro $n = 2^k$:

[B] Se $n = 1$ si ha che $T(n) = (n - 1)|_{n=1} = 0$

[HP] Sia $T(m) = m - 1$ per valori di m potenze di due e minori di n , con $n > 1$

[TH] Poiché per $n > 1$ si ha che $T(n) = 2T(n/2) + 1$ e inoltre $n/2 < n$ è anch'essa una potenza di due, posso usare l'ipotesi induttiva.

Si ottiene che $T(n) = 2((n/2) - 1) + 1 = n - 1$.

Possiamo concludere che la formula chiusa ottenuta è corretta.

2.4 Induzione parametrica

Nella sottosezione precedente, nell'analisi della complessità dell'algoritmo MAX, abbiamo ottenuto risolto la ricorrenza imponendo dei vincoli sull'argomento n . Risolvere esattamente la ricorrenza per valori "speciali" del parametro permette di farsi un'idea (detta **guess**) dell'andamento asintotico nel caso generale. Partendo dalla guess, possiamo ricavare l'andamento generale (per qualsiasi valore dell'argomento) sfruttando opportune tecniche.

L'**induzione parametrica** è una tecnica che permette di "simulare" la verifica per induzione (paragrafo 2.3.2) su una formula chiusa con **costanti parametriche** non fissate a priori. L'idea è quella di collezionare vincoli sui parametri che, se verificati contemporaneamente, portano a termine correttamente la verifica.

Nota che l'induzione parametrica viene utilizzata per ottenere limitazioni superiori/inferiori e non formule esatte. Per esempio, se si ipotizza che una certa funzione abbia un andamento

limitato superiormente da una funzione lineare, si può dimostrare che esistono delle costanti $c, d \in \mathbb{R}$ per cui $T(n) \leq cn + d$, ovvero che $T(n) = O(n)$.

Al termine del processo di induzione parametrica

1. se l'intersezione dei vincoli **non corrisponde all'insieme vuoto**, vorrà dire che esistono valori con cui istanziare i parametri in modo che l'induzione parametrica diventi una vera e propria prova induttiva della correttezza del guess;
2. se invece l'intersezione dei vincoli collezionati conduce ad un insieme vuoto di valori, allora il metodo di prova dell'induzione parametrica **ha fallito**. Ciò accade quando:
 - (a) la guess iniziale non è corretta;
 - (b) la guess iniziale può essere corretta ma la tecnica non è abbastanza potente.

Riassumendo:

Guess scorretta $\implies$ *Fallimento*

Fallimento $\not\Rightarrow$ *Guess scorretta*

2.4.1 Applicazione dell'induzione parametrica alla ricorrenza generale di MAX

Applichiamo il metodo dell'induzione parametrica alla ricorrenza generale di MAX.

$$T(n) = \begin{cases} 0 & \text{se } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + 1 & \text{se } n > 1 \end{cases}$$

Formuliamo la seguente *guess*

$$\exists c, d : \forall n > 0 : T(n) \leq cn + d$$

ovvero supponiamo che l'algoritmo del MAX sia lineare non solo per $n = 2^k$ ma per qualsiasi valore di n . Ora simuliamo il ragionamento induttivo per ricavare i vincoli sulle variabili c, d :

$$[B] \quad T(1) = 0 \leq cn + d|_{n=1} = c + d \Leftrightarrow c + d \geq 0 \text{ (primo vincolo)}$$

[HP] $\Rightarrow$ [TH] Applico la definizione implicita per ottenere valori del parametro indipendente $< n$ a cui poter applicare **HP** :

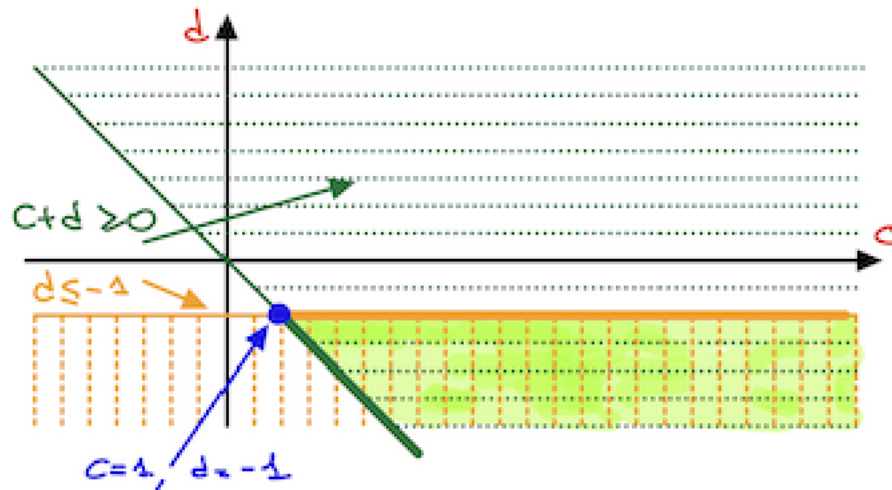
$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\text{applicando HP} \\ &\leq (c(\lfloor n/2 \rfloor) + d) + (c(\lceil n/2 \rceil) + d) + 1 \\ &= cn + (2d + 1) \end{aligned}$$

Per completare **HP** $\Rightarrow$ **TH**, bisogna imporre una disequazione per determinare una condizione sufficiente sui valori dei parametri che permetta di ottenere la tesi induttiva. Dunque si tratta di verificare per quali valori di c, d è verificata la seguente disequazione

$$cn + (2d + 1) \stackrel{?}{\leq} cn + d \Leftrightarrow d \leq -1 \text{ (secondo vincolo)}$$

Ora che abbiamo ottenuto tutti i vincoli, risolviamo il sistema con i vincoli trovati.

$$\begin{cases} c + d \geq 0 \\ d \leq -1 \end{cases}$$



Dalla soluzione grafica notiamo che l'insieme delle soluzioni del sistema non è vuoto (infinito in questo caso). Siccome stiamo provando una limitazione superiore, ha senso selezionare la coppia di valori ammissibili di parametri c e d con il valore di c più piccolo possibile, in quanto questa coppia fornirà la migliore costante del termine di ordine superiore di $T(n)$. Quindi selezioniamo $c = 1$ e di conseguenza $d = -1$; infine possiamo concludere che

$$T(n) \leq cn + d = n - 1.$$

Come abbiamo anticipato l'induzione parametrica **non è infallibile**. Infatti, proviamo ad utilizzare la *guess*

$$T(n) \leq cn$$

che è vera per $c \geq 1$. Proviamo ad applicare la tecnica:

$$[B] \quad T(1) = 0 \leq cn|_{n=1} \Leftrightarrow c \geq 0 \text{ (primo vincolo)}$$

[HP] $\Rightarrow$ [TH] Applichiamo la definizione implicita e **HP**:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$$

applicando **HP**

$$\begin{aligned} &\leq (c(\lfloor n/2 \rfloor)) + (c(\lceil n/2 \rceil)) + 1 \\ &= cn + 1 \end{aligned}$$

Otteniamo una disequazione

$$cn + 1 \stackrel{?}{\leq} cn$$

che non è mai verificata e quindi l'insieme individuato dai vincoli è vuoto.

Quante variabili conviene utilizzare quando si vuole sfruttare l'induzione parametrica? In generale conviene partire dalla *guess* più semplice ($T(n) \leq cn$ nel caso lineare) e, se l'induzione parametrica fallisce, si passa ad una *guess* più dettagliata ($T(n) \leq cn + d$ nel caso lineare).

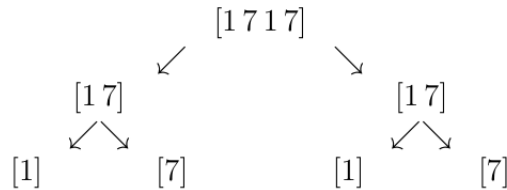
2.5 Studio dell'albero della ricorrenza

Si tratta di un metodo alternativo per studiare la complessità di un algoritmo D&C al caso peggiore. Possiamo associare alla ricorrenza che stiamo studiando l'**albero della ricorrenza**: tale albero ha la stessa forma dell'albero delle chiamate della peggiore istanza di taglia n . Ad ogni nodo dell'albero:

- se il nodo è **interno**: associamo taglia della sotto-istanza relativa (per convenzione viene scritta dentro al nodo stesso) e il costo complessivo per l'esecuzione degli algoritmi A_D e A_C sull'istanza corrisponde (per convenzione viene riportata di fianco al nodo);
- se il nodo è una **foglia**: associamo la taglia di base e il costo della risoluzione diretta.

Il costo cumulativo del divide e del conquer per una istanza non di base e il costo della risoluzione diretta per un'istanza di base viene chiamato **lavoro (work)** associato a quell'istanza. Il valore di $T(n)$ può essere ottenuto come somma di tutti i lavori associati ai nodi dell'albero.

Albero della ricorsione



Albero della ricorrenza

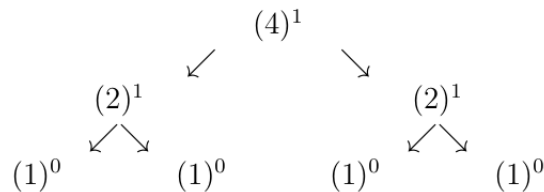


Figura 2.2: Albero della ricorsione e della ricorrenza dell'algoritmo MAX per $n = 4$. $T(4) = 3$.

2.5.1 L'albero delle chiamate di MAX

Nel caso dell'algoritmo di MAX:

- Per $n = 2^k$ l'albero della ricorrenza è
 - binario completo¹;
 - ha $n = 2^k$ foglie (che hanno lavoro nullo).

Quindi ha $n - 1$ nodi interi (che hanno lavoro = 1 per nodo). Dunque si ottiene:

$$T(n) = n - 1 \quad \text{per } n = 2^k.$$

- Per n generico l'albero della ricorrenza è
 - binario proprio²;
 - ci sono esattamente n foglie (che hanno lavoro nullo).

Si dimostra che un albero pieno con n foglie ha sempre $n - 1$ nodi interni (che hanno lavoro = 1 per nodo). Dunque si ottiene:

$$T(n) = n - 1 \quad \forall n.$$

¹Un albero binario è completo se è triangolare.

²Un albero proprio o pieno è un albero in cui tutti i nodi interni hanno due figli.

2.6 Formula generale di risoluzione di ricorrenze associate ad algoritmi Divide and Conquer

In questa sezione utilizziamo la tecnica dell'analisi dell'albero delle chiamate per derivare una formula generale che può essere applicata per ottenere l'espressione analitica (esatta) di ricorrenze associate ad algoritmi D&C appartenenti a una famiglia molto generale.

Innanzitutto definiamo:

- $s(n), f(n) : \mathbb{N} \rightarrow \mathbb{N}$ due funzioni aventi dominio e codominio l'insieme dei numeri naturali, entrambe **non decrescenti**.
 - La funzione $s(n)$ rappresenta in maniera informale il numero di sotto-istanze generate³ da un'istanza (non di base) di taglia n .
 - La funzione $f(n)$ rappresenta la taglia delle sotto-istanze generate⁴ da un'istanza (non di base) di taglia n . Tale funzione deve essere una **contrazione**, ovvero si deve avere $f(n) < n$, per tutti i valori di taglia (non di base) n .
- $w(n) : \mathbb{N} \rightarrow \mathbb{R}^+ \cup \{0\}$ funzione non decrescente. Si tratta della funzione che rappresenta il lavoro (divide + conquer) associato ad un'istanza (non di base) di taglia n . Nota che il costo associato alle istruzioni eseguite non è per forza un numero intero.

Possiamo riscrivere il *metacodice* di un generico algoritmo D&C come segue:

```

1 D&C(i) {i ∈ I}
2   n ← |i|
3
4   [B]: if (n ≤ n0) then *risolvi direttamente*
5         k ← s(n)
6
7   [D]: ⟨i1, ..., ik⟩ ← AD(i)
8         // si generano sempre s(n) istanze tutte di taglia f(n)
9
10  [R]: for j ← 1 to k do sj ← D&C(ij)
11
12  [C]: return AC(s1, s2, ..., sk)
    
```

Poniamo $n = |i|$. Per scrivere la ricorrenza di associata all'algoritmo generico poniamo inoltre:

- T_0 tempo per la risoluzione diretta di un'istanza di taglia $n \leq n_0$.
- Se $A_D(i) = \langle i_1, \dots, i_k \rangle$, allora $|i_j| = f(n)$, per $1 \leq j \leq k = s(n)$.
- Per $n > n_0$, sia $w(n) = T_{A_D}(n) + T_{A_C}(n)$, ovvero $w(n)$ rappresenta il lavoro associato a un'istanza (peggiore, nel caso worst-case) di taglia n .

$$T(n) = \begin{cases} T_0 & n \leq n_0 \\ s(n) \cdot T(f(n)) + w(n) & n > n_0 \end{cases}$$

Nota che, poiché $s(n), f(n), w(n)$ sono tutte non decrescenti, anche $T(n)$ è non decrescente. Se $s(n), f(n), w(n)$ sono delle limitazioni superiori/inferiori alle quantità relative, allora anche la soluzione $T(n)$ sarà anche essa una limitazione dello stesso tipo.

³Si ipotizza che generino lo stesso numero di sotto-istanze (in generale può non essere vero).

⁴Si ipotizza che queste abbiano tutte la stessa taglia.

Esempio: applicazione a MERGESORT Dimensioniamo i parametri della ricorrenza nel caso dell'algoritmo MERGESORT nel caso in cui $n = 2^k$. Abbiamo:

- $n_0 = 1, A \ T_0 = 0$;
- $s(n) = 2; f(n) = n/2$;
- $w(n) = T_{A_D}(n) + T_{A_C}(n) = 0 + T_{\text{MERGE}}(n) = n - 1$

Dunque

$$T(n) = \begin{cases} 0 & n \leq 1 \\ 2 \cdot T(n/2) + n - 1 & n > 1 \end{cases}$$

Se consideriamo n arbitrari come facciamo? Possiamo utilizzare la formula generale per ottenere un limite superiore ponendo $f(n) = \lceil n/2 \rceil$ (sovrastima della dimensione delle sotto-istanze)

$$T(n) = \begin{cases} 0 & n \leq 1 \\ 2 \cdot T(\lceil n/2 \rceil) + n - 1 & n > 1 \end{cases}$$

oppure si può ottenere un limite superiore ponendo ponendo $f(n) = \lfloor n/2 \rfloor$ (sottostima della dimensione delle sotto-istanze).

2.6.1 Le iterate di $f(n)$

L'iterata di una funzione f consiste nell'applicazione ripetuta di f a se stessa.

$$\begin{cases} f^{(0)}(n) = n \\ f^{(l)}(n) = f(f^{(l-1)}(n)) \quad l > 0 \end{cases}$$

Le iterate forniscono le taglie delle sotto-istanze ottenute nei vari livelli di ricorsione.

$$\begin{array}{ccc} f^{(0)}(n) = n, & f^{(1)}(n) = f(n), & f^{(2)}(n) = f(f(n)), \\ \text{Radice} & \text{Figli radice} & \text{Figli dei figli della radice} \end{array}$$

Vediamo due esempi:

- Sia $n = 2^k$ e $f(n) = n/2$. Si ha che:

$$\begin{aligned} f^{(0)}(n) &= n \\ f^{(1)}(n) &= f(n) = n/2 \\ f^{(2)}(n) &= n/2^2 \\ &\vdots \\ f^{(l)}(n) &= n/2^l \end{aligned}$$

- Sia $n = 2^{2^k}$ e $f(n) = \sqrt{n}$. Si ha che:

$$\begin{aligned} f^{(0)}(n) &= n \\ f^{(1)}(n) &= f(n) = \sqrt{n} = n^{1/2} \\ f^{(2)}(n) &= (n^{1/2})^{1/2} = n^{1/2^2} \\ &\vdots \\ f^{(l)}(n) &= n^{1/2^l} \end{aligned}$$

Data una contrazione $f(n)$ e un valore n_0 , definiamo la seguente **funzione ausiliaria**:

$$f^*(n, n_0) = \max \left\{ k : f^{(k)}(n) > n_0 \right\}, \quad n > n_0.$$

la funzione ausiliaria ritorna il massimo indice di iterata k tale che la k -esima iterata di f applicata ad n risulti essere strettamente maggiore di n_0 . In altre parole $f^*(n, n_0)$ torna il livello dell'albero della ricorrenza più profondo avente istanze non di base (taglia $> n_0$). Il valore $f^*(n, n_0) + 1$ indica il livello in cui si trovano le istanze di base (le foglie dell'albero).

Nota che se $n \leq n_0$ allora $f^*(n, n_0)$ è indefinita (max di un insieme vuoto), per convenzione si pone

$$f^*(n, n_0) = -1, \quad \text{per } n \leq n_0$$

Vediamo degli esempi in cui ricaviamo la funzione ausiliaria.

- Sia $n = 2^k$ e $f(n) = n/2$, si ha che $f^{(i)}(n) = n/2^i$ e $f^*(n, n_0) = \max \left\{ k : n/2^k > n_0 \right\}$. Si tratta ora di risolvere la seguente disequazione rispetto a k :

$$n/2^k > n_0 \Rightarrow 2^k < n/n_0 \Rightarrow k < \log_2 n/n_0$$

Quindi: $f^*(n, n_0) = \log_2(n/n_0) - 1$ (il -1 perché k è intero e quindi il massimo è dato dall'uguaglianza - 1). Per $n_0 = 1$: $f^*(n, 1) = \log_2 n - 1$.

- Generalizzando, per $n = b^k$ e $f(n) = n/b$, si ha che $f^{(i)}(n) = n/b^i$ e $f^*(n, n_0) = \max \left\{ k : n/b^k > n_0 \right\}$. Da cui:

$$n/b^k > n_0 \Rightarrow b^k < n/n_0 \Rightarrow k < \log_b n/n_0$$

Quindi: $f^*(n, n_0) = \log_b(n/n_0) - 1$. Per $n_0 = 1$: $f^*(n, 1) = \log_b n - 1$.

- Sia $n = 2^{2^k}$ e $f(n) = \sqrt{n}$, si ha che $f^{(i)}(n) = n^{1/2^i}$ e $f^*(n, n_0) = \max \left\{ k : n^{1/2^k} > n_0 \right\}$. Da cui:

$$n^{1/2^k} > n_0 \Rightarrow 1/2^k \log_2 n > \log_2 n_0 \Rightarrow 2^k < \log_2 n / \log_2 n_0$$

$$\Rightarrow k < \log_2(\log_2 n / \log_2 n_0) \Rightarrow k < \log_2(\log_2 n) - \log_2(\log_2 n_0)$$

Quindi: $f^*(n, n_0) = \log_2(\log_2 n) - \log_2(\log_2 n_0) - 1$. Per $n_0 = 2$: $f^*(n, 2) = \log_2(\log_2 n) - 1$

- Generalizzando, per $n = a^{b^k}$ e $f(n) = n^{1/b}$, si ha che $f^{(i)}(n) = n^{1/b^i}$ e $f^*(n, n_0) = \max \left\{ k : n^{1/b^k} > n_0 \right\}$. Da cui:

$$n^{1/b^k} > n_0 \Rightarrow 1/b^k \log_a n > \log_a n_0 \Rightarrow b^k < \log_a n / \log_a n_0$$

$$\Rightarrow k < \log_b(\log_a n / \log_a n_0) \Rightarrow k < \log_b(\log_a n) - \log_b(\log_a n_0)$$

Quindi: $f^*(n, n_0) = \log_b(\log_a n) - \log_b(\log_a n_0) - 1$. Per $n_0 = a$: $f^*(n, a) = \log_b(\log_a n) - 1$.

2.6.2 Analisi dell'albero della ricorrenza

Studiamo l'albero della ricorrenza

$$T(n) = \begin{cases} T_0 & n \leq n_0 \\ s(n) \cdot T(f(n)) + w(n) & n > n_0 \end{cases}$$

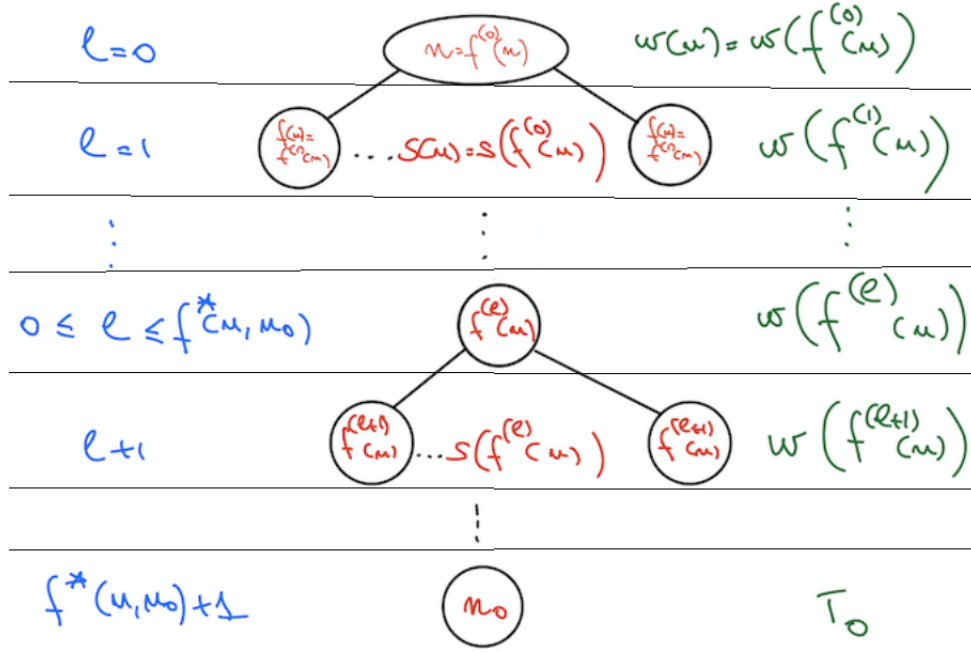


Figura 2.3: Analisi albero della ricorrenza per livello.

e analizziamo le informazioni che possiamo ricavare sul numero di sotto-istanze, sulla dimensione delle sotto-istanze e sul lavoro delle sotto-istanze appartenenti ad un certo livello dell'albero della ricorrenza.

Come abbiamo detto nella sezione 2.5 possiamo ricavarci l'espressione della $T(n)$ sommando tutti i lavori associati ai nodi dell'albero.

$$T(n) = \sum_l (\# \text{ nodi al livello } l) \cdot (\text{lavoro/nodo al livello } l).$$

Il **numero di nodi al livello** l è dato dal prodotto tra il numero di figli per nodo nei livelli precedenti:

$$s(n) \cdot s(f(n)) \cdot s(f^{(2)}(n)) \cdot \dots \cdot s(f^{(l-1)}(n)) = \prod_{j=0}^{l-1} s(f^{(j)}(n)).$$

LIVELLO	TAGLIA	# NODI/LIVELLO	LAVORO/NODO
0	$f^{(0)}(n)$	1	$w(f^{(0)}(n))$
1	$f^{(1)}(n)$	$s(f^{(0)}(n))$	$w(f^{(1)}(n))$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$0 \leq \ell \leq f^*(n, n_0)$	$f^{(\ell)}(n)$	$\prod_{j=0}^{\ell-1} s(f^{(j)}(n))$	$w(f^{(\ell)}(n))$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$f^*(n, n_0) + 1$	$\leq n_0$	$\prod_{j=0}^{f^*(n, n_0)} s(f^{(j)}(n))$	T_0

Per ricavare $T(n)$ ci basta sommare, su tutte le righe, il prodotto delle ultime due colonne (# nodi per livello e lavoro per nodo), in questo modo si ottiene il lavoro cumulativo associato al livello corrispondente. Si ottiene infine:

$$T(n) = \sum_{l=0}^{f^*(n, n_0)} \left(\left[\prod_{j=0}^{l-1} s(f^{(j)}(n)) \right] w(f^{(l)}(n)) \right) + \left[\prod_{j=0}^{f^*(n, n_0)} s(f^{(j)}(n)) \right] T_0$$

Come convenzione $\sum_{i=a}^b \dots = 0$ se $(a > b)$.

2.7 Il Master Theorem

Il **Master Theorem (MT)** è uno strumento molto potente che ci permette di ricavare l'ordine di grandezza di una vasta classe di ricorrenze (più stretta rispetto alla formula generale) senza una risoluzione esplicita. Si tratta di uno strumento molto utile che viene sfruttato nella fase di progetto e analisi di algoritmi D&C.

2.7.1 Versione semplificata del MT

Partiamo introducendo una versione semplificata del MT, successivamente vedremo la sua versione estesa. La classe di ricorrenze che consideriamo ha:

$$s(n) = a > 0, \quad f(n) = \frac{n}{b}, \quad b > 1, \quad n = b^k, \quad T_0 > 0, \quad n_0 = 1, \quad w(n) \text{ arbitraria}$$

$$T(n) = \begin{cases} T_0 & n = 1 \\ a \cdot T(\frac{n}{b}) + w(n) & n > 1 \end{cases} \quad (2.2)$$

Ogni ricorrenza appartenente alla famiglia della 2.2 ha associata una **funzione di soglia** (o **threshold function**) nella forma⁵

$$n^{\log_b a}. \quad (2.3)$$

Il MT è composto da tre differenti casi che dipendono dal confronto con la funzione di soglia e $w(n)$.

Teorema 2.1 (Master Theorem). *Per $a \geq 1$, $b > 1$, $n = b^k$, la soluzione di $T(n)$ alla ricorrenza 2.2 soddisfa:*

$$T(n) = \begin{cases} \text{Caso 1: } \Theta(n^{\log_b a}) & \text{se } \exists \varepsilon > 0 : w(n) = O(n^{\log_b a - \varepsilon}) \\ \text{Caso 2: } \Theta(n^{\log_b a} \cdot \log n) & \text{se } w(n) = \Theta(n^{\log_b a}) \\ \text{Caso 3: } \Theta(w(n)) & \text{se } \underbrace{\begin{cases} \exists \varepsilon > 0 : w(n) = \Omega(n^{\log_b a + \varepsilon}) \\ \exists c \in \mathbb{R}, 0 < c < 1 : \forall n : a \cdot w(n/b) \leq c \cdot w(n) \end{cases}}_{\text{Condizione di regolarità}} \end{cases}$$

Scrivere $w(n) = O(n^{\log_b a - \varepsilon})$ significa dire che la funzione $w(n)$ deve essere limitata superiormente da una funzione avente grado un po' più piccolo rispetto al grado della funzione di soglia, mentre la scrittura $w(n) = \Omega(n^{\log_b a + \varepsilon})$ dice che la funzione deve essere limitata inferiormente da una funzione avente grado un po' più grande della funzione di soglia.

Nota che il MT non è esaustivo: esistono ricorrenze che appartengono alla famiglia della 2.2 ma che non rientrano in nessuno dei tre casi del MT.

⁵Nota che la funzione di soglia assume solo valori interi in quanto $n = b^k$.

La **condizione di regolarità** $\exists c \in \mathbb{R}, 0 < c < 1 : \forall n : a \cdot w(n/b) \leq c \cdot w(n)$ comprende una famiglia infinita di disequazioni. La condizione di regolarità indica che $w(n)$ ha una crescita "regolare". Vedremo che si tratta di una condizione che non ci complica troppo la vita.

2.7.2 Prova del MT semplificato

Prima di passare alla prova del MT introduciamo una semplificazione che ci tornerà utile. Quando si lavora con funzioni intere positive, il concetto di ordine di grandezza può essere semplificato. Sappiamo che

$$w(n) = O(g(n)) \text{ se } \exists c > 0, \bar{n} \geq 0 \text{ tali che } w(n) \leq c \cdot g(n) \forall n > \bar{n}.$$

Se però $w(n), g(n)$ sono funzioni positive, allora possiamo estendere la maggiorazione a tutti i valori di $n > 0$ selezionando la nuova costante

$$\bar{c} = \max\{c, w(1)/g(1), w(2)/g(2) \dots, w(\bar{n})/g(\bar{n})\}.$$

Infatti per $1 \leq n \leq \bar{n}$ si ha che $w(n) = (w(n)/g(n)) \cdot g(n) \leq \bar{c}g(n)$, mentre per $n > \bar{n}$ si ha che $w(n) \leq cg(n) \leq \bar{c}g(n)$.

Analogamente, sappiamo che

$$w(n) = \Omega(g(n)) \text{ se } \exists c > 0, \hat{n} \geq 0 \text{ tali che } w(n) \geq c \cdot g(n) \forall n > \hat{n}.$$

Se però $w(n), g(n)$ sono funzioni positive, allora possiamo estendere la maggiorazione a tutti i valori di $n > 0$ selezionando la nuova costante

$$\hat{c} = \min\{c, w(1)/g(1), w(2)/g(2) \dots, w(\hat{n})/g(\hat{n})\}.$$

Infatti per $1 \leq n \leq \hat{n}$ si ha che $w(n) = (w(n)/g(n)) \cdot g(n) \geq \hat{c}g(n)$, mentre per $n > \hat{n}$ si ha che $w(n) \geq cg(n) \geq \hat{c}g(n)$.

Per la **prova del MT** applichiamo la formula generale imponendo come parametri:

$$s(n) = a, \quad f(n) = \frac{n}{b} \quad \Rightarrow \quad f^{(l)}(n) = \frac{n}{b^l}, \quad f^*(n, 2) = \log_b n - 1$$

Otteniamo dunque:

$$\begin{aligned} T(n) &= \sum_{l=0}^{\log_b n - 1} a^l \cdot w(n/b^l) + a^{\log_b n} \cdot T_0 \\ &= \sum_{l=0}^{\log_b n - 1} a^l \cdot w(n/b^l) + \underbrace{n^{\log_b a}}_{\text{Funzione di soglia}} \cdot T_0 \end{aligned}$$

abbiamo scoperto che la funzione di soglia indica il numero di foglie e che (per $T_0 > 0$) il work delle foglie è $\Theta(n^{\log_b a})$. Ci rimane da studiare la sommatoria. Consideriamo caso per caso.

- **Caso 1:** $\exists \varepsilon > 0 : w(n) = O(n^{\log_b a - \varepsilon})$.

Ciò implica che $\exists \bar{c} > 0 : \forall n > 0 : w(n) \leq \bar{c} \cdot n^{\log_b a - \varepsilon}$. Quindi:

$$\begin{aligned}
 \sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) &\leq \sum_{l=0}^{\log_b n - 1} a^l \cdot \bar{c} \left(\frac{n}{b^l} \right)^{\log_b a - \varepsilon} \\
 &= \bar{c} \sum_{l=0}^{\log_b n - 1} a^l \frac{n^{\log_b a - \varepsilon}}{b^{\log_b a^l - l\varepsilon}} \\
 &= \bar{c} \sum_{l=0}^{\log_b n - 1} a^l \frac{n^{\log_b a - \varepsilon}}{a^l \cdot b^{-l\varepsilon}} \\
 &= \bar{c} \cdot n^{\log_b a - \varepsilon} \sum_{l=0}^{\log_b n - 1} (b^\varepsilon)^l \\
 &= \bar{c} \cdot n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon \log_b a} - 1}{b^\varepsilon - 1} \\
 &= \bar{c} \cdot n^{\log_b a - \varepsilon} \cdot \frac{n^\varepsilon - 1}{b^\varepsilon - 1} \\
 &\leq \frac{\bar{c}}{b^\varepsilon - 1} \cdot n^{\log_b a}
 \end{aligned}$$

Ne consegue che:

$$\sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) = O(n^{\log_b a})$$

da cui infine:

$$T(n) = \underbrace{O(n^{\log_b a})}_{\text{Nodi interni}} + \underbrace{\Theta(n^{\log_b a})}_{\text{Foglie}} = \Theta(n^{\log_b a}).$$

Nota che se $T_0 = 0$ allora $T(n) = O(n^{\log_b a})$.

- **Caso 2:** $w(n) = \Theta(n^{\log_b a})$.

Ciò implica che $\exists \hat{c}, \bar{c} > 0 : \forall n : \hat{c} \cdot n^{\log_b a} \leq w(n) \leq \bar{c} \cdot n^{\log_b a}$.

– Consideriamo la maggiorazione $w(n) \leq \bar{c} \cdot n^{\log_b a}$:

$$\begin{aligned}
 \sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) &\leq \sum_{l=0}^{\log_b n - 1} a^l \cdot \bar{c} \left(\frac{n}{b^l} \right)^{\log_b a} \\
 &= \bar{c} \cdot \sum_{l=0}^{\log_b n - 1} a^l \cdot \frac{n^{\log_b a}}{a^l} \\
 &= \bar{c} \cdot \sum_{l=0}^{\log_b n - 1} n^{\log_b a} \\
 &= \bar{c} \cdot n^{\log_b a} \sum_{l=0}^{\log_b n - 1} 1 \\
 &= \bar{c} \cdot n^{\log_b a} \cdot \log_b n
 \end{aligned}$$

Ne consegue che:

$$\sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) = O(n^{\log_b a} \cdot \log n)$$

da cui:

$$T(n) = \underbrace{O(n^{\log_b a} \cdot \log n)}_{\text{Nodi interni}} + \underbrace{\Theta(n^{\log_b a})}_{\text{Foglie}} = O(n^{\log_b a} \cdot \log n).$$

– Consideriamo la minorazione $w(n) \geq \hat{c} \cdot n^{\log_b a}$:

$$\begin{aligned} \sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) &\geq \sum_{l=0}^{\log_b n - 1} a^l \cdot \hat{c} \left(\frac{n}{b^l} \right)^{\log_b a} \\ &= \hat{c} \cdot \sum_{l=0}^{\log_b n - 1} a^l \cdot \frac{n^{\log_b a}}{a^l} \\ &= \hat{c} \cdot \sum_{l=0}^{\log_b n - 1} n^{\log_b a} \\ &= \hat{c} \cdot n^{\log_b a} \sum_{l=0}^{\log_b n - 1} 1 \\ &= \hat{c} \cdot n^{\log_b a} \cdot \log_b n \end{aligned}$$

Ne consegue che:

$$\sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) = \Omega(n^{\log_b a} \cdot \log n)$$

da cui:

$$T(n) = \underbrace{\Omega(n^{\log_b a} \cdot \log n)}_{\text{Nodi interni}} + \underbrace{\Theta(n^{\log_b a})}_{\text{Foglie}} = \Omega(n^{\log_b a} \cdot \log n).$$

Dai due punti precedenti consegue che:

$$T(n) = O(n^{\log_b a} \cdot \log n) + \Omega(n^{\log_b a} \cdot \log n) = \Theta(n^{\log_b a} \cdot \log n)$$

- **Caso 3:** $\begin{cases} \exists \varepsilon > 0 : w(n) = \Omega(n^{\log_b a + \varepsilon}) \\ \exists c \in \mathbb{R}, 0 < c < 1 : \forall n : a \cdot w(n/b) \leq c \cdot w(n) \end{cases}$.

Dimostriamo che la condizione di regolarità implica che:

$$\forall l > 0 : \forall n : a^l \cdot w(n/b^l) \leq c^l \cdot w(n).$$

Per induzione:

$$[\mathbf{B}] : l = 0 \Rightarrow w(n) \leq w(n)$$

$$[\mathbf{HP}] : \text{Vera per } l - 1.$$

$$\begin{aligned} [\mathbf{TH}] : a^l \cdot w(n/b^l) &= a^{l-1} \cdot a \cdot w\left(\frac{n}{b^{l-1}}\right) \\ &\leq c \cdot a^{l-1} \cdot w\left(\frac{n}{b^{l-1}}\right) \text{ per la condizione di regolarità} \\ &\stackrel{\text{HP}}{\leq} c \cdot c^{l-1} \cdot w(n) \\ &= c^l \cdot w(n) \end{aligned}$$

Sfruttando questa implicazione si ottiene che:

$$\begin{aligned}
 \sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) &\leq \sum_{l=0}^{\log_b n - 1} c^l \cdot w(n) \\
 &= w(n) \cdot \sum_{l=0}^{\log_b n - 1} c^l \\
 &< w(n) \cdot \sum_{l=0}^{\infty} c^l \\
 &= \frac{w(n)}{1 - c}
 \end{aligned}$$

Quindi si ottiene:

$$T(n) = \underbrace{O(w(n))}_{\text{Nodi interni}} + \underbrace{\Theta(n^{\log_b a})}_{\text{Foglie}} = O(w(n))$$

perché $w(n) = \Omega(n^{\log_b a + \epsilon})$. Inoltre per $n > 1$ vale anche che

$$T(n) = a \cdot T(n/b) + w(n) \geq w(n)$$

perché $T(n) \geq 0 \forall n$, e quindi implica che $T(n) = \Omega(w(n))$. Possiamo infine concludere che:

$$T(n) = \Omega(w(n)).$$

2.7.3 Utilità del MT

Il MT è uno strumento molto utile che permette di capire l'andamento asintotico stretto di una ricorrenza senza la necessità di risolverla. Quando si progetta un algoritmo D&C, il primo passo è quello di trovare una proprietà di sotto-struttura. Ricavata questa, si va di conseguenza a fissare i valori dei parametri $a, b, w(n)$ della ricorrenza (facendo riferimento a 2.2). Arrivati a questo punto possiamo sfruttare il MT e verificare in quale caso ci troviamo. In base al caso possiamo trarre delle considerazioni sulla proprietà di sotto-struttura che stiamo considerando:

- se siamo nel *Caso 1*: la complessità è dominata dal numero delle foglie ($O(n^{\log_b a})$). Se vogliamo migliorare la strategia, dobbiamo ridurre il numero di sotto-istanze (ad esempio andando a diminuire a);
- se siamo nel *Caso 3*: la complessità è dominata dal lavoro $w(n)$ fatto alla radice (istanza principale). Per migliorare la strategia bisogna cercare di abbassare il lavoro $w(n) = T_{A_D}(n) + T_{A_C}(n)$;
- se siamo nel *Caso 2*: la strategia ricavata è bilanciata (non significa che sia il caso migliore). Apportare miglioramenti richiede di ricavare una nuova proprietà di sotto-struttura.

2.7.4 Versione estesa del MT

Il MT può essere applicato in casi più generali di quello che abbiamo studiato nelle sottosezioni precedenti. In particolare per ricorrenze del tipo

$$T(n) = \begin{cases} T_0 & n \leq n_0 \\ a \cdot T(\lfloor n/b \rfloor) + w(n) & n > n_0 \end{cases}, \quad T(n) = \begin{cases} T_0 & n \leq n_0 \\ a \cdot T(\lceil n/b \rceil) + w(n) & n > n_0 \end{cases}$$

possiamo applicare il MT e ricondurci agli stessi tre casi e ottenere gli stessi risultati che abbiamo trovato nel caso semplificato. Notiamo che nel caso generale n e T_0 sono arbitrari. Non vediamo la dimostrazione del MT esteso, diciamo soltanto che si dimostra che il MT è indipendente dagli arrotondamenti e dalle costanti associate al caso di base.

Per **convenzione**, se si intende studiare una ricorrenza con il MT, questa può essere scritta nella forma

$$T(n) = a \cdot T(n/b) + w(n)$$

andando ad ignorare T_0 , $\lceil \cdot \rceil$, $\lfloor \cdot \rfloor$, n_0 . Ciò è dovuto al fatto che questi parametri non influiscono sull'andamento asintotico; tuttavia la scrittura non sarebbe corretta a livello matematico (n/b dovrebbe essere intero).

2.7.5 Applicare il MT nel caso del MAX e del MERGESORT

MAX Se si ignorano gli arrotondamenti, la ricorrenza diventa

$$T(n) = 2T(n/2) + 1$$

dunque si ha

$$\begin{aligned} a &= b = 2, w(n) = 1 \\ n^{\log_b a} &= n \text{ (funzione di soglia)} \\ \Rightarrow w(n) &= O(n^{1-\varepsilon}), \text{ con } \varepsilon = 1 \end{aligned}$$

Dunque siamo nel *Caso 1* e quindi $T(n) = \Theta(n)$.

MERGESORT Se si ignorano gli arrotondamenti, la ricorrenza diventa

$$T(n) = 2T(n/2) + n - 1$$

dunque si ha

$$\begin{aligned} a &= b = 2, w(n) = n - 1 \\ n^{\log_b a} &= n \text{ (funzione di soglia)} \\ \Rightarrow w(n) &= \theta(n) \end{aligned}$$

Dunque siamo nel *Caso 2* e quindi $T(n) = \Theta(w(n)) = \Theta(n \log n)$.

2.7.6 Casi in cui la condizione di regolarità è semplice

Il *Caso 3* è il più problematico in quanto dobbiamo verificare la condizione di regolarità.

$$\exists c \in \mathbb{R}, 0 < c < 1 : \forall n : a \cdot w(n/b) \leq c \cdot w(n)$$

Tuttavia se $w(n) = n^k$ (polinomiale) e siamo nel *Caso 3*, si dimostra che la condizione di regolarità è sempre verificata. Infatti: se siamo nel *Caso 3*, deve essere che

$$w(n) = n^k = \Omega(n^{\log_b a + \varepsilon}), \quad \varepsilon > 0 \quad \Rightarrow k \geq \log_b a + \varepsilon.$$

Ma allora (dalla condizione di regolarità):

$$a \cdot w\left(\frac{n}{b}\right) = a \cdot \left(\frac{n}{b}\right)^k = \frac{a}{b^k} \cdot n^k = \frac{a}{b^k} \cdot w(n) \stackrel{k \geq \log_b a + \varepsilon}{\leq} \frac{a}{b^{\log_b a + \varepsilon}} \cdot w(n) = \frac{a}{a \cdot b^\varepsilon} \cdot w(n) = \frac{w(n)}{b^\varepsilon}$$

e quindi basta scegliere $c = 1/b^\varepsilon < 1$.

In maniera analoga si dimostra che la condizione di regolarità è verificata per funzioni "miste", come per esempio $w(n) = n^k \log_b^k n$.

Capitolo 3

Moltiplicazioni di matrici

In questo capitolo studiamo algoritmi che sono importanti applicazioni del D&C. Gli algoritmi efficienti per operazioni su matrici sono un ambito di interesse in moltissime applicazioni.

Considereremo matrici quadrate a componenti complesse $A \in {}_n\mathbb{C}_n$. Indicheremo con $rows(A)$ o con $A.rows$ il numero di righe di A e con $columns(A)$ o $A.columns$ il numero di colonne di A . Indicheremo la matrice con $A = [a_{ij}]_{1 \leq i,j \leq n}$; usando questa notazione possiamo facilmente riferirci ai blocchi (sotto-matrici) di una matrice. Il modello di costo che consideriamo è il seguente:

- costo unitario per operazioni aritmetiche tra scalari e complessi;
- taglia: esprimeremo $T(n)$ in funzione di $n = rows(A)$.

Nota che, dato che una matrice ha n^2 elementi, $T(n) = n^2$ è una complessità lineare nel numero di elementi $N = n^2$ (si può anche scrivere $T(N) = N$), ma è una complessità quadratica nella dimensione (consideriamo taglia = $rows(A) = n$). In ogni caso il numero di operazioni riportate è sempre lo stesso.

3.1 Somma e sottrazione di matrici

Definiamo la somma di matrici come

$$A, B \in {}_n\mathbb{C}_n, \quad C = A \pm B, \quad c_{ij} = a_{ij} \pm b_{ij}$$

Scriviamo il metacodice che realizza l'algoritmo della somma **basandosi sulla definizione**.

```
1 SUM(A,B):  
2   n ← rows(A)  
3   for i ← 1 to n do  
4     for j ← 1 to n do  
5       cij ← aij + bij  
6   return C
```

Dove è presente il vincolo statico $rows(A) = columns(A) = rows(B) = columns(B)$. Per come abbiamo scritto il metacodice, abbiamo utilizzato l'indice **row major**: la matrice viene scandita per righe. Si poteva anche utilizzare l'indice **column major** per scandire la matrice per colonne, andando ad invertire i due for della terza e quarta riga. Le prestazioni dell'algoritmo possono variare fortemente in base all'indice che si utilizza. La scelta dipende dal linguaggio di programmazione che si utilizza per implementare l'algoritmo, più precisamente dipende da come viene memorizzata la matrice in memoria: per righe o per colonne. Nella maggior parte dei casi le matrici vengono memorizzate per riga e quindi utilizzando l'indice

row major. Utilizzando l'indice column major quando la matrice viene memorizzata per righe in memoria, causa moltissimi cache miss e le prestazioni peggiorano notevolmente¹.

Analizziamo le prestazioni dell'algoritmo. Nel caso di algoritmi iterativi, ci basta sostituire ogni for con una sommatoria (usando lo stesso indice). Si ottiene:

$$T_S(n) = \sum_{i=1}^n \left(\sum_{j=1}^n 1 \right) = \sum_{i=1}^n n = n^2.$$

Dunque la complessità è quadratica nella dimensione della matrice ma lineare nel numero degli elementi. Siccome è il meglio che si può fare, possiamo concludere che l'algoritmo è ottimo.

Per l'algoritmo SUB(A,B) si ottengono le stesse considerazioni.

3.2 Prodotto di matrici (versione iterativa)

Definiamo il prodotto di matrici come segue (prodotto riga per colonna)

$$A, B \in {}_n\mathbb{C}_n, \quad C = A \times B, \quad c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}, \quad 1 \leq i, j \leq n.$$

Per matrici rettangolari

$$A \in {}_p\mathbb{C}_q, \quad B \in {}_q\mathbb{C}_r \quad C = A \times B \in {}_p\mathbb{C}_r, \quad c_{ij} = \sum_{k=1}^q a_{ik} b_{kj}.$$

Scriviamo il metacodice che realizza l'algoritmo del prodotto **basandosi sulla definizione**.

```

1 MUL(A,B):
2   n ← rows(A)
3   for i ← 1 to n do
4     for j ← 1 to n do
5       cij ← ai1 · b1j // evito una somma
6       for k ← 2 to n do
7         cij ← cij + aik · bkj
8   return C

```

Analisi della complessità:

$$\begin{aligned}
 T_M(n) &= \sum_{i=1}^n \left(\sum_{j=1}^n \left(1 + \sum_{k=2}^n 2 \right) \right) \\
 &= \sum_{i=1}^n \left(\sum_{j=1}^n (1 + 2(n - 2 + 1)) \right) \\
 &= \sum_{i=1}^n \left(\sum_{j=1}^n (2n - 1) \right) \\
 &= \sum_{i=1}^n (n(2n - 1)) \\
 &= n^2(2n - 1) = 2n^3 - n^2
 \end{aligned}$$

¹Una lettura in cache è circa 100 volte più veloce di una in RAM.

3.3 Prodotto di matrici (algoritmo ricorsivo)

Gli algoritmi iterativi di somma e differenza che abbiamo studiato nelle precedenti sezioni sono ottimi, mentre l'algoritmo iterativo che si basa sulla definizione può essere migliorato. L'algoritmo che ad oggi ha le migliori prestazioni nel calcolo del prodotto di due matrici è l'algoritmo di Strassen. Prima di studiare tale algoritmo, andiamo a studiare un altro algoritmo che non è migliore dell'algoritmo iterativo (ha esattamente la stessa complessità) ma è ricorsivo e introduce dei concetti importanti.

Per semplificare le analisi dell'algoritmo che andiamo ad analizzare, assumiamo che $n = 2^k$. Questa assunzione non ci fa perdere di generalità, infatti se abbiamo una matrice con $n \neq 2^k$, allora ci basta aggiungere tante righe e colonne di zeri finché non si raggiunge la più piccola potenza di due più vicina. Questa tecnica prende il nome di **padding**. In particolare, ponendo $n \neq 2^k$ e chiamando n' il nuovo numero di righe e colonne della matrice, si ha che

$$n' = 2^{\lceil \log_2 n \rceil} \leq 2^{\log_2 n + 1} = 2n.$$

Ovvero possiamo ricavare direttamente il nuovo valore di righe e colonne e sappiamo inoltre che al massimo può essere uguale a $2n$.

Avendo una qualsiasi matrice con $n = 2^k > 1$, allora la matrice può essere divisa in 4 quadranti:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad \text{con } A_{ij} \in \mathbb{C}_{\frac{n}{2} \times \frac{n}{2}}.$$

Analogamente si fa con le matrici B e C .

La proprietà di sotto-struttura che sfruttiamo è la **proprietà frattale del prodotto righe per colonne**. Si ha che:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{ij} = \sum_{k=1}^2 A_{ik} B_{kj} = A_{i1} B_{1j} + A_{i2} B_{2j}.$$

In sostanza si tratti applicare la regola del prodotto riga per colonna ai blocchi di matrici. Abbiamo definito una proprietà di sotto-struttura: il prodotto $n \times n$ è stato espresso in termini di prodotto $n/2 \times n/2$.

Dimostriamo la correttezza della proprietà di sotto-struttura per C_{11} . Utilizziamo la notazione $(A_{hk})_{ij}$ per indicare il termine di indice (i, j) (con $1 \leq i, j \leq n/2$) della sotto-matrice

A_{hk} (con $1 \leq h, k, \leq 2$). Abbiamo:

$$\begin{aligned}
 (C_{11})_{ij} &= \sum_{k=1}^n a_{ik} b_{kj} \\
 &\text{spezzo la sommatoria in due sommatorie} \\
 &= \sum_{k=1}^{n/2} a_{ik} b_{kj} + \sum_{k=n/2+1}^n a_{ik} b_{kj} \\
 &\text{pongo } k = h + n/2 \\
 &= \sum_{k=1}^{n/2} a_{ik} b_{kj} + \sum_{h=1}^{n/2} a_{i(h+n/2)} b_{(h+n/2)j} \\
 &\text{sfrutto la notazione con le sotto-matrici} \\
 &= \sum_{k=1}^{n/2} (A_{11})_{ik} (B_{11})_{kj} + \sum_{h=1}^{n/2} (A_{12})_{ih} (B_{21})_{hj} \\
 &= (A_{11} B_{11})_{ij} + (A_{12} B_{21})_{ij}
 \end{aligned}$$

$$\begin{aligned}
 (C_{12})_{ij} &= \sum_{k=1}^n a_{ik} b_{k(j+n/2)} \\
 &= \sum_{k=1}^{n/2} a_{ik} b_{k(j+n/2)} + \sum_{k=n/2+1}^n a_{ik} b_{k(j+n/2)} \\
 &\text{pongo } k = h + n/2 \\
 &= \sum_{k=1}^{n/2} a_{ik} b_{k(j+n/2)} + \sum_{h=1}^{n/2} a_{i(h+n/2)} b_{(h+n/2)(j+n/2)} \\
 &= \sum_{k=1}^{n/2} (A_{11})_{ik} (B_{12})_{kj} + \sum_{h=1}^{n/2} (A_{12})_{ih} (B_{22})_{hj} \\
 &= (A_{11} B_{12})_{ij} + (A_{12} B_{22})_{ij}
 \end{aligned}$$

$$\begin{aligned}
 (C_{21})_{ij} &= \sum_{k=1}^n a_{(i+n/2)k} b_{kj} \\
 &= \sum_{k=1}^{n/2} a_{(i+n/2)k} b_{kj} + \sum_{k=n/2+1}^n a_{(i+n/2)k} b_{kj} \\
 &\text{pongo } k = h + n/2 \\
 &= \sum_{k=1}^{n/2} a_{(i+n/2)k} b_{kj} + \sum_{h=1}^{n/2} a_{(i+n/2)(h+n/2)} b_{(h+n/2)j} \\
 &= \sum_{k=1}^{n/2} (A_{21})_{ik} (B_{11})_{kj} + \sum_{h=1}^{n/2} (A_{22})_{ih} (B_{21})_{hj} \\
 &= (A_{21} B_{11})_{ij} + (A_{22} B_{21})_{ij}
 \end{aligned}$$

$$\begin{aligned}
 (C_{22})_{ij} &= \sum_{k=1}^n a_{(i+n/2)k} b_{k(j+n/2)} \\
 &= \sum_{k=1}^{n/2} a_{(i+n/2)k} b_{k(j+n/2)} + \sum_{k=n/2+1}^n a_{(i+n/2)k} b_{k(j+n/2)} \\
 &\text{pongo } k = h + n/2 \\
 &= \sum_{k=1}^{n/2} a_{(i+n/2)k} b_{k(j+n/2)} + \sum_{h=1}^{n/2} a_{(i+n/2)(h+n/2)} b_{(h+n/2)(j+n/2)} \\
 &= \sum_{k=1}^{n/2} (A_{21})_{ik} (B_{12})_{kj} + \sum_{h=1}^{n/2} (A_{22})_{ih} (B_{22})_{hj} \\
 &= (A_{11}B_{12})_{ij} + (A_{12}B_{22})_{ij}
 \end{aligned}$$

Il caso base:

$$n = 1 : \quad A = [a_{11}], B = [b_{11}] \rightarrow C = [a_{11} \cdot b_{11}].$$

Possiamo scrivere l'algoritmo ricorsivo che calcola il prodotto tra due matrici:

```

1 RMUL(A,B) {A,B ∈ nCn}, n = 2k
2   n ← rows(A)
3   [B] if (n = 1) then
4       return [a11 b11]
5   [D] * Sia A =  $\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ , B =  $\begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$  *
6       for i ← 1 to 2 do
7   [R+C]   for j ← 1 to 2 do
8           Cij ← SUM(RMUL(Ai1, Bj1), RMUL(Ai2, Bj2))
9   return C
    
```

Nota che la divisione in quadrati della matrice non viene fatta ricopiando la matrice ma si sfruttano gli indici come abbiamo visto per l'algoritmo MAX. In particolare si sfruttano 3 indici: di riga i , di colonna j e una variabile che indica la dimensione della sotto-matrice n' .

L'algoritmo RMUL è corretto in quanto abbiamo dimostrato che la proprietà di sotto-struttura è corretta (non dimostriamo anche l'algoritmo). La relazione di ricorrenza è:

$$T(n) = \begin{cases} 1 & n = 1 \\ 8T(n/2) + n^2 & n > 1 \end{cases}$$

dove $s(n) = 8$ in quanto si fanno $2 \cdot 2 \cdot 2 = 8$ chiamate ricorsive (basta guardare i due for annidati); $w(n) = n^2$ in quanto si sommano due matrici $\frac{n}{2} \times \frac{n}{2}$ per quattro volte, quindi (basandosi sulla complessità dell'algoritmo SUM): $w(n) = 4 \cdot n^2/4 = n^2$.

Studiando la ricorrenza con il MT si ottiene che:

- la funzione di soglia è $n^{\log_b a} = n^{\log_2 8} = n^3$;
- $w(n) = n^2 = O(n^2)$;
- ponendo per esempio $\varepsilon = 1$, abbiamo che $w(n) = O(n^{3-\varepsilon})$ e quindi siamo nel Caso 1 e quindi possiamo concludere che

$$T(n) = \Theta(n^3).$$

Come avevamo anticipato, la complessità dell'algoritmo ricorsivo non è migliore di quella dell'algoritmo iterativo. Non solo hanno la stessa complessità (asintotica), ma si dimostra (per induzione sfruttando la guess) che anche nel caso ricorsivo si ha che $T(n) = 2n^3 - n^2$ esattamente come nel caso iterativo.

Differenze tra la versione iterativa e quella ricorsiva Siamo giunti alla conclusione che entrambi gli algoritmi hanno esattamente la stessa complessità temporale. La domanda che ci poniamo è: quale dei conviene usare? Uno è meglio dell'altro? Si è indotti a pensare che in generale un algoritmo iterativo sia migliore di uno ricorsivo. Tuttavia possiamo notare che in questo caso, nell'algoritmo iterativo c'è il problema degli elevati cache miss dovuti allo scorrere di una colonna (o riga, dipende dal linguaggio) della matrice. Dall'altra parte il problema dell'algoritmo ricorsivo è la gestione delle chiamate. Un fenomeno può dominare sull'altro a seconda del sistema su cui si fa girare l'algoritmo. Quindi non c'è una risposta, bisogna effettuare dei test sperimentali. Spesso capita che la miglior soluzione sia un ibrido: un algoritmo che sfrutta la ricorsione fino ad un certo punto e poi diventa iterativo.

Detto ciò, la domanda sorge spontanea: cosa abbiamo ottenuto con l'aver scritto l'algoritmo sfruttando la ricorsione? L'algoritmo ricorsivo effettua le stesse operazioni di quello iterativo ma lo fa in ordine diverso. Non abbiamo migliorato le prestazioni ma abbiamo individuato una **proprietà di sotto-struttura migliorabile**.

Sfruttiamo il MT per capire dove si trova l'inefficienza dell'algoritmo ricorsivo Come abbiamo visto, rientriamo nel *Caso 1* dell'MT e abbiamo visto nel capitolo precedente che se rientriamo in questo caso significa che la complessità è dominata dal numero delle foglie. Dunque se vogliamo migliorare l'algoritmo che calcola il prodotto in maniera ricorsiva dobbiamo diminuire il numero di foglie che nel nostro caso si traduce in diminuire il numero di chiamate ricorsive (attualmente generiamo 8 sotto-istanze ad ogni chiamata).

L'idea quindi è quella di diminuire il numero di sotto-istanze senza aumentare troppo il lavoro nella fase di conquer. L'algoritmo di Strassen che studieremo nella prossima sezione si basa proprio su questa idea.

3.4 L'algoritmo di Strassen

3.4.1 L'idea

L'algoritmo di Strassen realizza il prodotto tra matrici quadrate, ha prestazioni migliori rispetto a quelle dell'algoritmo basato sulla definizione. L'algoritmo riduce a 7 le sotto-istanze generate ad ogni nodo; le istanze $A_i, B_i \in \frac{n}{2} \times \frac{n}{2}$ con $1 \leq i \leq 7$ si ottengono come combinazioni lineari dei quadranti di A e B (somme o differenze di due quadranti di A e B). Nella fase di recurse si calcolano ricorsivamente i 7 prodotti $P_i = A_i \times B_i$ con $1 \leq i \leq 7$. Infine, nella fase di conquer, si calcolano i quattro quadranti di $C = A \times B$ come combinazioni lineari dei 7 prodotti P_i .

Vedremo che le combinazioni lineari per ottenere le sotto-istanze (A_i, B_i) $1 \leq i \leq 7$ e i quattro quadranti C_{ij} $1 \leq i, j \leq 2$ ottenuti dai 7 prodotti ricorsivi P_i $1 \leq i \leq 7$, richiedono 18 somme e sottrazioni di matrici $\frac{n}{2} \times \frac{n}{2}$. In questo aumenta il lavoro nelle fasi Divide e Conquer che influisce sulla complessità solo sulla costante moltiplicativa, l'andamento asintotico rimane $\Theta(n^2)$. Infatti facendo 18 somme e sottrazioni di matrici $\frac{n}{2} \times \frac{n}{2}$ si ha che: $w(n) = 18(\frac{n}{2})^2 = \frac{9}{2}n^2$.

La ricorrenza (che studiamo con il MT) ottenuta è:

$$T_S(n) = 7 \cdot T_S\left(\frac{n}{2}\right) + \frac{9}{2}n^2.$$

Siccome la funzione di soglia è $n^{\log_b a} = n^{\log_2 7} \approx n^{2.81}$ e $w(n) = \frac{9}{2}n^2 = O(n^{2.81-\epsilon})$ è verificata per esempio per $\epsilon = 0.8$, allora siamo nel *Caso 1* e possiamo concludere che $T(n) = \Theta(n^{2.81})$. Riducendo il numero di chiamate ricorsive abbiamo ottenuto un complessità asintotica migliore di $\Theta(n^3)$. Notiamo inoltre che grazie al MT siamo riusciti a valutare la complessità dell'algoritmo di Strassen senza vederne i dettagli (un prova della potenza del MT e della sua utilità in fase di progettazione).

3.4.2 L'algoritmo nel dettaglio

Veniamo all'algoritmo. La fase di **Divide** calcola le sotto-matrici A_i, B_i con $1 \leq i \leq 7$ di dimensione $\frac{n}{2} \times \frac{n}{2}$.

$$\begin{aligned} A_1 &= (A_{12} - A_{22}) & B_1 &= (B_{21} + B_{22}) \\ A_2 &= (A_{11} + A_{22}) & B_2 &= (B_{11} + B_{22}) \\ A_3 &= (A_{11} - A_{21}) & B_3 &= (B_{11} + B_{12}) \\ A_4 &= (A_{11} + A_{12}) & B_4 &= B_{22} \\ A_5 &= A_{11} & B_5 &= (B_{12} - B_{22}) \\ A_6 &= A_{22} & B_6 &= (B_{21} - B_{11}) \\ A_7 &= (A_{21} + A_{22}) & B_7 &= B_{11} \end{aligned}$$

Notiamo che in questa fase vengono effettuate 10 operazioni di somme/sottrazioni e che non c'è alcuna simmetria tra di queste. Nella fase di **Recurse** vengono calcolati i prodotti

$$P_i = A_i \times B_i \quad 1 \leq i \leq 7.$$

Infine nella fase di **conquer** si ricavano i quattro quadranti della matrice $C = A \times B$ mediante combinazione lineare dei prodotti P_i

$$\begin{aligned} C_{11} &= P_1 + P_2 - P_4 + P_6 \\ C_{12} &= P_4 + P_5 \\ C_{21} &= P_6 + P_7 \\ C_{22} &= P_2 - P_3 + P_5 - P_7 \end{aligned}$$

Possiamo scrivere il metacodice dell'algoritmo di Strassen:

```

1  SMUL(A,B) {A,B ∈  $_n\mathbb{C}_n$ },  $n = 2^k$ 
2      n ← rows(n)
3
4  [B] if (n = 1) then
5      return [a11 b11]
6
7  [D]  $\langle A_1, \dots, A_7, B_1, \dots, B_7 \rangle \leftarrow A_D(A, B)$ 
8
9  [R] for i ← 1 to 7 do
10      $P_i \leftarrow \text{SMUL}(A_i, B_i)$ 
11
12  [C]  $C \leftarrow A_C(P_1, \dots, P_7)$ 
13
14     return C
15
16
17  AD(A,B)
18     * Ricavo le sotto-matrici  $A_i, B_i$   $1 \leq i \leq 7$  *
19     return  $\langle A_1, \dots, A_7, B_1, \dots, B_7 \rangle$ 
    
```

```

20
21  $A_C(P_1, \dots, P_7)$ 
22     * Ricavo le 4 sotto-matrici  $C_{ij}$   $1 \leq i, j \leq 2$  *
23     return C
    
```

Non è chiaro il processo che ha portato Strassen a ricavare una tale proprietà di sotto-struttura, tuttavia possiamo andare a dimostrarne la **correttezza**. Proviamo che il calcolo delle 4 sotto-matrici C_{ij} è corretto:

$$\begin{aligned}
 C_{12} &= P_4 + P_5 \\
 &= A_4 \times B_4 + A_5 \times B_5 \\
 &= (A_{11} + A_{12})B_{22} + (B_{12} - B_{22})A_{11} \\
 &= A_{11}B_{22} + A_{12}B_{22} + A_{11}B_{12} - A_{11}B_{22} \\
 &= A_{11}B_{12} + A_{12}B_{22}
 \end{aligned}$$

Similmente si dimostrano anche i rimanenti blocchi della matrice C . Passiamo ora a studiare la **complessità** dell'algoritmo. L'algoritmo $A_D(A, B)$ effettua 10 somme/sottrazioni di matrici $\frac{n}{2} \times \frac{n}{2}$ e quindi $T_{A_D}(n) = 10(n/2)^2$; l'algoritmo $A_C(P_1, \dots, P_7)$ effettua 8 somme/sottrazioni tra matrici $\frac{n}{2} \times \frac{n}{2}$ e quindi $T_{A_C}(n) = 8(n/2)^2$. Dunque il lavoro in totale è dato da

$$w(n) = T_{A_D}(n) + T_{A_C}(n) = 8 \left(\frac{n}{2}\right)^2 + 10 \left(\frac{n}{2}\right)^2 = \frac{9}{2}n^2.$$

Sappiamo inoltre che $s(n) = 7$ e $f(n) = n/2$ (taglia delle sotto-matrici), quindi possiamo scrivere la relazione ricorrenza

$$T_S(n) = \begin{cases} 1 & n = 1 \\ 7 \cdot T_S(\frac{n}{2}) + \frac{9}{2}n^2 & n > 1 \end{cases}.$$

Per ricavare l'**ordine di grandezza** sfruttiamo il MT:

$$\begin{aligned}
 n^{\log_b a} &= n^{\log_2 7} \\
 w(n) &= \frac{9}{2}n^2 = O(n^{\log_2 7 - \epsilon}), \quad \text{vera per esempio per } \epsilon = 0.8 \\
 &\Rightarrow \text{Siamo nel Caso 1} \Rightarrow T(n) = \Theta(n^{\log_2 7})
 \end{aligned}$$

L'algoritmo di Strassen nella pratica Come abbiamo visto l'algoritmo di Strassen rientra nel *Caso 1* del MT. Ciò significa che la complessità è dominata dal lavoro svolto nelle foglie e sappiamo quindi dove possiamo agire per poter migliorare le prestazioni. Nella pratica l'algoritmo di Strassen è difficile da implementare a causa della generazione di matrici temporanee: creare spazio temporaneo (anche piccolo) in algoritmi ricorsivi è molto costoso. L'algoritmo di Strassen non è più l'algoritmo che calcola il prodotto tra due matrici con prestazioni temporali migliori. Ad oggi esiste un algoritmo (sviluppato da Jean-François Le Gall nel 2014) che ha complessità $\Theta(n^{2.372})$, tuttavia non esiste nessuna implementazione di tale algoritmo in quanto ha delle costanti moltiplicative altissime che in pratica lo rendono inutilizzabile. L'algoritmo di Strassen viene utilizzato nella pratica per calcolare il prodotto di matrici di grossa taglia.

3.4.3 Analisi dettagliata

Risolviamo la relazione di ricorrenza associata all'algoritmo di Strassen sfruttando la formula generale. Abbiamo:

$$\begin{aligned} s(n) &= 7, \quad f(n) = \frac{n}{2}, \quad w(n) = \frac{9}{2}n^2 \\ f^{(l)}(n) &= \frac{n}{2^l}, \quad f^*(n, 1) = \log_2 n - 1 \\ \prod_{k=0}^{l-1} s(f^{(k)}(n)) &= 7^l, \quad w(f^{(l)}(n)) = \frac{9}{2} \frac{n^2}{4^l} \end{aligned}$$

dunque:

$$\begin{aligned} T_S(n) &= \sum_{l=0}^{\log_2 n - 1} 7^l \frac{9}{2} \frac{n^2}{4^l} + 7^{\log_2 n} \cdot 1 \\ &= \frac{9}{2} n^2 \sum_{l=0}^{\log_2 n - 1} \left(\frac{7}{4}\right)^l + n^{\log_2 7} \\ &= \frac{9}{2} n^2 \frac{(7/4)^{\log_2 n} - 1}{7/4 - 1} + n^{\log_2 7} \\ &= \frac{9}{2} n^2 \frac{n^{\log_2 7 - \log_2 4} - 1}{3/4} + n^{\log_2 7} \\ &= 6n^2 \left(\frac{n^{\log_2 7}}{n^2} - 1 \right) + n^{\log_2 7} \\ &= 6n^{\log_2 7} + n^{\log_2 7} - 6n^2 \\ &= 7n^{\log_2 7} - 6n^2 \end{aligned}$$

L'algoritmo che si basa sulla definizione aveva $T_{\text{MUL}}(n) = 2n^3 - n^2$ quindi la costante del termine di ordine superiore di $T_S(n)$ è 3.5 volte più grande di quella di $T_{\text{MUL}}(n)$. Il primo valore $n = 2^k$ per cui

$$T_S(n) = 7n^{\log_2 7} - 6n^2 < 2n^3 - n^2$$

è $n = 1024 = 2^{10}$ (matrici che hanno $2^{10} \cdot 2^{10} = 2^{20}$ elementi). Dunque l'algoritmo di Strassen ha prestazioni migliori di quello basato sulla definizione per matrici molto grandi².

3.5 Matrici rettangolari

Nelle sezioni precedenti abbiamo studiato due algoritmi che calcolano il prodotto di matrici quadrate. Come facciamo se le matrici sono rettangolari? Esistono diverse soluzioni per risolvere tale problema, una di queste è quella di usare la proprietà frattale e il padding. Per esempio:

$$m \begin{bmatrix} & n \end{bmatrix} \times \begin{bmatrix} q \\ & \end{bmatrix} n = {}_m C_q$$

²Nota che non stiamo considerando i costi che si porta dietro la ricorsione e nemmeno i costi dovuti alla creazione di dati temporanei. Quindi nella pratica non è competitivo nemmeno per valori di $n > 1024$.

Poniamo che $m < q < n$. L'idea è quella di trasformare q e n nel più piccolo multiplo di m (dove m in generale è la dimensione più piccola delle 3).

$$n \rightarrow n' = km$$

$$q \rightarrow q' = km$$

Per raggiungere le nuove dimensioni n' , q' si aggiungono colonne e righe di zeri. In questo modo è possibile dividere le due matrici in blocchi di sotto-matrici di dimensione $m \times m$ ed applicare la proprietà frattale per ottenere i blocchi della matrice prodotto C .

3.6 Ibridazione

L'ibridazione è uno strumento che ci permette di migliorare le costanti moltiplicative di un algoritmo D&C andandolo a combinare con un algoritmo che può avere una complessità asintotica peggiore ma con costanti moltiplicative più basse. In generale chiamiamo A_1 l'algoritmo D&C, A_2 un altro algoritmo arbitrario, e poniamo che $T_{A_1}(n) = o(T_{A_2}(n))$ (cioè A_1 è asintoticamente migliore di A_2) e che T_{A_2} abbia costanti moltiplicative migliori rispetto al termine di ordine superiore di T_{A_1} . Nel nostro caso abbiamo $A_1 = \text{SMUL}$ ($T_S(n) \approx 7n^{\log_2 7}$), $A_2 = \text{MUL}$ ($T_M(n) \approx 2n^3$). Abbiamo visto nelle sezioni precedenti che $T_S(n) < T_M(n)$ per $n \geq 1024$. Il nostro obiettivo è quello di combinare questi due algoritmi in maniera tale da ottenere un algoritmo che calcoli il prodotto tra due matrici che abbia prestazioni migliori di quelle dell'algoritmo di Strassen.

Una **prima idea** (banale) per creare un algoritmo ibrido è la seguente:

```

1 STUPID_HYBRID(A,B)
2   n ← rows(A)
3   if n < 1024 then
4     return MUL(A,B)
5   else
6     return SMUL(A,B)

```

Usando MUL per taglie inferiori di 1024 e SMUL per taglie superiori non risolve il nostro problema: la relazione di ricorrenza che si ottiene ha le stesse costanti moltiplicative di T_S per $n \geq 1024$ mentre noi stiamo cercando un miglioramento delle prestazioni per ogni n .

Per ottenere un miglioramento per ogni n possiamo modificare il codice dell'algoritmo D&C A_1 (SMUL) e usare A_2 (MUL) come caso base per $n \leq n_0$. Nota che il valore di n_0 va fissato mediante un'analisi rigorosa.

```

1 H_MUL(A,B)
2   n ← rows(A)
3
4   if n ≤ n0 then
5     return MUL(A,B)
6
7   // Il resto e' l'algoritmo SMUL
8
9   ⟨A1, ..., A7, B1, ..., B7⟩ ← AD(A, B)
10
11  for i ← 1 to 7 do
12    Pi ← H_MUL(Ai, Bi)
13
14  C ← AC(P1, ..., P7)
15
16  return C

```

L'albero di ricorsione delle chiamate di H_MUL è identico a quello di SMUL fino al livello $f^*(n, n_0)$, mentre le foglie (livello $f^*(n, n_0) + 1$) si trovano istanze di MUL. La relazione di ricorrenza è la seguente:

$$T_{SH}(n, n_0) = \begin{cases} 2n^3 - n^2 & n \leq n_0 \\ 7 \cdot T(n/2, n_0) + \frac{9}{2}n^2 & n > n_0 \end{cases}.$$

Notiamo che la ricorrenza è parametrizzata in n_0 , attraverso l'analisi andremo a fissare tale valore in modo tale da minimizzare la costante moltiplicativa del termine di ordine superiore.

3.6.1 Analisi dettagliata

L'analisi di correttezza non è necessaria: l'algoritmo deriva dalla composizione di due algoritmi corretti e quindi possiamo concludere che anche l'algoritmo ibrido è corretto. La complessità dell'algoritmo ibrido è la stessa dell'algoritmo di Strassen, l'unica differenza sta nella costante moltiplicativa del termine di ordine superiore.

Il nostro obiettivo ora è quello di risolvere la relazione di ricorrenza e di fissare il valore di n_0 . Ricaviamo l'espressione di $T(n, n_0)$ sfruttando la formula generale. Abbiamo:

$$\begin{aligned} s(n) &= 7, \quad f(n) = \frac{n}{2}, \quad w(n) = \frac{9}{2}n^2 \\ f^{(l)}(n) &= \frac{n}{2^l}, \quad f^*(n, n_0) = \log_2 \frac{n}{n_0} - 1 \\ \prod_{k=0}^{l-1} s(f^{(k)}(n)) &= 7^l, \quad w(f^{(l)}(n)) = \frac{9}{2} \frac{n^2}{4^l} \end{aligned}$$

dunque:

$$\begin{aligned} T_{SH}(n, n_0) &= \sum_{l=0}^{\log_2 n/n_0 - 1} 7^l \frac{9}{2} \frac{n^2}{4^l} + 7^{\log_2 n/n_0} (2n_0^3 - n_0^2) \\ &= \frac{9}{2} n^2 \sum_{l=0}^{\log_2 n/n_0 - 1} \left(\frac{7}{4}\right)^l + \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (2n_0^3 - n_0^2) \\ &= \frac{9}{2} n^2 \frac{(7/4)^{\log_2 n/n_0} - 1}{7/4 - 1} + \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (2n_0^3 - n_0^2) \\ &= 6n^2 \left(\frac{n^{\log_2 7 - 2}}{n_0^{\log_2 7 - 2}} - 1 \right) + \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (2n_0^3 - n_0^2) \\ &= 6n^2 \left(\frac{n^{\log_2 7} n_0^2}{n_0^{\log_2 7} n^2} - 1 \right) + \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (2n_0^3 - n_0^2) \\ &= \frac{6n^{\log_2 7} n_0^2}{n_0^{\log_2 7}} - 6n^2 + \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (2n_0^3 - n_0^2) \\ &= \frac{6n^{\log_2 7} n_0^2}{n_0^{\log_2 7}} - 6n^2 + \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (2n_0^3 - n_0^2) \\ &= \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (6n_0^2 + 2n_0^3 - n_0^2) - 6n^2 \\ &= \frac{n^{\log_2 7}}{n_0^{\log_2 7}} (n_0^2 (2n_0 + 5)) - 6n^2 \\ &= n^{\log_2 7} \left(\frac{2n_0 + 5}{n_0^{\log_2 7 - 2}} \right) - 6n^2 \end{aligned}$$

Dall'espressione ottenuta notiamo che: se $n_0 = 1$ allora otteniamo $T_{SH} = T_S$; se $n_0 = n$ (equivale a non usare mai SMUL ma solo MUL) allora otteniamo $T_{SH} = T_M$.

Definiamo

$$g(n_0) = \frac{2n_0 + 5}{n_0^{\log_2 7 - 2}}$$

il coefficiente del termine di ordine superiore. Analizziamo $g(n_0)$ per determinare il minimo, quindi andiamo studiare il segno della derivata di $g(n_0)$ rispetto a n_0 :

$$\text{sign} \left(\frac{\partial g(n_0)}{\partial n_0} \right) = \text{sign}(2n_0 - (2n_0 + 5)(\log_2 7 - 2))$$

da cui si ottiene:

$$n_0 \geq \frac{5(\log_2 7 - 2)}{2(3 - \log_2 7)} \approx 10.48.$$

Ricordiamoci che n_0 deve essere una potenza di due (abbiamo $n = 2^k$) e quindi dobbiamo fissare n_0 a 8 oppure a 16 (le potenze di due più vicine a 10.48).

$$g(8) \approx 3.92 \quad g(16) \approx 3.94$$

dunque poniamo $n_0 = 8$ e quindi possiamo scrivere l'espressione completa di T_{SH} :

$$T_{SH}(n, 8) = 3.92 \cdot n^{\log_2 7} - 6n^2.$$

Siamo riusciti a migliorare le prestazioni dell'algoritmo di Strassen.

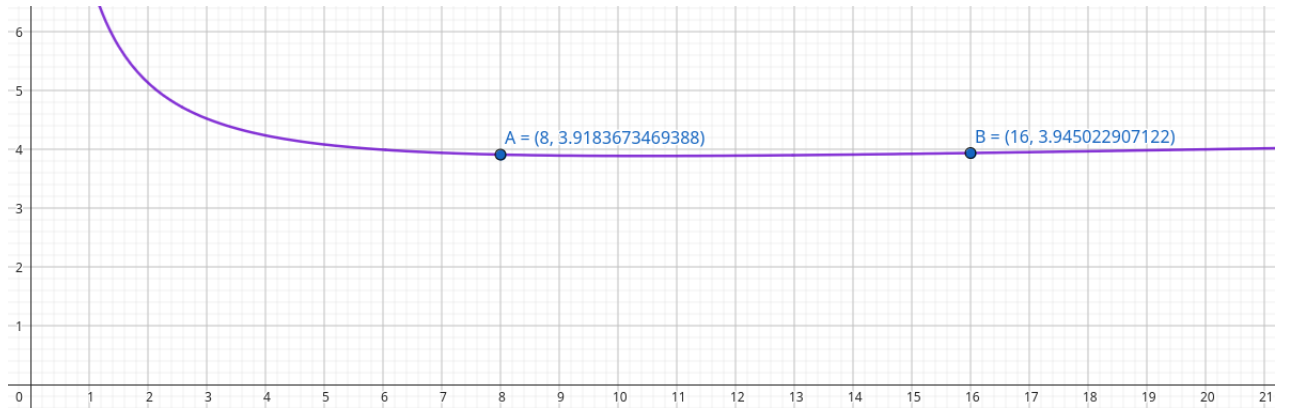


Figura 3.1: Grafico della funzione $g(n_0)$.

Capitolo 4

Polinomi e FFT

Analizziamo diversi algoritmi per operazioni tra polinomi.

4.1 Polinomi e le loro rappresentazioni

Consideriamo polinomi in campo complesso, ovvero funzioni $p : \mathbb{C} \rightarrow \mathbb{C}$ che associano ad ogni valore della variabile $x \in \mathbb{C}$ (detta **indeterminata**) una serie troncata di potenze dell'indeterminata a coefficienti complessi

$$p(x) = \sum_{i=0}^k a_i x^i.$$

Consideriamo due diverse rappresentazioni dei polinomi in quanto vedremo nelle successive sezioni che la scelta influenza significativamente le prestazioni dei vari algoritmi. Vedremo inoltre dei metodi per passare dall'una all'altra efficientemente in maniera tale da sfruttare gli algoritmi più efficienti per il calcolo dell'operazione tra polinomi che stiamo considerando.

Terminologia: per passare da una rappresentazione a coefficienti a una rappresentazione a punti si parla di **valutazione**; viceversa si parla di **interpolazione**. In particolare vedremo che esistono delle conversioni generali (verso/da base arbitraria) che però sono inefficienti; vedremo che la Discrete Fourier Transform e il suo algoritmo associato, la Fast Fourier Transform, garantiscono valutazione/interpolazione molto efficienti verso una specifica base della rappresentazione per punti.

4.1.1 Rappresentazione tramite coefficienti

Chiameremo il polinomio

$$p(x) = \sum_{i=0}^{n-1} a_i x^i = a_0 + a_1 x + a_2 x^2 + \dots + a_{n-1} x^{n-1}$$

polinomio rappresentato con n coefficienti. Definiamo inoltre:

- $a_i x^i$ viene detto **monomio** di grado i di $p(x)$;
- il **grado del polinomio** $p(x)$ è $\deg(p(x)) = \max \{i : a_i \neq 0\}$.

Un polinomio rappresentato su n coefficienti ha $\deg(p(x)) \leq n - 1$. Per convenzione chiameremo tali polinomi **polinomi di grado limitato da n** .

La **rappresentazione per coefficienti** di un polinomio di grado limitato da n , $p(x) = \sum_{i=0}^{n-1} a_i x^i$ è:

$$p(x) \equiv \vec{a} = (a_0, a_1, \dots, a_{n-1}) \in \mathbb{C}^n.$$

Precisiamo che " $\equiv$ " indica la relazione di rappresentazione tra l'oggetto matematico $p(x)$ e la sua rappresentazione $\vec{a}$. Tale rappresentazione è **estendibile**: un polinomio di grado limitato da n è anche un polinomio di grado limitato da $n' > n$ se aggiungiamo dei monomi a coefficienti nulli $a_i = 0$ per ogni $n \leq i < n'$. Dunque possiamo scrivere che:

$$p(x) \equiv \vec{a} \in \mathbb{C}^n, \quad p(x) \equiv (\vec{a} | \underbrace{\vec{0}_{n'-n}}_{\text{zero-padding}}) \in \mathbb{C}^{n'}.$$

Per esempio: $p(x) = x^2$ polinomio di grado limitato da $n = 3$:

$$p(x) \equiv (0, 0, 1) \in \mathbb{C}^3$$

visto come un polinomio di grado limitato da $n' = 4$:

$$P(x) \equiv (0, 0, 1, 0) \in \mathbb{C}^4$$

4.1.2 Rappresentazione per punti

La rappresentazione per punti è meno intuitiva rispetto alla rappresentazione per coefficienti. Vedremo che questa rappresentazione tornerà molto utile per operazioni che con la rappresentazione tramite coefficienti sono meno efficienti. La rappresentazione per punti si basa sul teorema di interpolazione:

Teorema 4.1 (Teorema di interpolazione). *Data n coppie $(x_i, y_i) \in \mathbb{C} \times \mathbb{C}$, $0 \leq i \leq n-1$, con $x_i \neq x_j$ se $i \neq j$, allora esiste ed è unico un polinomio $p(x)$ di grado limitato da n tale che, per $0 \leq i \leq n-1$,*

$$p(x_i) = y_i.$$

Quindi ci bastano n punti del grafico per individuare univocamente un polinomio di grado $< n$. Basta pensare a una retta che è un polinomio di grado limitato da 2: con due punti identifico una retta.

Dato un vettore $\vec{x} \in \mathbb{C}^n$ a n componenti distinte, la **rappresentazione per punti in base** $\vec{x}$ di un polinomio $p(x)$ di grado limitato da n è

$$p(x) \equiv (\vec{x}, \vec{y}), \quad \text{con } \vec{y} \in \mathbb{C}^n : y_i = p(x_i), \quad 0 \leq i \leq n-1$$

La base della rappresentazione sono i punti su cui viene valutato il polinomio. Anche la rappresentazione per punti è **estendibile**: un polinomio di grado limitato da n può essere rappresentato con una coppia di vettori $(\vec{x}^E, \vec{y}^E) \in \mathbb{C}^{n'} \times \mathbb{C}^{n'}$ con $n' > n$ valutando $p(x)$ sulle n' componenti di $\vec{x}^E$. Per esempio: $p(x) = x^2$ sulla base $\vec{x} = (0, 1, 2) \in \mathbb{C}^3$:

$$\vec{p}(x) \equiv (\vec{x}, \underbrace{(0, 1, 4)}_{\vec{y}})$$

mentre sulla base $\vec{x}^E = (0, 1, 2, 3) \in \mathbb{C}^4$:

$$\vec{p}(x) \equiv (\vec{x}, \underbrace{(0, 1, 4, 9)}_{\vec{y}^E})$$

4.1.3 Somma nella rappresentazione per coefficienti

Consideriamo due polinomi $p(x) \equiv \vec{a} \in \mathbb{C}^n$ e $q(x) \equiv \vec{b} \in \mathbb{C}^n$ (sono entrambi di grado limitato da n). Definiamo $s(x) = p(x) + q(x)$, allora

$$s(x) = p(x) + q(x) = \sum_{i=0}^{n-1} a_i x^i + \sum_{i=0}^{n-1} b_i x^i = \sum_{i=0}^{n-1} (a_i + b_i) x^i.$$

Il coefficiente di grado i di $s(x)$ è $s_i = a_i + b_i$. Quindi se $s(x) \equiv \vec{s}$ si ha che

$$\vec{s} = \vec{a} + \vec{b}.$$

Ponendo invece che i polinomi abbiano limite di grado diverso: $p(x) \equiv \vec{a} \in \mathbb{C}^m, q(x) \equiv \vec{b} \in \mathbb{C}^n$ e $m < n$, allora

$$\vec{s} = (\vec{a} \vec{0}_{n-m}) + \vec{b}$$

sfruttando il padding.

```

1 SUM( $\vec{a}, \vec{b}$ ) {  $\vec{a}, \vec{b} \in \mathbb{C}^n$  }
2    $n \leftarrow \vec{a}.size$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $c_i \leftarrow a_i + b_i$ 
5   return  $\vec{c}$ 
6
7 SUM( $\vec{a}, \vec{b}$ ) {  $\vec{a} \in \mathbb{C}^n, \vec{b} \in \mathbb{C}^m$  }
8    $n \leftarrow \vec{a}.size$ 
9    $m \leftarrow \vec{b}.size$ 
10
11   if ( $m < n$ ) then
12     for  $i \leftarrow 0$  to  $n - 1$  do
13        $c_i \leftarrow a_i + b_i$ 
14
15     for  $i \leftarrow n$  to  $m$  do
16        $c_i \leftarrow b_i$  // assegnamenti e non somme
17
18   else if ( $m > n$ ) then
19     for  $i \leftarrow 0$  to  $m - 1$  do
20        $c_i \leftarrow a_i + b_i$ 
21
22     for  $i \leftarrow n$  to  $n$  do
23        $c_i \leftarrow a_i$  // assegnamenti e non somme
24
25   return  $\vec{c}$ 

```

La complessità di $SUM(\vec{a}, \vec{b})$

$$T_S(n) = n \quad \text{o} \quad T(n, m) = \min\{m, n\}$$

4.1.4 Prodotto nella rappresentazione per coefficienti

Il calcolo del prodotto usando la rappresentazione di un polinomio con i coefficienti è più complessa. Consideriamo il prodotto tra polinomi con limite di grado uguale: $p(x) \equiv \vec{a} \in \mathbb{C}^n$ e $q(x) \equiv \vec{b} \in \mathbb{C}^n$. Allora il prodotto

$$\vec{c} \equiv c(x) = p(x) \cdot q(x) = \left(\sum_{j=0}^{n-1} a_j x^j \right) \cdot \left(\sum_{j=0}^{n-1} b_j x^j \right).$$

Notiamo che, a differenza della somma, il limite di grado di $c(x)$ aumenta, in particolare il grado massimo del polinomio è $(n-1) + (n-1) = 2n-2$ e di conseguenza il limite di grado è $2n-1$ (limite di grado = grado del polinomio + 1).

Soffermiamoci ora su coefficiente c_j del monomio di grado j del polinomio prodotto $c(x)$, con $0 \leq j \leq 2n-2$. A tale coefficiente contribuiscono tutti i prodotti di tutti i coefficienti dei

monomi $a_k x^k$ e $b_h x^h$ per cui $k+h=j$ (con $0 \leq h, k \leq n-1$). Siccome $h+k=j \Rightarrow h=j-k$, possiamo scrivere che

$$c_j = \sum a_k b_{j-k} \quad \text{per } 0 \leq j \leq 2n-2.$$

Ci rimane da ricavare gli estremi della sommatoria:

$$0 \leq k, h \leq n-1, h=j-k \Rightarrow 0 \leq k, j-k \leq n-1$$

dunque

$$\begin{cases} 0 \leq k \leq n-1 \\ 0 \leq j-k \leq n-1 \end{cases} \quad \begin{cases} k \geq 0 \\ k \leq n-1 \\ j-k \geq 0 \\ j-k \leq n-1 \end{cases} \quad \begin{cases} k \geq 0 \\ k \leq n-1 \\ k \leq j \\ k \geq j-n+1 \end{cases} \quad \begin{cases} 0 \leq k \leq n-1 \\ j-n+1 \leq k \leq j \end{cases}$$

Dalle due limitazioni inferiori e superiori si ricava che

$$\max\{0, j-n+1\} \leq k \leq \min\{j, n-1\}$$

infine possiamo scrivere gli indici della sommatoria

$$c_j = \sum_{k=\max\{0, j-n+1\}}^{\min\{j, n-1\}} a_k b_{j-k} \quad \text{per } 0 \leq j \leq 2n-2.$$

La relazione tra coefficienti del polinomio prodotto e coefficienti dei polinomi fattore che abbiamo ricavato, definisce un nuovo **operatore vettoriale**:

$$\begin{aligned} * : \mathbb{C}^n \times \mathbb{C}^n &\rightarrow \mathbb{C}^{2n-1} \\ \vec{c} &= \vec{a} * \vec{b} \end{aligned}$$

chiamato **convoluzione lineare**. Nel caso generale si ha $* : \mathbb{C}^m \times \mathbb{C}^n \rightarrow \mathbb{C}^{m+n-1}$. Dunque la convoluzione lineare, dati come operandi i coefficienti di due polinomi, ritorna i coefficienti del polinomio prodotto.

```

1 DEF_LIN_CONV( $\vec{a}$ ,  $\vec{b}$ ) {  $\vec{a}, \vec{b} \in \mathbb{C}^n$  }
2    $n \leftarrow \vec{a}.size$ 
3
4   for  $j \leftarrow 0$  to  $2n-2$  do
5      $k \leftarrow \max\{0, j-n+1\}$ 
6      $c_j \leftarrow a_k \cdot b_{j-k}$  // risparmiamo una somma ad ogni iterazione (2n-1 in totale)
7     for  $k \leftarrow k+1$  to  $\min\{j, n-1\}$  do
8        $c_j \leftarrow c_j + a_k \cdot b_{j-k}$ 
9
10  return  $\vec{c}$ 
    
```

La **complessità** dell'algoritmo che calcola la convoluzione lineare basandosi sulla definizione, calcola n^2 prodotti (basti pensare che stiamo calcolando il prodotto tra due polinomi che hanno n monomi ciascuno) per calcolare tutti i termini $a_k \cdot b_n$ e $n^2 - (2n-1)$ somme (dove $2n-1$ somme si risparmiano grazie all'assegnamento prima del secondo for) per accumularle nelle $2n-1$ sommatorie. Dunque possiamo concludere che

$$T_{LC}(n) = n^2 + n^2 - (2n-1) = \Theta(n^2).$$

Si tratta quindi di un algoritmo molto costoso che non è utilizzabile nell'ambito dei big data. Vedremo che sfruttando la rappresentazione dei polinomi per punti le cose miglioreranno.

4.1.5 Somma nella rappresentazione per punti

Perché le operazioni di somma e prodotto siano ben definite, è necessario che tutti i polinomi operando siano rappresentati per punti sulla stessa base. Poniamo $p(x) \equiv (\vec{x}, \vec{y}_p)$ e $q(x) \equiv (\vec{x}, \vec{y}_q)$ con $\vec{x}, \vec{y}_p, \vec{y}_q \in \mathbb{C}^n$. Sia $s(x) = p(x) + q(x)$, allora

$$s(x_i) = p(x_i) + q(x_i) = (y_p)_i + (y_q)_i \quad \text{per } 0 \leq i \leq n-1.$$

Quindi se $s(x) \equiv (\vec{x}, \vec{y}_s)$ si ha che

$$\vec{y}_s = \vec{y}_p + \vec{y}_q.$$

Dunque, come nel caso della somma di polinomi rappresentati tramite coefficienti, anche in questo caso il numero di somme effettuate è n e quindi $T_S(n) = n$.

4.1.6 Prodotto nella rappresentazione per punti

Poniamo $p(x) \equiv (\vec{x}, \vec{y}_p)$ e $q(x) \equiv (\vec{x}, \vec{y}_q)$ con $\vec{x}, \vec{y}_p, \vec{y}_q \in \mathbb{C}^n$ (stessa base). Anche per il prodotto sembra valere la stessa regola che abbiamo trovato per calcolare la somma: sia $c(x) = p(x) \cdot q(x)$

$$c(x_i) = p(x_i) \cdot q(x_i) \quad \text{per } 0 \leq i \leq n-1.$$

Tuttavia, facendo il prodotto componente per componente (indicato con il simbolo $\odot$), **non si ottiene una valutazione corretta** di $c(x)$; infatti sappiamo che il prodotto di due polinomi $p(x), q(x)$ di grado limitato da n ritorna un polinomio $c(x)$ di grado limitato da $2n-1$ e non di grado limitato n come si ottiene con il prodotto componente per componente. Per valutare correttamente $c(x)$ abbiamo bisogno di $2n-1$ punti.

Per risolvere il problema dobbiamo utilizzare le rappresentazioni estese di $p(x)$ e $q(x)$: poniamo $p(x) \equiv (\vec{x}^E, \vec{y}_p^E)$ e $q(x) \equiv (\vec{x}^E, \vec{y}_q^E)$, con $\vec{x}^E, \vec{y}_p^E, \vec{y}_q^E \in \mathbb{C}^{\geq 2n-1}$. Ora possiamo calcolare correttamente $c(x)$ sfruttando il prodotto componente per componente sia $c(x) \equiv (\vec{x}^E, \vec{y}_c^E)$

$$\vec{y}_c = \vec{y}_p^E \odot \vec{y}_q^E.$$

Avendo a disposizione rappresentazioni punto per punto estese dei due operandi, il prodotto tra i due polinomi può essere eseguito in tempo $T_P(n) = \Theta(n)$, quindi molto meglio rispetto al prodotto sfruttando la rappresentazione per coefficienti.

L'obiettivo a cui vogliamo arrivare è quello di calcolare la convoluzione lineare $\vec{c} = \vec{a} * \vec{b}$ sfruttando l'efficienza del prodotto tra due polinomi rappresentati per punti. L'unico problema è che ci rimane da trovare un algoritmo molto efficiente che permetta di effettuare delle conversioni veloci da una rappresentazione all'altra.

4.1.7 Conversione da coefficienti a punti: la valutazione

Come abbiamo anticipato, il processo di conversione per passare dalla rappresentazione a coefficienti a quella per punti, viene chiamato **valutazione**. Dato un polinomio di grado limitato da n rappresentato per coefficienti, la sua rappresentazione per punti rispetto alla base $\vec{x}$ si ottiene *valutando* le n componenti $y_i = p(x_i)$ del vettore $\vec{y}$.

```

1 V( $\vec{a}$ ,  $z$ )
2    $n \leftarrow \vec{a}.size$ 
3    $y \leftarrow a_0$ 
4    $pow \leftarrow 1$ 
5   for  $i \leftarrow 1$  to  $n-1$  do
6      $pow \leftarrow pow \cdot z$ 
7      $y \leftarrow y + a_i \cdot pow$ 
8   return  $y$  // componente di  $\vec{y}$ 

```

Invocando l'algoritmo V (basato sulla definizione) sul vettore $\vec{a}$ che contiene i coefficienti del polinomio per ogni valore z della base $\vec{x}$, si ottiene la conversione cercata. La **complessità** dell'algoritmo si ottiene immediatamente: $T_V(n) = 3(n-1)$ (per singola invocazione non per la conversione completa).

Si può ottenere un algoritmo con una costante moltiplicativa inferiore grazie alla **regola di Horner**. Tale regola si basa sul fatto che l'equazione che descrive un polinomio può essere riscritta come segue:

$$p(x) = \sum_{i=0}^{n-1} a_i x^i = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-2} + x(a_{n-1} \underbrace{\hspace{1cm}}_{n-1} \dots))).$$

L'algoritmo di Horner l'espressione viene valutata iterativamente partendo da destra verso sinistra.

```

1 HORNER( $\vec{a}$ ,  $z$ )
2    $n \leftarrow \vec{a}.\text{size}$ 
3    $y \leftarrow a_{n-1}$ 
4   for  $j \leftarrow 2$  to  $n$  do
5      $y \leftarrow a_{n-j} + z \cdot y$ 
6   return  $y$ 
    
```

La **complessità** dell'algoritmo: $T_H(n) = 2(n-1)$ (per singola invocazione), quindi migliore rispetto all'algoritmo basato sulla definizione.

Infine l'algoritmo per convertire $p(x) \equiv \vec{a} \in \mathbb{C}^n$ in $p(x) \equiv (\vec{x}, \vec{y})$ consiste nell'invocare l'algoritmo di Horner per ogni componente di $\vec{x}$.

```

1 VAL( $\vec{a}$ ,  $\vec{x}$ )
2    $n \leftarrow \vec{a}.\text{size}$ 
3   for  $i \leftarrow 0$  to  $n-1$  do
4      $y_i \leftarrow \text{HORNER}(\vec{a}, x_i)$ 
5   return  $\vec{y}$ 
    
```

La **complessità** dell'algoritmo: $T_{\text{VAL}}(n) = n \cdot 2(n-1) = \Theta(n^2)$. La conversione generale (che funziona per qualsiasi base $\vec{x}$) è molto costosa: con questo algoritmo non riusciamo a realizzare la convoluzione lineare sfruttando la velocità dell'algoritmo che calcola il prodotto partendo dalle rappresentazioni per punti di due polinomi. Vedremo che considerando una base specifica si otterranno migliori prestazioni.

4.1.8 Conversione da punti a coefficienti: l'interpolazione

Consideriamo $p(x) \equiv (\vec{x}, \vec{y})$, $\vec{x}, \vec{y} \in \mathbb{C}^n$. L'interpolazione determina il vettore di coefficienti $\vec{a} \in \mathbb{C}^n$ tali che

$$p(x) = \sum_{j=0}^{n-1} a_j x^j.$$

Dalla definizione di rappresentazione per punti, possiamo ricavare che deve essere soddisfatto il seguente sistema di equazioni:

$$p(x_i) = \sum_{j=0}^{n-1} a_j x_i^j = y_i, \quad \text{con } 0 \leq i \leq n-1.$$

Si tratta di un **sistema lineare** in cui: a_j sono le incognite; x_i^j sono i coefficienti dei monomi; y_i sono i termini noti. Definiamo quindi

$$[X]_{ij} = x_i^j \quad 0 \leq i, j \leq n-1, \quad X \in {}_n\mathbb{C}_n$$

$$X\vec{a} = \vec{y}.$$

La matrice X appartiene alla classe delle **matrici di Vandermonde**.

Definizione 4.1 (Matrice di Vandermonde). Dato un vettore $\vec{z} \in \mathbb{C}^n$, la matrice di Vandermonde $V(\vec{z}) = V(z_0, \dots, z_{n-1})$ ha come elemento (i, j) -esimo la potenza j -esima di z_i , per $0 \leq i, j \leq n-1$.

Si nota che la riga i -esima di $V(\vec{z})$ ha come elementi tutte le potenze consecutive di z_i .

$$V(\vec{z}) = \begin{bmatrix} z_0^0 & z_0^1 & z_0^2 & \dots & z_0^{n-1} \\ z_1^0 & z_1^1 & z_1^2 & \dots & z_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ z_{n-1}^0 & z_{n-1}^1 & z_{n-1}^2 & \dots & z_{n-1}^{n-1} \end{bmatrix}$$

Si dimostra che il determinante di una matrice di Vandermonde è dato da

$$\det(V(\vec{z})) = \prod_{i=0}^{n-1} \prod_{j=i+1}^{n-1} (z_j - z_i).$$

Se $z_i \neq z_j$ per $0 \leq i \neq j \leq n-1$ allora $\det(V(\vec{z})) \neq 0$ (quindi la matrice è invertibile). Siccome $X = V(\vec{x})$ è una Vandermonde costruita su una base $\vec{x}$ avente componenti distinti, è invertibile. Ciò implica che il sistema $X\vec{a} = \vec{y}$ ammette una e una sola soluzione. Per ricavare il vettore di incognite $\vec{a}$ si usa l'algoritmo di Gauss-Jordan, si ottiene

$$X\vec{a} = \vec{y} \xrightarrow{G.J.} I_n \vec{a} = X^{-1}\vec{y}.$$

L'algoritmo richiede n operazioni di pivot con trasformazione di tutti gli elementi, di conseguenza si ottiene come **complessità**: $T_I(n) = \Theta(n \cdot n^2) = \Theta(n^3)$. Complessità decisamente troppo elevata per essere utilizzata in pratica.

Interpolazione efficiente: formula di Lagrange Esiste un algoritmo più efficiente che permette di ottenere i coefficienti del polinomio interpolante sfruttando la formula di Lagrange. La formula di Lagrange descrive il polinomio interpolante in funzione delle componenti della sua rappresentazione per punti $(\vec{x}, \vec{y})$.

$$p(x) = \sum_{k=0}^{n-1} y_k \cdot \frac{\left[\sum_{\substack{j=0 \\ j \neq k}}^{n-1} (x - x_j) \right]}{\left[\sum_{\substack{i=0 \\ i \neq k}}^{n-1} (x_k - x_i) \right]} = \sum_{k=0}^{n-1} y_k \cdot \frac{Q_k(x)}{Q_k(x_k)}$$

Usando la formula di Lagrange si possono ricavare i coefficienti di $p(x)$ da $(\vec{x}, \vec{y})$ in tempo $T_L(n) = \Theta(n^2)$. Rispetto all'algoritmo che si basa sulla risoluzione del sistema lineare abbiamo ottenuto un miglioramento; tuttavia non è sufficiente da renderlo utilizzabile.

4.2 Discrete Fourier Transform e teorema di convoluzione lineare

In questa sezione vediamo come si realizzano conversioni efficienti da una rappresentazione di polinomi all'altra. Il nostro obiettivo rimane quello di trovare un modo di calcolare i coefficienti del polinomio prodotto in maniera efficiente, quindi evitando di utilizzare l'algoritmo che realizza il calcolo della convoluzione lineare basandosi sulla definizione. Nelle sottosezioni

precedenti abbiamo trovato un algoritmo che calcola il prodotto tra due polinomi rappresentati per punti avente complessità lineare. Tuttavia, per poterlo sfruttare, dobbiamo trovare degli algoritmi che effettuino delle conversioni di rappresentazione in maniera efficiente: non ne abbiamo ancora trovato uno. Il problema degli algoritmi di conversione che abbiamo analizzato è la *generalità* delle conversioni. Siccome non ci interessa convertire verso/da basi arbitrarie, si possono sfruttare delle basi specifiche che offrono prestazioni efficienti. Questa è l'idea che ci porta alla definizione di trasformata discreta di Fourier.

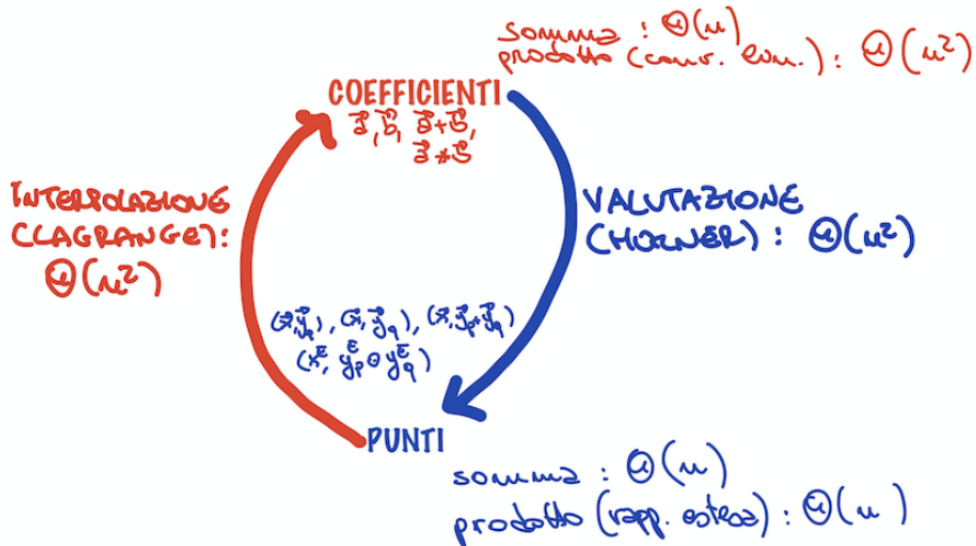


Figura 4.1: Riassunto costi legati alla rappresentazione per coefficienti/per punti e costi di conversione.

Definizione 4.2 (Discrete Fourier Transform). La Discrete Fourier Transform (DFT) di ordine n di $\vec{a}$, $\text{DFT}_n : \mathbb{C}^n \rightarrow \mathbb{C}^n$ è una trasformazione vettoriale che trasforma un generico vettore $\vec{a}$ di coefficienti del polinomio $p(x) \equiv \vec{a}$ di grado limitato da n , nel vettore $\vec{y}$ delle valutazioni $p(x)$ rispetto a una particolare base, indicata con $\vec{\Omega}_n$. Le componenti di $\vec{\Omega}_n$ sono le prime n potenze di un particolare numero complesso, ω_n , che denota la radice n -esima principale dell'unità in campo complesso. Ovvero

$$\vec{\Omega}_n = (\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}).$$

Si ha quindi che se $p(x) \equiv \vec{a}$, allora il vettore delle valutazioni $\vec{y} = \text{DFT}_n(\vec{a})$ è tale che

$$y_i = p(\omega_n^i) = \sum_{j=0}^{n-1} a_j \cdot (\omega_n^i)^j = \sum_{j=0}^{n-1} a_j \cdot \omega_n^{i \cdot j}.$$

Utilizziamo la trasformata discreta di Fourier per valutare $p(x) \equiv (\vec{\Omega}_n, \vec{y} = \text{DFT}_n(\vec{a}))$ e utilizziamo l'anti-trasformata per ricavare l'interpolazione $\text{DFT}^{-1}(\vec{y}) = \vec{a} \equiv p(x)$ da $p(x) \equiv (\vec{\Omega}_n, \vec{y} = \text{DFT}_n(\vec{a}))$. Nelle prossime sezioni vediamo come, grazie alle proprietà della base $\vec{\Omega}_n$, sia possibile realizzare un algoritmo (chiamato **Fast Fourier Transform (FFT)**, che ha complessità $\Theta(n \log n)$) che calcoli la Discrete Fourier Transform e la sua inversa in maniera molto efficiente. Sfruttando l'algoritmo FFT saremo in grado di realizzare la convoluzione lineare tramite il passaggio per la rappresentazione per punti.

L'utilizzo che fremo delle due trasformazioni per ottenere la convoluzione lineare è riassunto dal seguente teorema:

Teorema 4.2 (Teorema di convoluzione lineare). Dati $\vec{a}, \vec{b} \in \mathbb{C}^n$, sia $c = a * b \in \mathbb{C}^{2n-1}$ la loro convoluzione lineare. Allora:

$$(\vec{c}|0) = \text{DFT}_{2n}^{-1}(\text{DFT}_{2n}[(\vec{a}|\vec{0}_n)] \odot \text{DFT}_{2n}[(\vec{b}|\vec{0}_n)])$$

dove $\text{DFT}_{2n}[(\vec{a}|\vec{0}_n)]\text{DFT}_{2n}[(\vec{b}|\vec{0}_n)]$ sono valutazioni estese su $\vec{\Omega}_{2n}$; il prodotto componente per componente tra queste due ritorna la valutazione del polinomio prodotto $c(x) = p(x) \cdot q(x)$ su $2n$ punti; $(\vec{c}|0)$ viene aggiunto uno zero al vettore in quanto il polinomio $c(x)$ ha grado limitato da $2n - 1$.

4.3 La matrice di Fourier

La $\text{DFT}_n(\vec{a})$ rappresenta la valutazione di $p(x)$ sulla base $\vec{\Omega}_n$. La valutazione del polinomio $p(x)$ si traduce nel seguente calcolo:

$$\vec{y} = \text{DFT}_n(\vec{a}) = F_n \vec{a}, \quad F_n = V(\vec{\Omega}_n) = V(\omega_n^0, \dots, \omega_n^{n-1}).$$

dove $F_n = V(\vec{\Omega}_n)$, detta **matrice di Fourier**, è una matrice di Vandermonde. La matrice di Fourier ha la seguente struttura:

$$F_n = \begin{bmatrix} \omega_n^{0 \cdot 0} & \omega_n^{0 \cdot 1} & \dots & \omega_n^{0 \cdot j} & \dots & \omega_n^{0 \cdot (n-1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \omega_n^{i \cdot 0} & \omega_n^{i \cdot 1} & \dots & \omega_n^{i \cdot j} & \dots & \omega_n^{i \cdot (n-1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \omega_n^{(n-1) \cdot 0} & \omega_n^{(n-1) \cdot 1} & \dots & \omega_n^{(n-1) \cdot j} & \dots & \omega_n^{(n-1) \cdot (n-1)} \end{bmatrix}, \quad [F_n]_{ij} = \omega_n^{ij}$$

si nota che in tutta la prima riga e in tutta la prima colonna si trovano solo uni. Siccome $\omega_n^i \neq \omega_n^j$ per $0 \leq i \neq j \leq n-1$, F_n è invertibile e quindi

$$\vec{a} = \text{DFT}_n^{-1}(\vec{y}) = F_n^{-1} \vec{y}.$$

Vedremo che la struttura della matrice F_n^{-1} è molto simile a quella della F_n e vedremo che l'algoritmo di interpolazione DFT_n^{-1} sarà molto simile all'algoritmo di valutazione DFT_n .

4.4 Numeri complessi e radici n-esime dell'unità

4.4.1 Rappresentazione polare ed esponenziale

Ricordiamo che:

- rappresentazione cartesiana: $z \in \mathbb{C}$, $z \equiv (a, b) = a + ib$;
- rappresentazione polare: $z \in \mathbb{C}$, $z \equiv (\rho, \theta) = \rho(\cos \theta + i \sin \theta)$;
- rappresentazione esponenziale: $z \in \mathbb{C}$, $z = \rho e^{i\theta}$ dove convenzionalmente $e^{i\theta} = \cos \theta + i \sin \theta$.

La rappresentazione esponenziale è adatta per esprimere l'esponenziazione e l'estrazione di radici. Si nota infatti che

$$z = \rho e^{i\theta}, \quad z^n = \rho^n e^{i \cdot n \cdot \theta}.$$

Ricordiamo inoltre che ogni numero complesso $z \neq 0$ ammette n radici n -esime distinte $z_0, z_1, \dots, z_{n-1}$, con $z_j^n = z$ per $0 \leq j \leq n-1$. Sfruttando la rappresentazione esponenziale,

possiamo ottenere facilmente le rappresentazioni esponenziali di tali radici n -esime. Sia $z_j = \rho_j e^{i\theta_j}$, allora

$$z_j^n = \rho_j^n e^{in\theta_j} = z = \rho e^{i\theta}$$

da cui

$$\begin{aligned} \rho_j^n &= \rho \Rightarrow \rho_j = \rho^{1/n} \quad 0 \leq j \leq n-1 \\ n\theta_j &= \theta + 2\pi j \Rightarrow \theta_j = \frac{\theta}{n} + \frac{2\pi j}{n} \quad 0 \leq j \leq n-1 \end{aligned}$$

Dunque, dato $z = \rho e^{i\theta} \neq 0$, le sue n radici n -esime distinte sono:

$$z_j = \rho^{1/n} e^{i\left(\frac{\theta}{n} + \frac{2\pi j}{n}\right)} \quad 0 \leq j \leq n-1.$$

Consideriamo il **caso speciale** $z = 1 = 1 \cdot e^{i \cdot 0}$ (**unità complessa**). Per $n > 0$ le radici n -esime di z possono essere ottenute applicando la formula precedente. Si ottiene:

$$z_j = e^{i \cdot \frac{2\pi j}{n}} \quad 0 \leq j \leq n-1$$

Definiamo **radice principale n -esima dell'unità** ω_n la radice z_1 :

$$\omega_n = z_1 = e^{i \cdot \frac{2\pi}{n}}.$$

Notiamo che

$$z_j = e^{i \cdot \frac{2\pi j}{n}} = \left(e^{i \cdot \frac{2\pi}{n}} \right)^j = \omega_n^j \quad 0 \leq j \leq n-1$$

ovvero tutte le radici n -esime dell'*unità complessa* sono potenze successive di ω_n . Di conseguenza, la base $\tilde{\Omega}_n$ della rappresentazione per punti ritornata dalla DFT_n è il vettore $(\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1})$ delle radici n -esime dell'unità, ordinate secondo l'esponente di potenza. Osserviamo che:

- ω_n giace sulla circonferenza goniometrica di raggio unitario e il suo vettore forma un angolo di $2\pi/n$ con l'asse reale positivo;
- le altre radici ω_n^j si ottengono ruotando ω_n per incrementi successivi di $2\pi/n$ nell'angolo;
- sono definite le potenze negative di ω_n ottenute ruotando con incrementi di $-2\pi/n$ nell'angolo;
- le potenze sono periodiche: $\forall j \in \mathbb{Z}, \omega_n^{j \bmod n}$.

Nota che: avremmo potuto definire come radice principale n -esima $\bar{\omega}_n = e^{-i \cdot \frac{2\pi}{n}} (= \omega_n^{-1} = \omega_n^{n-1})$. Le altre radici si sarebbero ottenute comunque come potenze di $\bar{\omega}_n$ (in ordine diverso). Questa osservazione sarà utile per il calcolo di DFT^{-1} .

4.4.2 Proprietà delle radici n -esime dell'unità

Elenchiamo una serie di proprietà delle radici n -esime che ci torneranno utili per lo sviluppo dell'algoritmo FFT (l'algoritmo che calcola in maniera veloce la DFT).

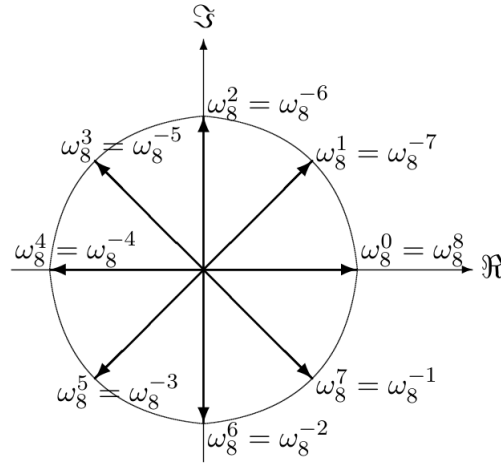


Figura 4.2: Radici ottave dell'unità sul piano complesso. Potenze positive in senso antiorario; negative in senso orario.

Proprietà di gruppo L'insieme $\vec{\Omega}_n$ delle radici n -esime dell'unità è un *gruppo moltiplicativo* rispetto al prodotto. Sono infatti verificate tutte le condizioni:

- *Chiusura rispetto al prodotto*: per $0 \leq i, j \leq n-1$ si ha che

$$\omega_n^i \cdot \omega_n^j = \omega_n^{i+j} = \omega_n^{(i+j) \bmod n}$$

cioè il prodotto di due radici n -esime dell'unità è ancora una radice n -esima dell'unità.

- *Elemento neutro*: poiché $1 = \omega_n^0$, $\vec{\Omega}_n$ contiene l'elemento neutro del prodotto in campo complesso.
- *Esistenza dell'inverso moltiplicativo*: l'inverso di ogni radice n -esima dell'unità è ancora una radice n -esima dell'unità. Infatti si ha che per $0 \leq i \leq n-1$:

$$\left(\omega_n^i\right)^{-1} = \omega_n^{-i} = \omega_n^{-i \bmod n} = \begin{cases} \omega_n^0 = 1 & i = 0 \\ \omega_n^{n-i} & 1 \leq i \leq n-1 \end{cases}$$

(proprietà utile per lo sviluppo della DFT⁻¹).

Lemma di cancellazione Si ha che $\forall n, d \in \mathbb{N}^+, \forall k \in \mathbb{Z}$

$$\omega_{dn}^{dk} = \omega_n^k.$$

Infatti:

$$\omega_{dn}^{dk} = \left(e^{j\frac{2\pi}{dn}}\right)^{dk} = e^{j\frac{2\pi}{n}k} = \omega_n^k.$$

Corollario del lemma di cancellazione Si ha che $\forall n \in \mathbb{N}^+$ pari

$$\omega_n^{n/2} = \omega_2 = -1.$$

Infatti, sfruttando il lemma di cancellazione:

$$\omega_n^{n/2} = \omega_{\frac{n}{2} \cdot 2}^{n/2} = \omega_2 = e^{j\frac{2\pi}{2}} = e^{j\pi} = -1.$$

Lemma di dimezzamento Lemma fondamentale per la proprietà di sotto-struttura: mette in relazione radici n -esime e radici $n/2$ -esime.

Sia $n > 0$ pari, allora gli n quadrati

$$\left(\omega_n^i\right)^2 \quad 0 \leq i \leq n-1$$

coincidono a coppie. In particolare, per $0 \leq i \leq n/2$, si ha che

$$\left(\omega_n^i\right)^2 = \left(\omega_n^{i+n/2}\right)^2 = \omega_{n/2}^i.$$

In altre parole, se eleviamo al quadrato tutte le radici n -esime otteniamo $n/2$ valori distinti (non n), dove ogni valore viene ottenuto due volte. Inoltre i valori ottenuti sono le $n/2$ radici $n/2$ -esime dell'unità. Infatti: per $0 \leq i \leq n/2 - 1$

$$\left(\omega_n^i\right)^2 = \omega_{2 \cdot \frac{n}{2}}^{2i} = \omega_{n/2}^i$$

$$\left(\omega_n^{i+n/2}\right)^2 = \omega_n^{2i+n} = \omega_n^{2i} \cdot \omega_n^n = \omega_n^{2i} \cdot \omega_n^{n \bmod n} = \omega_n^{2i} \cdot \omega_n^0 = \left(\omega_n^i\right)^2 = \omega_{n/2}^i.$$

Lemma di somma Siano $n \in \mathbb{N}^+$, $k \in \mathbb{Z}$, allora

$$\sum_{j=0}^{n-1} \left(\omega_n^k\right)^j = \begin{cases} n & \text{se } k \text{ multiplo di } n \\ 0 & \text{altrimenti} \end{cases}$$

Infatti: per il caso k multiplo di n

$$\begin{aligned} \sum_{j=0}^{n-1} \left(\omega_n^k\right)^j &= \sum_{j=0}^{n-1} \left(\omega_n^{k \bmod n}\right)^j \\ &= \sum_{j=0}^{n-1} \left(\omega_n^0\right)^j \\ &= \sum_{j=0}^{n-1} (1)^j \\ &= n \end{aligned}$$

per il caso k non multiplo di n

$$\begin{aligned} \sum_{j=0}^{n-1} \left(\omega_n^k\right)^j &= \frac{\omega_n^{kn} - 1}{\omega_n^k - 1} \\ &= \frac{\omega_n^{kn \bmod n} - 1}{\omega_n^k - 1} \\ &= \frac{\omega_n^0 - 1}{\omega_n^k - 1} \\ &= \frac{1 - 1}{\omega_n^k - 1} \\ &= 0 \end{aligned}$$

Osservazione 4.1. Si verifica facilmente che tutte le proprietà e i lemmi introdotti valgono anche rispetto alla rappresentazione delle radici n -esime dell'unità come potenze di $\overline{\omega_n} = \omega_n^{-1}$.

4.5 Proprietà di sotto-struttura per il calcolo di DFT_n

L'idea è quella di ricondurre la valutazione di un polinomio $p(x)$ di grado $\leq n$ su $\vec{\Omega}_n$ a valutazioni di due polinomi di grado $\leq n/2$ su $\vec{\Omega}_{n/2}$. D'ora in avanti considereremo $n = 2^k$, ciò non ci fa perdere di generalità per quanto riguarda il calcolo della convoluzione lineare.

4.5.1 Il caso base

Nel caso base (polinomio costante) si ha $n = 1$, quindi

$$\vec{\Omega}_1 = (\omega_1^0) = (1).$$

Se $p(x) \equiv \vec{a} \in \mathbb{C}^1$ allora

$$p(x) = a_0 \Rightarrow p(\omega_1^0) = a_0$$

quindi

$$DFT_1(\vec{a}) = \vec{a} = (a_0).$$

4.5.2 Passo generico

Ora consideriamo il caso generico $n = 2^k > 1$. Da $p(x) \equiv \vec{a}$ possiamo ottenere due polinomi con limite di grado $n/2$ separando le componenti di indice pari e di indice dispari di $\vec{a}$. Ovvero definiamo:

$$\begin{aligned} \vec{a}^{[0]} &= (a_0, a_2, \dots, a_{2i}, \dots, a_{n-2}) \quad (\text{indici pari}) \\ \vec{a}^{[1]} &= (a_1, a_3, \dots, a_{2i+1}, \dots, a_{n-1}) \quad (\text{indici dispari}) \end{aligned} \quad 0 \leq i \leq n/2 - 1$$

Possiamo considerare i vettori $\vec{a}^{[0]}$ e $\vec{a}^{[1]}$ come i coefficienti di due polinomi $p^{[0]}(x)$, $p^{[1]}(x)$ di grado limitato da $n/2$. Tra il polinomio $p(x)$ e i due polinomi appena definiti, esiste una relazione che li lega chiamata **identità polinomiale**:

$$\forall x \in \mathbb{C} : p(x) = p^{[0]}(x^2) + x \cdot p^{[1]}(x^2).$$

Per esempio:

$$\begin{aligned} n &= 4, \quad p(x) = 3 + 4x + 5x^2 + 6x^3, \quad \vec{a} = (3, 4, 5, 6) \\ \vec{a}^{[0]} &= (3, 5) \equiv p^{[0]}(x) = 3 + 5x, \quad \vec{a}^{[1]} = (4, 6) \equiv p^{[1]}(x) = 4 + 6x \\ p(x) &= 3 + 4x + 5x^2 + 6x^3 = 3 + 5x^2 + x \cdot (4 + 6x^2) = p^{[0]}(x^2) + p^{[1]}(x^2) \end{aligned}$$

Si può dimostrare che vale in generale. L'idea è quella di ottenere la valutazione di p su x valutando $p^{[0]}$ e $p^{[1]}$ su x^2 e combinando insieme le due valutazioni ottenute. Se poniamo $x = \omega_n^i$, allora $x^2 = (\omega_n^i)^2$ è una radice $n/2$ -esima (per il lemma di dimezzamento). L'identità polinomiale trasforma la valutazione di $p(x)$ su $\vec{\Omega}_n$ nelle valutazioni di $p^{[0]}(x)$ e $p^{[1]}(x)$ su $\vec{\Omega}_{n/2}$.

Possiamo definire formalmente la proprietà di sotto-struttura: poniamo

$$\vec{y} = DFT_n(\vec{a}), \quad \vec{y}^{[0]} = DFT_{n/2}(\vec{a}^{[0]}), \quad \vec{y}^{[1]} = DFT_{n/2}(\vec{a}^{[1]})$$

allora:

- le prime $n/2$ componenti y_i di $\vec{y}$, per $0 \leq i \leq n/2 - 1$ possono essere ottenute come:

$$\begin{aligned} y_i &= p(\omega_n^i) \\ &= p^{[0]}(\omega_n^{2i}) + \omega_n^i \cdot p^{[1]}(\omega_n^{2i}) \quad (\text{per l'identità polinomiale}) \\ &= p^{[0]}(\omega_{n/2}^i) + \omega_n^i \cdot p^{[1]}(\omega_{n/2}^i) \quad (\text{per il lemma di dimezzamento}) \\ &= y_i^{[0]} + \omega_n^i \cdot y_i^{[1]} \end{aligned}$$

- le ultime $n/2$ componenti $y_{i+n/2}$ di $\vec{y}$, per $0 \leq i \leq n/2 - 1$ possono essere ottenute come:

$$\begin{aligned}
 y_{i+n/2} &= p(\omega_n^{i+n/2}) \\
 &= p^{[0]}(\omega_n^{2(i+n/2)}) + \omega_n^{i+n/2} \cdot p^{[1]}(\omega_n^{2(i+n/2)}) \quad (\text{per l'identità polinomiale}) \\
 &= p^{[0]}(\omega_{n/2}^i) + \omega_n^{n/2} \cdot \omega_n^i \cdot p^{[1]}(\omega_{n/2}^i) \quad (\text{per il lemma di dimezzamento}) \\
 &= y_i^{[0]} - \omega_n^i \cdot y_i^{[1]} \quad (\text{per corol. lemma di cancellazione})
 \end{aligned}$$

In questo modo le componenti y_i e $y_{i+n/2}$ si ottengono con le due combinazioni lineari $y_i^{[0]} \pm \omega_n^i \cdot y_i^{[1]}$, con $0 \leq i \leq n/2 - 1$. Grazie alle proprietà di questi particolari numeri complessi, le valutazioni su punti distinti della base condividono molti termini in comune che vengono calcolati una sola volta. Questo è il motivo del risparmio computazionale.

4.6 La Fast Fourier Transform (FFT)

Dalla proprietà di sotto-struttura enunciata nella precedente sezione, discende direttamente l'algoritmo.

```

1  FFT( $\vec{a}$ ) { $n = 2^k$ }
2       $n \leftarrow \vec{a}.size$ 
3
4  [B] if ( $n = 1$ ) then
5      return  $\vec{a}$ 
6
7  [D]  $\vec{a}^{[0]} \leftarrow (a_0, a_2, a_4, \dots, a_{n-2})$ 
8       $\vec{a}^{[1]} \leftarrow (a_1, a_3, a_5, \dots, a_{n-1})$ 
9
10 [R]  $\vec{y}^{[0]} = \text{FFT}(\vec{a}^{[0]})$ 
11      $\vec{y}^{[1]} = \text{FFT}(\vec{a}^{[1]})$ 
12
13 [C]  $ON \leftarrow e^{i \cdot \frac{2\pi}{n}}$  // sarebbe  $\omega_n$ 
14      $ONI \leftarrow 1$  // conteggio di  $\omega_n^i$  inizializzato a  $\omega_n^0$ 
15
16 for  $i \leftarrow 0$  to  $n/2 - 1$  do
17      $t \leftarrow ONI \cdot y_i^{[1]}$ 
18      $y_i \leftarrow y_i^{[0]} + t$ 
19      $y_{i+n/2} \leftarrow y_i^{[0]} - t$ 
20      $ONI \leftarrow ONI \cdot ON$ 
21
22 return  $\vec{y}$ 
    
```

La **correttezza** dell'algoritmo discende direttamente dalla correttezza della proprietà di sotto-struttura. Per studiare la **complessità** sfruttiamo il MT:

$$T_{\text{FFT}}(n) = 2T_{\text{FFT}}(n/2) + (n/2) \cdot 4 = 2T_{\text{FFT}}(n/2) + 2n$$

da cui

$$\begin{aligned}
 n^{\log_b a} &= n^{\log_2 2} = n, \quad w(n) = 2n \\
 w(n) &= \Theta(n^{\log_b a}) = \Theta(n) \\
 \Rightarrow \text{Caso 2} &\Rightarrow T_{\text{FFT}}(n) = \Theta(n \log n)
 \end{aligned}$$

Rispetto all'algoritmo per la valutazione di Horner che abbiamo analizzato nella sottosezione 4.1.7, con l'algoritmo FFT, si ottiene un miglioramento di quasi un ordine di grandezza. Vedremo che utilizzando l'algoritmo FFT è possibile realizzare l'operazione di convoluzione lineare in un tempo $\Theta(n \log n)$.

4.7 La trasformata discreta inversa di Fourier

L'ultimo mattoncino che ci serve per realizzare l'algoritmo che realizza in modo efficiente l'operazione di convoluzione lineare, è il calcolo della trasformata inversa $\vec{a} = \text{DFT}^{-1}(\vec{y})$, che corrisponde all'interpolazione di un polinomio a partire da una sua rappresentazione per punti con base $\vec{\Omega}_n$.

4.7.1 La matrice F_n^{-1}

Nella sezione 4.3 abbiamo visto che $\vec{y} = \text{DFT}_n(\vec{a}) = F_n \vec{a}$ che è una funzione lineare e invertibile e abbiamo visto che la matrice associata $F_n = V(\vec{\Omega}_n)$ è una matrice di Vandermonde. La trasformata inversa $\vec{a} = \text{DFT}^{-1}(\vec{y}) = F_n^{-1} \vec{y}$ è ancora una funzione lineare con matrice associata F_n^{-1} (l'inversa della matrice di Fourier di ordine n).

Teorema 4.3 (Struttura matrice F_n^{-1}). *Si ha che*

$$F_n^{-1} = \frac{1}{n} \begin{bmatrix} 1 & 1 & \dots & 1 & \dots & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & \omega_n^{-i \cdot 1} & \dots & \omega_n^{-i \cdot j} & \dots & \omega_n^{-i \cdot (n-1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & \omega_n^{-(n-1) \cdot 1} & \dots & \omega_n^{-(n-1) \cdot j} & \dots & \omega_n^{-(n-1) \cdot (n-1)} \end{bmatrix}, \quad [F_n^{-1}]_{ij} = \frac{1}{n} \cdot \omega_n^{-ij}$$

Notiamo che F_n^{-1} è quasi una Vandermonde (non lo è per via del $1/n$) ed è molto simile a F_n :

$$F_n^{-1} = \frac{1}{n} \cdot V(\omega_n^0, \omega_n^{-1}, \dots, \omega_n^{-(n-1)}).$$

Per provare la veridicità del teorema, possiamo verificare che $F_n^{-1} \times F_n$ ritorna la matrice identità di ordine n I_n .

$$\begin{aligned} [F_n^{-1} \times F_n]_{ij} &= \sum_{k=0}^{n-1} [F_n^{-1}]_{ik} \cdot [F_n]_{kj} \\ &= \sum_{k=0}^{n-1} \frac{1}{n} \omega_n^{-ik} \cdot \omega_n^{kj} \\ &= \frac{1}{n} \sum_{k=0}^{n-1} \omega_n^{k(j-i)} \\ &= \frac{1}{n} \sum_{k=0}^{n-1} \left(\omega_n^{(j-i)} \right)^k \end{aligned}$$

Se $i = j$, allora

$$= \frac{1}{n} \sum_{k=0}^{n-1} \left(\omega_n^0 \right)^k = \frac{1}{n} \cdot n = [I_n]_{ij}.$$

Se $i \neq j$, allora

$$-(n-1) \leq j-i \neq 0 \leq n-1$$

ma in $[-(n-1), (n-1)]$ l'unico multiplo di n è 0. Quindi $j-i$ non è multiplo di n e dunque (sfruttando il lemma della somma)

$$= \frac{1}{n} \sum_{k=0}^{n-1} (\omega_n^{(j-i)})^k = \frac{1}{n} \cdot 0 = 0.$$

Possiamo concludere che

$$\left[F_n^{-1} \times F_n \right]_{ij} = [I_n]_{ij}$$

come si voleva dimostrare.

Abbiamo visto nelle precedenti sezioni che le potenze negative di ω_n sono esse stesse radici n -esime dell'unità, infatti: $\omega_n^{-i} = \omega_n^{-i \bmod n} = \omega_n^{n-i}$. Da ciò ne consegue che le righe della matrice $F_n^{-1} = V(\omega_n^0, \omega_n^{-1}, \dots, \omega_n^{-(n-1)})$ sono una permutazione delle righe di F_n .

4.7.2 DFT⁻¹ come valutazione su potenze negative di ω_n

L'espressione analitica della trasformata inversa discreta di Fourier $\vec{a} = \text{DFT}^{-1}(\vec{y}) = F_n^{-1}\vec{y}$ è

$$a_i = \frac{1}{n} \cdot \sum_{k=0}^{n-1} y_k \cdot \omega_n^{-ik} = \frac{1}{n} \sum_{k=0}^{n-1} y_k \cdot (\omega_n^{-i})^k, \quad 0 \leq i \leq n-1.$$

La serie troncata di potenze può essere vista come la valutazione di $q(x) \equiv \vec{y}$ su ω_n^{-1} (vedi definizione 4.2). Così facendo possiamo scrivere

$$a_i = \frac{1}{n} q(\omega_n^{-i}).$$

Una proprietà della rappresentazione per punti in base $\vec{\Omega}_n$ è che anche l'interpolazione da quella base può essere riportata ad un'ulteriore valutazione: viene valutato il polinomio avente come coefficienti il vettore $\vec{y}$ rispetto alle potenze negative della radice principale n -esima dell'unità. Come abbiamo già detto, definendo $\bar{\omega}_n = \omega_n^{-1}$, le radici n -esime dell'unità possono essere ottenute come potenze successive di $\bar{\omega}_n$, inoltre i lemmi e le definizioni che abbiamo enunciato nella sottosezione 4.4.2 rimangono validi.

Possiamo definire l'algoritmo FFT_NEG per la valutazione $q(x) \equiv \vec{y}$ sulle potenze $\bar{\omega}_n = \omega_n^{-1}$.

```

1 FFT_NEG( $\vec{y}$ ) { $n = 2^k$ }
2    $n \leftarrow \vec{y}.size$ 
3
4 [B] if ( $n = 1$ ) then
5     return  $\vec{a}$ 
6
7 [D]  $\vec{y}^{[0]} \leftarrow (y_0, y_2, y_4, \dots, y_{n-2})$ 
8      $\vec{y}^{[1]} \leftarrow (y_1, y_3, y_5, \dots, y_{n-1})$ 
9
10 [R]  $\vec{a}^{[0]} = \text{FFT\_NEG}(\vec{y}^{[0]})$ 
11      $\vec{a}^{[1]} = \text{FFT\_NEG}(\vec{y}^{[1]})$ 
12
13 [C] ON_NEG  $\leftarrow e^{-i \cdot \frac{2\pi}{n}}$  // sarebbe  $\bar{\omega}_n$ 
14     ON_NEGI  $\leftarrow 1$  // conteggio di  $\bar{\omega}_n^i$  inizializzato a  $\bar{\omega}_n^0$ 
15
16 for  $i \leftarrow 0$  to  $n/2 - 1$  do
17      $t \leftarrow \text{ON\_NEGI} \cdot y_i^{[1]}$ 
18      $a_i = a_i^{[0]} + t$ 
    
```

```

19      $a_{i+n/2} = a_i^{[0]} - t$ 
20     ON_NEGI  $\leftarrow$  ON_NEGI  $\cdot$  ON_NEG
21
22     return  $\vec{a}$ 
    
```

Si tratta di un algoritmo del tutto simmetrico a FFT. Infine possiamo scrivere l'algoritmo che calcola l'antitrasformata:

```

1  INV_FFT( $\vec{y}$ ) {n =  $2^k$ }
2      n  $\leftarrow$   $\vec{y}$ .size
3       $\vec{a} \leftarrow$  FFT_NEG( $\vec{y}$ )
4      for i  $\leftarrow$  0 to n-1 do
5           $a_i \leftarrow a_i \cdot \frac{1}{n}$ 
6      return  $\vec{a}$ 
    
```

La **correttezza** dell'algoritmo dalle considerazioni fatte sopra. La **complessità** è: $T_{\text{INV_FFT}}(n) = \Theta(n \log n + n) = \Theta(n \log n)$.

Esiste un **secondo approccio** per il calcolo della DFT^{-1} che si basa sul fatto che le prime n potenze di $\bar{\omega}_n = \omega_n^{-1}$ coincidono con le potenze di ω_n . Si ha infatti che

$$\bar{\omega}_n^i = \omega_n^{-i} = \begin{cases} \omega_n^0 = 1 & i = 0 \\ \omega_n^{-i \bmod n} = \omega_n^{n-i} & 1 \leq i \leq n-1 \end{cases}$$

Quindi le valutazioni eseguite da FFT_NEG su $(\bar{\omega}_0, \dots, \bar{\omega}_{n-1})$ sono sempre valutazioni sulle n radici n -esime dell'unità ma in ordine diverso. Se $q(x) = \vec{y}$:

$$[\text{DFT}_n^{-1}(\vec{y})]_i = \frac{1}{n} q(\omega_n^{-i}) = \begin{cases} \frac{1}{n} \cdot [\text{DFT}_n(\vec{y})]_0 & i = 0 \\ \frac{1}{n} \cdot [\text{DFT}_n(\vec{y})]_{n-i} & 1 \leq i \leq n-1 \end{cases}$$

Possiamo riscrivere in forma compatta:

$$[\text{DFT}_n^{-1}(\vec{y})]_i = \frac{1}{n} \cdot [\text{DFT}_n(\vec{y})]_{(n-i) \bmod n} \quad 0 \leq i \leq n-1.$$

Dato un vettore $\vec{x} = (x_0, \dots, x_{n-1}) \in \mathbb{C}^n$, chiamiamo $\vec{x}^R = (x_0, x_{n-1}, \dots, x_1) \in \mathbb{C}^n$ il suo **reverse** (la componente x_0 e l'unica che rimane al suo posto), tale che $[\vec{x}^R]_i = x_{(n-i) \bmod n}$ per $0 \leq i \leq n-1$. Sfruttando il *reverse* possiamo scrivere che

$$\text{DFT}_n^{-1}(\vec{y}) = \frac{1}{n} \text{DFT}_n^R(\vec{y}).$$

Sfruttando quest'ultima relazione si ricava l'algoritmo alternativo per il calcolo della trasformata inversa:

```

1  INV_FFT( $\vec{y}$ )
2      n  $\leftarrow$   $\vec{y}$ .size
3       $\vec{a}' \leftarrow$  FFT( $\vec{y}$ )
4       $a_0 \leftarrow \frac{1}{n} \cdot a'_0$  //  $a_0$  e' in prima posizione anche in reverse
5      for i  $\leftarrow$  1 to n-1 do
6           $a_i \leftarrow \frac{1}{n} \cdot a'_{n-i}$ 
7      return  $\vec{a}$ 
    
```


4.8 Calcolo efficiente della convoluzione lineare

Sfruttando le conversioni veloci implementate dagli algoritmi FFT e INV_FFT, possiamo finalmente calcolare la convoluzione lineare $\vec{c} = \vec{a} * \vec{b}$ di due vettori $n = 2^k$ componenti in tempo $\Theta(n \log n)$.

```

1 LIN_CONV( $\vec{a}$ ,  $\vec{b}$ )
2    $n \leftarrow \vec{a}.\text{size}$ 
3
4   // Valutazione di  $p(x) \equiv \vec{a}$  e  $q(x) \equiv \vec{b}$  su  $\vec{\Omega}_{2n}$ .
5    $\vec{y}_a \leftarrow \text{FFT}(\vec{a}|\vec{0}_n)$ 
6    $\vec{y}_b \leftarrow \text{FFT}(\vec{b}|\vec{0}_n)$ 
7
8   // Prodotto componente per componente:  $\vec{y}_c = \vec{y}_a \odot \vec{y}_b$ .
9   for  $i \leftarrow 0$  to  $2n-1$  do
10      $y_{ci} \leftarrow y_{ai} \cdot y_{bi}$ 
11
12   return INV_FFT( $\vec{y}_c$ ) // Ritorna ( $\vec{c}|0$ ).
```

L'algoritmo si può applicare anche a vettori di taglia $n \neq 2^k$ invocando LIN_CONV su vettori estesi (usando il *padding*)

$$(\vec{a}|\vec{0}_{2^{\lceil \log_2 n \rceil} - n}), \quad (\vec{b}|\vec{0}_{2^{\lceil \log_2 n \rceil} - n})$$

di taglia $2^{\lceil \log_2 n \rceil} \leq 2n$. Abbiamo esteso $\vec{a}$ e $\vec{b}$ aggiungendo il minimo numero di zeri per far sì che i vettori estesi abbiano una lunghezza che sia una potenza di due.

Se siamo nel caso in cui $\vec{a} \in \mathbb{C}^m$ e $\vec{b} \in \mathbb{C}^n$ con $m < n$, allora

$$(\vec{a}|\vec{0}_{n-m}|\vec{0}_{2^{\lceil \log_2 n \rceil} - n}), \quad (\vec{b}|\vec{0}_{2^{\lceil \log_2 n \rceil} - n}).$$

In particolare a $\vec{b}$ (vettore più lungo) abbiamo aggiunto un numero di zeri in modo da rendere la lunghezza del vettore esteso una potenza di due; al vettore $\vec{a}$ abbiamo aggiunto un numero di zeri sufficiente per portarlo alla stessa dimensione di $\vec{b}$ più lo stesso numero di zeri precedentemente aggiunti a $\vec{b}$ per ottenere la sua versione estesa. In questo modo si ottengono due rappresentazioni estese dei vettori $\vec{a}$, $\vec{b}$ aventi come taglia una potenza di due.

Capitolo 5

Programmazione dinamica

Il paradigma D&C che abbiamo studiato e sfruttato nei capitoli precedenti, ha un problema: è **puramente funzionale**, ovvero il processo di risoluzione è *memoryless*. Negli algoritmi D&C se capitano n sotto-istanze identiche all'interno dello stesso albero delle chiamate, queste verranno risolte n volte; non vengono sfruttati i risultati calcolati in precedenza. Gli algoritmi dinamici migliorano questo aspetto andando a memorizzare le soluzioni delle sotto-istanze già risolte in una tabella.

5.1 Il problema del D&C: i numeri di Fibonacci

Consideriamo il calcolo dell' n -esimo numero di Fibonacci. Per definizione:

$$F_n = \begin{cases} 1 & n = 0, 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}.$$

Dalla definizione si può ricavare direttamente l'algoritmo D&C:

```
1 RFIB(n)
2   if (n = 0 or n = 1) then return 1
3   return RFIB(n-1) + RFIB(n-2)
```

La correttezza deriva direttamente dalla definizione. Studiamo la **complessità** partendo dalla relazione di ricorrenza:

$$T(n) = \begin{cases} 0 & n = 0, 1 \\ T(n-1) + T(n-2) + 1 & n > 1 \end{cases}.$$

Limitiamoci a ricavare un limite inferiore sfruttando l'unfolding:

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &\geq 2T(n-2) + 1 \quad (T(n) \text{ è non decrescente}) \\ &\geq 2[2T(n-4) + 1] + 1 \\ &\geq 4T(n-4) + 2 + 1 \\ &\geq 4[2T(n-6) + 1] + 2 + 1 \\ &\geq 8T(n-6) + 4 + 2 + 1 \\ &\vdots \\ &\geq 2^i T(n-2i) + \sum_{j=0}^{i-1} 2^j \end{aligned}$$

Il numero di passi necessari per arrivare ai casi di base è sempre $i^* = \lfloor n/2 \rfloor$, infatti:

$$\begin{cases} n - 2i^* = n - 2n/2 = 0 & \text{se } i \text{ è pari} \\ n - 2i^* = n - 2(n-1)/2 = 1 & \text{se } i \text{ è dispari} \end{cases}$$

dunque infine si ottiene:

$$T(n) \geq 2^{\lfloor n/2 \rfloor} T(n - 2\lfloor n/2 \rfloor) + 2^{\lfloor n/2 \rfloor} - 1 = 2^{\lfloor n/2 \rfloor} - 1 \Rightarrow T(n) = \Omega(2^{n/2}).$$

Abbiamo ottenuto una minorazione che è una funzione esponenziale in n ! La figura 5.1 mostra l'albero delle chiamate associato all'esecuzione di $\text{RFIB}(5)$: notiamo che alcuni sotto-istanze vengono calcolate più di una volta. Immaginiamo di dover calcolare $\text{RFIB}(n)$ per n molto grande, questo comportamento di replicazione dei calcoli esplode ed è per questo che la complessità dell'algoritmo è esponenziale.

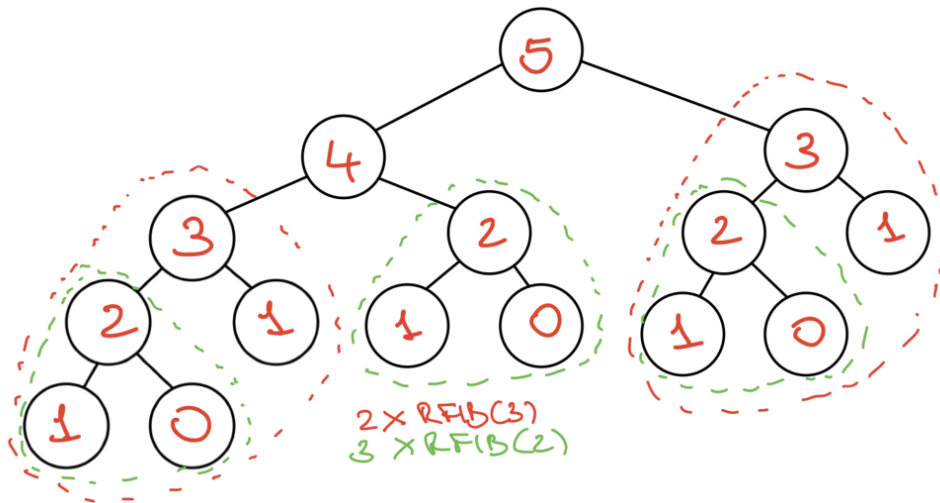


Figura 5.1: Albero delle chiamate per $\text{RFIB}(5)$. Notiamo che $\text{RFIB}(3)$ viene calcolato 2 volte; $\text{RFIB}(2)$ viene calcolato 3 volte. Con n molto grande effettuano gli stessi calcoli moltissime volte.

L'approccio più naturale è quello di calcolare i numeri di Fibonacci partendo "dal basso" (**bottom-up**) e l'approccio top-down (e successivamente bottom-up) che offre il paradigma D&C. L'idea è quella di memorizzare in una tabella le sotto-istanze delle soluzioni che vengono calcolate.

```

1 ITFIB(n)
2   if (n = 0 or n = 1) then return 1
3   // Memorizzo i risultati nella tabella F[i]
4   F[0] ← 1 // Inserisco le soluzioni per i = 0, 1
5   F[1] ← 1
6   for i ← 2 to n do
7       F[i] ← F[i-1] + F[i-2]
8   return F[n]
```

La **complessità** in questo caso è:

$$T(n) = \sum_{i=2}^n 1 = n - 2 + 1 = n - 1 = \Theta(n).$$

Abbiamo ottenuto un algoritmo che è esponenzialmente più veloce della versione D&C.

Osservazione 5.1. Va detto che, sebbene l'algoritmo ITFIB sia esponenzialmente migliore rispetto a RFIB, questo risulta tuttavia inefficiente dal punto di vista della vera taglia dell'istanza. Tale taglia, che indichiamo con $\text{size}(n)$, non è n (il valore dell'indice del numero di Fibonacci da calcolare), bensì il numero di cifre necessarie a rappresentare n in notazione posizionale. Se $\mathbb{I} = \mathbb{N}$, per $n \in \mathbb{I}$ si ha che $\text{size}(n) = \Theta(\log n)$ per una qualsiasi rappresentazione posizionale a base costante b . Di conseguenza si ha che una complessità lineare in n risulta essere esponenziale nella taglia dell'istanza: $\Theta(n) = \Theta(b^{\text{size}(n)})$. Secondo questa logica, l'algoritmo ricorsivo risulta avere una complessità doppiamente esponenziale nella taglia dell'istanza: $\Theta(2^{n/2}) = \Theta(2^{b^{\text{size}(n)}})$.

L'esempio dell'algoritmo di Fibonacci ci ha permesso di introdurre la potenza della memorizzazione delle soluzioni delle sotto-istanze, tuttavia siamo ancora lontani da un algoritmo efficiente. In realtà, l'algoritmo per il calcolo dei numeri di Fibonacci sfrutta la seguente formula chiusa:

$$F_n = \frac{\phi^{n+1} - \psi^{n+1}}{\sqrt{5}} \quad \text{dove} \quad \phi = \frac{1 + \sqrt{5}}{2}, \quad \psi = \frac{1 - \sqrt{5}}{2} (< 0).$$

Sfruttando tale formula e un algoritmo di esponenziazione adeguato, si può calcolare F_n in $\Theta(\log n)$ ($= \Theta(\text{size}(n))$). Siccome l'utilizzo di ϕ e ψ richiede di lavorare con numeri reali, esiste un algoritmo ancora più efficiente che lavora su interi. Tale algoritmo si basa sulla seguente identità:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}.$$

Applicando la tecnica dell'unfolding si ottiene che:

$$\begin{aligned} \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-2} \\ F_{n-3} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^2 \begin{bmatrix} F_{n-2} \\ F_{n-3} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^3 \begin{bmatrix} F_{n-3} \\ F_{n-4} \end{bmatrix} \\ &\vdots \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{1+i} \begin{bmatrix} F_{n-(1+i)} \\ F_{n-(2+i)} \end{bmatrix} \end{aligned}$$

L'unfolding termina quando:

$$\begin{cases} n - (1 + i) = 1 \\ n - (2 + i) = 0 \end{cases} \Rightarrow i = n - 2.$$

Da cui si ottiene infine:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}.$$

Utilizzando un algoritmo di esponenziazione avente complessità $\Theta(\log n)$, è possibile calcolare il numero di Fibonacci n -esimo in tempo logaritmico operando solamente su numeri interi.

5.2 Prime considerazioni sulla programmazione dinamica

Come abbiamo detto nella sezione precedente, l'idea di base della programmazione dinamica è quella di memorizzare le soluzioni delle sotto-istanze già calcolate. Elenchiamo alcune altre caratteristiche del paradigma:

- Come il paradigma D&C, anche il DP (*Dinamic-Programming*) si basa sulla proprietà di sotto-struttura.
- Si tratta di una tecnica di risoluzione iterativa che parte dalle istanze di taglia più piccola per costruire progressivamente le soluzioni delle istanze più grandi (**bottom-up**).
- Rispetto al paradigma D&C, in DP scompare la fase di *top-down*: questa fase era fondamentale per preparare la gerarchia di sotto-istanze da risolvere.
 - La mancanza della fase di top-down, comporta che in DP, nella risoluzione di una certa istanza, vengano calcolate più sotto-istanze di quelle necessarie per risolvere quella principale (si procede più "alla cieca"). Quindi l'utilizzo di un approccio DP conviene se il numero di tutte le possibili sotto-istanze da risolvere è "piccolo".
 - L'organizzazione della sola parte di bottom-up, in certi casi, non è banale dal punto di vista implementativo (vedremo degli esempi).

Riassumendo, per applicare un approccio di risoluzione DP: dobbiamo individuare una proprietà di sotto-struttura del problema considerato; devono esserci sotto-istanze ripetute; lo spazio delle sotto-istanze da risolvere deve essere sufficientemente piccolo.

5.3 La memoizzazione

La memoizzazione (dalla parola *memo* - promemoria) è un approccio ibrido che permette di mantenere i vantaggi della fase di top-down dell'approccio D&C (sfruttando la ricorsione sulla proprietà di sotto-struttura) e aggiunge all'algoritmo una struttura dati (tabella) per memorizzare le soluzioni delle istanze già risolte. Un controllo (**lookup**) nella tabella permette di evitare risoluzioni multiple.

Un implementazione generica di un algoritmo X di DP, comprende le seguenti due procedure:

1. una procedura di inizializzazione (non ricorsiva) (che indichiamo con il nome di **INIT_X**);
2. una procedura ricorsiva di risoluzione, invocata alla fine della procedura di inizializzazione (che indicheremo con il nome di **REC_X**).

Più precisamente, la procedura **INIT_X**:

1. risolve direttamente i casi base;
2. inizializza la tabella con le soluzioni di base e inizializza ad un valore di default (*sentinella*) le locazioni delle sotto-istanze non di base non ancora calcolate. Così facendo, sfruttando un lookup sulla tabella, si riescono a distinguere le sotto-istanze già risolte da quelle ancora da risolvere;
3. invoca la procedura di calcolo ricorsiva **REC_X**.

Per quanto riguarda la procedura **REC_X** (invocata sulla generica sotto-istanza):

1. controlla (tramite il valore di default) se la soluzione è presente o meno nella tabella;

2. se la soluzione non è presente, allora viene calcolata applicando la proprietà di sotto-struttura (e facendo chiamate ricorsive) e successivamente viene memorizzata nella tabella;
3. infine viene ritornata la soluzione delle sotto-istanza (letta dalla tabella o calcolata).

Vediamo il codice memoizzato per il calcolo dell' n -esimo numero di Fibonacci:

```

1 INIT_FIB(n)
2   if (n <= 1) then return 1; // Risoluzione diretta del caso base.
3
4   // Inizializzazione casi base nella tabella F[i].
5   F[0] ← 1
6   F[1] ← 1
7
8   for i ← 2 to n do
9       F[i] ← 0 // Valore di default (Fib(i) > 0).
10
11  // Invocazione procedura ricorsiva.
12  return REC_FIB(i)
13
14 REC_FIB(i)
15  if (F[i] = 0) then // Controllo se la soluzione e' nella tabella.
16      F[i] ← REC_FIB(i-1) + REC_FIB(i-2) // Calcolo soluzione e memorizzo.
17      return F[i]
18  return F[i] // Se la soluzione e' presente nella tabella non la ricalcolo.

```

La correttezza dell'algoritmo deriva dalla proprietà di sotto-struttura e dalla scelta del valore di default. Per quanto riguarda la **complessità**, non possiamo studiare le ricorrenze come abbiamo fatto per gli algoritmi che si basano sul paradigma D&C, dobbiamo effettuare un'analisi dell'albero delle chiamate.

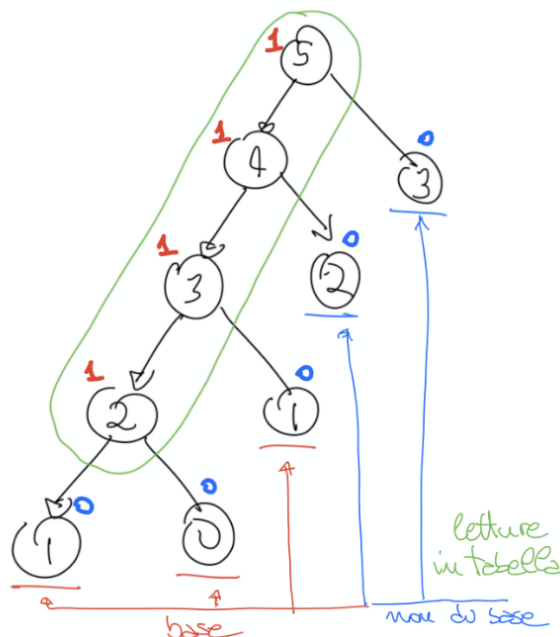


Figura 5.2: Albero delle chiamate dell'algoritmo di Fibonacci memoizzato.

Ogni istanza può essere un nodo interno una volta sola, tutte le altre volte è una foglia. L'albero che si ottiene per un generico valore di n ha esattamente $n - 1$ nodi interi ai quali è associato un lavoro unitario. Di conseguenza la complessità è $T(n) = n - 1 = \Theta(n)$. Si tratta della stessa complessità che abbiamo ottenuto sfruttando l'approccio iterativo bottom-up (sezione 5.1).

Nota che la memoizzazione nasconde i costi della ricorsione: se esiste un approccio iterativo bottom-up, della stessa efficienza, allora va preferito. Ci sono tuttavia casi in cui l'approccio top-down permette la risoluzione di un numero minore di sotto-istanze, in questi casi la memoizzazione è più efficiente dell'approccio iterativo. C'è inoltre da considerare che il codice memoizzato è spesso molto più intuitivo e "naturale" del codice bottom-up, ed è anche di più facile scrittura.

5.4 Problemi di ottimizzazione e proprietà di sotto-struttura ottima

La programmazione dinamica viene spesso utilizzata per risolvere *problemi di ottimizzazione*. Per definire formalmente questa classe di problema andiamo ad estendere la definizione di problema computazionale. Nel primo capitolo abbiamo definito formalmente un problema Π come una relazione tra un insieme delle istanze $\mathbb{I}$ e un insieme delle soluzioni $\mathbb{S}$, cioè $\Pi \subseteq \mathbb{I} \times \mathbb{S}$. Siccome la relazione non è univoca, ad ogni istanza possono essere associate più soluzioni. Per una generica istanza $i \in \mathbb{I}$, definiamo l'insieme delle **soluzioni ammissibili** di i come:

$$S(i) = \{s \in \mathbb{S} : i \Pi s\}.$$

Un problema di ottimizzazione può essere un problema di minimizzazione o massimizzazione rispetto ad una **funzione di costo** $c : \mathbb{S} \rightarrow \mathbb{R}$, che associa ad ogni soluzione un numero reale. Risolvere un problema di ottimizzazione per l'istanza i significa trovare la migliore soluzione ammissibile $S(i)$ rispetto alla funzione di costo, o dimostrare che $S(i) = \emptyset$. Una **soluzione ottima** $s^* \in S(i)$ soddisfa il **criterio di ottimalità**:

$$c(s^*) = \min/\max \{c(s) : s \in S(i)\}.$$

Nei problemi di ottimizzazione si definiscono **proprietà di sotto-struttura ottima** che mettono in relazione soluzioni ottime di istanze con soluzioni ottime di sotto-istanze.

Nota che i problemi di ottimizzazione non vengono risolti per forza di cose con un approccio DP, ma possono anche essere risolti con un approccio D&C. La scelta della metodologia di risoluzione viene fatta basandosi sulle considerazioni che abbiamo fatto nelle precedenti sezioni.

5.5 Paradigma DP

Elenchiamo i passi da seguire per risolvere un problema sfruttando la DP:

1. **Caratterizzazione della proprietà di sotto-struttura ottima.** Vanno identificate le soluzioni per le istanze di base e la relazione che lega la soluzione ottima s^* di un'istanza i con le soluzioni ottime $s_1^*, \dots, s_k^*$ di istanze di taglia strettamente più piccola.
2. **Costo della soluzione ottima.** Determinare una relazione matematica tra i costi, cioè una relazione del tipo:

$$c(s^*) = f(c(s_1^*), c(s_2^*), \dots, c(s_k^*)).$$

- 2'. **Informazione aggiuntiva.** Determinare la minima informazione strutturale necessaria a ricostruire la soluzione s^* dell'istanza a partire dalle soluzioni delle sotto-istanze $s_1^*, \dots, s_k^*$. Serve per non dover memorizzare esplicitamente tutte le soluzioni alle sotto-istanze. La soluzione ottima dell'istanza principale viene ottenuta (in *postprocessing*) combinando le soluzioni ottime delle sotto-istanze.
3. **Calcolo dei costi.** Impostare il calcolo di $c(s^*)$ in forma bottom-up, oppure top-down memoizzata.
- 3'. **Calcolo dell'informazione aggiuntiva.** Concorrentemente al calcolo dei costi, impostare il calcolo dell'informazione aggiuntiva.

I passi (2') e (3') possono essere ignorati se si è interessati a ricavare solamente il costo della soluzione ottima e non l'informazione necessaria a ricostruire la soluzione stessa (situazione che si verifica spesso in diversi problemi).

5.6 Problemi su stringhe

Partiamo introducendo la nomenclatura e le notazioni che utilizzeremo nel seguito. Dato un alfabeto finito di simboli Σ , una stringa $X = \langle x_1, \dots, x_m \rangle$ è una concatenazione finita di simboli $x_i \in \Sigma$. Con ϵ si indica la stringa vuota. L'insieme di tutte le stringhe di lunghezza finita su Σ viene indicato con Σ^* . L'operatore $*$ che, dato un alfabeto, ritorna l'insieme di tutte le sue stringhe finite, prende il nome di *stella di Kleene*.

Data una stringa $X = \langle x_1, \dots, x_m \rangle$, definiamo:

- I **prefissi** di X sono

$$X_i = \langle x_1, \dots, x_i \rangle, \quad 0 \leq i \leq m$$

dove $X_0 = \langle \rangle = \epsilon$ è il prefisso vuoto.

- I **suffissi** di X sono

$$X^i = \langle x_i, \dots, x_m \rangle, \quad 0 \leq i \leq m + 1.$$

dove $X^{m+1} = \langle \rangle = \epsilon$ è il suffisso vuoto.

- Una **sottostringa** di X è

$$X_{i..j} = \begin{cases} \langle x_i, \dots, x_j \rangle & 1 \leq i \leq j \leq m \\ \epsilon & i > j \end{cases}$$

Nota che prefissi e suffissi sono particolari sottostringhe. Data una stringa di lunghezza m , il **numero di sottostringhe totali** è dato da

$$\binom{m}{2} = \frac{m(m-1)}{2}$$

stringhe di lunghezza ≥ 2 più una sola sottostringa di lunghezza m più la sottostringa vuota ϵ . In tutto sono:

$$\frac{m(m+1)}{2} + 1 = \Theta(m^2).$$

- Una **sottosequenza** di X è una stringa $Z = \langle z_1, \dots, z_k \rangle$ per cui esiste una successione di indici strettamente crescente estratta da $\{1, 2, \dots, m\}$ di k valori:

$$1 \leq j_1 < j_2 < \dots < j_k \leq m$$

tali per cui $Z_i = X_{j_i}$ per ogni $1 \leq i \leq k$. Ovvero si ha che $Z = \langle x_{j_1}, x_{j_2}, \dots, x_{j_k} \rangle$. Si dice che la successione di indici *realizza* Z in X . Per esempio, poniamo

$$X = \langle A, B, C, B, B, D \rangle, \quad Z = \langle A, C, B \rangle.$$

Z è sottosequenza di X in quanto la successione di indici $\langle j_1 = 1, j_2 = 3, j_3 = 4 \rangle$ realizza Z in X . Notiamo che anche la successione crescente $\langle j_1 = 1, j_2 = 3, j_3 = 5 \rangle$ realizza Z in X . La differenza tra una sottostringa e una sottosequenza è che quest'ultima può essere fatta di caratteri non contigui (ma estratti in senso crescente) di X . Notiamo infine che X_i , X^i e $X_{i..j}$ sono anche sottosequenze di X . Il **numero di sottosequenze** di una stringa di lunghezza m è data da

$$\sum_{k=0}^m \binom{m}{k} = 2^m.$$

Il numero di sottosequenze è molto più grande del numero di sottostringhe.

5.7 Longest Common Subsequence (LCS)

Date due stringhe $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$, il Longest Common Subsequence richiede di determinare una stringa Z di massima lunghezza che sia sottosequenza di X e di Y . Per esempio:

$$\begin{aligned} X &= \langle A, B, C, B, B, D \rangle \\ Y &= \langle A, C, C, C, B, D \rangle \\ Z &= \text{LCS}(X, Y) = \langle A, C, B, D \rangle \end{aligned}$$

dove Z è realizzata da $\langle 1, 3, 4, 6 \rangle$ in X e da $\langle 1, 3, 5, 6 \rangle$ in Y . La LCS è una metrica di similarità: più la lunghezza di Z si avvicina al $\min\{m, n\}$ (che è il massimo valore della lunghezza di Z) e più le due stringhe sono "simili".

Trovare una LCS di due stringhe è un tipo di problema in cui non siamo tanto interessati alla stringa in sé ma alla sua lunghezza. Nella prossima sottosezione vediamo come realizzare un algoritmo DP che risolve il problema della LCS.

5.7.1 Proprietà di sotto-struttura ottima

Risolvere il problema dell'individuare una LCS utilizzando un approccio enumerativo richiede tempo esponenziale al caso peggiore. Dunque vediamo come sviluppare una proprietà di sotto-struttura ottima che permetta di risolvere il problema in modo più efficiente. Partiamo da delle considerazioni:

1. Se abbiamo che $X = \varepsilon$ oppure $Y = \varepsilon$, allora $Z = \text{LCS}(X, Y) = \varepsilon$ (caso base).
2. Se abbiamo $X = \langle X', a \rangle$ e $Y = \langle Y', a \rangle$, ovvero le due stringhe terminano con lo stesso simbolo, allora possiamo dire che sicuramente anche Z terminerà con il simbolo a . Inoltre se $Z = \langle Z', a \rangle$, allora $Z' = \text{LCS}(X', Y')$, altrimenti Z non potrebbe essere la sottosequenza comune più lunga di X e Y .
3. Se invece abbiamo $X = \langle X', a \rangle$ e $Y = \langle Y', b \rangle$ con $a \neq b$, allora sicuramente Z non termina con uno tra a e b (il che non significa che debba terminare per forza con uno tra questi due, potrebbe infatti terminare con un altro simbolo diverso da entrambi). Possiamo ricondurci a due sotto-istanze più piccole (dato che uno dei due simboli è inutile): $\text{LCS}(X, Y')$ e $\text{LCS}(X', Y)$ prendendo la LCS più lunga tra le due.

Abbiamo trovato un modo per mettere in relazione l'istanza (X, Y) con sotto-istanze che sono coppie di prefissi delle due stringhe:

$$(X', Y') = (X_{m-1}, Y_{n-1}), \quad (X, Y') = (X_m, Y_{n-1}), \quad (X', Y) = (X_{m-1}, Y_n).$$

Il vantaggio è che lo spazio formato dalle coppie di prefissi di una stringa di lunghezza m e una di lunghezza n ha cardinalità $\Theta(n \cdot m)$, quindi quadratica e non esponenziale come il numero di sottosequenze di una delle due stringhe.

Possiamo enunciare formalmente la proprietà di struttura ottima per la generica sotto-istanza (X_i, Y_j) .

Teorema 5.1. *Siano X_i e Y_j prefissi di X e Y , con $0 \leq i \leq m$ e $0 \leq j \leq n$. Sia $Z = \text{LCS}(X_i, Y_j) = \langle z_1, \dots, z_k \rangle$ la sottosequenza comune più lunga di X_i e Y_j . Vale che:*

1. *Se $i = 0$ o $j = 0$, allora $Z = \varepsilon$ (caso base).*
2. *Se $i > 0$ e $j > 0$, allora abbiamo due casi:*
 - (a) *Se $x_i = y_j$ allora $z_k = x_i = y_j$ e $Z_{k-1} = \text{LCS}(X_{i-1}, Y_{j-1})$.*
 - (b) *Se $x_i \neq y_j$ allora Z è la stringa di lunghezza maggiore tra $\text{LCS}(X_{i-1}, Y_j)$ e $\text{LCS}(X_i, Y_{j-1})$.*

5.7.2 Dimostrazione della proprietà di sotto-struttura ottima

Consideriamo il **caso base**, abbiamo $i = 0$ o $j = 0$, dunque $X = \varepsilon$ oppure $Y = \varepsilon$. Se uno dei due prefissi è la stringa vuota, allora la loro $\text{LCS}(X_i, Y_j)$ deve essere per forza la stringa vuota.

Consideriamo ora il **caso (2.a)**:

- Poniamo per assurdo che $Z = \text{LCS}(X_i, Y_j)$ di lunghezza k , abbia $z_k \neq x_i = y_j$, cioè poniamo che l'ultimo simbolo sia diverso. Siano $1 \leq i_1 < i_2 < \dots < i_k \leq i$ e $1 \leq j_1 < j_2 < \dots < j_k \leq j$ due successioni di indici che realizzano Z in X_i e in Y_j rispettivamente. Ovvero abbiamo:

$$Z = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle = \langle y_{j_1}, y_{j_2}, \dots, y_{j_k} \rangle$$

L'ipotesi che $z_k \neq x_i$ e $z_k \neq y_j$ implica che $i_k < i$ e $j_k < j$. Ma allora possiamo ottenere una nuova sottosequenza Z'

$$Z' = \langle Z, x_i \rangle = \langle Z, y_j \rangle$$

una sottosequenza comune di X_i, Y_j ottenuta aggiungendo alle successioni di indici $i_{k+1} = i$ e $j_{k+1} = j$. Dato che $|Z'| = |Z| + 1$, allora Z non è $\text{LCS}(X_i, Y_j)$ e dunque abbiamo trovato una contraddizione con l'ipotesi iniziale.

Abbiamo dimostrato che $z_k = x_i = y_j$. Ci rimane da dimostrare che $Z_{k-1} = \text{LCS}(X_{i-1}, Y_{j-1})$.

- Poiché $i_{k-1} < i$ e $j_{k-1} < j$, allora Z_{k-1} è sicuramente una sottosequenza comune di X_i e Y_j . Ci rimane da dimostrare che non solo è una sottosequenza comune, ma è anche la più lunga.
- Z_{k-1} deve essere la più lunga, altrimenti, se ne esistesse una più lunga, potrei sostituire Z_{k-1} con questa LCS ed aggiungendo z_k , otterrei una Z' sottosequenza di X_i e Y_j più lunga di Z . Questo è un assurdo visto che abbiamo ipotizzato che $Z = \text{LCS}(X_i, Y_j)$.

Dimostriamo il **caso (2.b)**:

- Se $z_k = x_i$, allora si può assumere che $i_k = i$ e $j_k < j$ (dato che $z_k = x_i \neq y_j$) e quindi Z è una sottosequenza comune di X_i e Y_{j-1} . Z deve essere la $\text{LCS}(X_i, Y_{j-1})$, altrimenti potrei trovare una $\text{LCS}(X_i, Y_j)$ più lunga di Z e in questo caso sarei in contraddizione con l'ipotesi del teorema.
- Se invece $z_k \neq x_i$, allora $i_k < i$ e quindi Z è sicuramente una sottosequenza comune di X_{i-1} e Y_j . Z deve essere la $\text{LCS}(X_{i-1}, Y_j)$, altrimenti potrei trovare una $\text{LCS}(X_i, Y_j)$ più lunga di Z e in questo caso sarei in contraddizione con l'ipotesi del teorema.
- Infine, siccome $\text{LCS}(X_i, Y_{j-1})$ e $\text{LCS}(X_{i-1}, Y_j)$ sono entrambe anche sottosequenze comuni dei prefissi X_i e Y_j , Z risulterà essere la stringa di lunghezza maggiore tra le due.

Per dimostrare la correttezza della proprietà di sottostruttura abbiamo sfruttato ripetutamente il ragionamento per assurdo: abbiamo negato la tesi e abbiamo dimostrato che in questo modo si può ottenere una sottosequenza comune più lunga di Z , e ciò contraddice l'ottimalità di Z . Nella dimostrazione per assurdo, per ricavare una sottosequenza più lunga, abbiamo sostituito un pezzo della (presente) LCS con un'altra sottosequenza, e questo ci ha permesso di arrivare all'assurdo. Si tratta di una tecnica standard che viene chiamata **cut-and-paste**.

5.7.3 Ricorrenza sui costi e informazione aggiuntiva

Ora che abbiamo ricavato e dimostrato la correttezza della proprietà di sottostruttura ottima, possiamo passare a ricavare la **ricorrenza sui costi**. Si tratta di ricavare una relazione di ricorrenza che legghi la lunghezza (cioè il costo) della soluzione ottima con le lunghezze delle sottosequenze ottime. Definiamo con $I(i, j) = |\text{LCS}(X_i, Y_j)|$ la lunghezza della LCS di X_i e Y_j . Per ricavare la relazione di ricorrenza, possiamo sfruttare direttamente la proprietà di sottostruttura ottima:

$$I(i, j) = \begin{cases} 0 & (i = 0) \vee (j = 0) \\ 1 + I(i - 1, j - 1) & (i > 0) \wedge (j > 0) \wedge (x_i = y_j) \\ \max \{I(i, j - 1), I(i - 1, j)\} & (i > 0) \wedge (j > 0) \wedge (x_i \neq y_j) \end{cases}$$

Passiamo all'**informazione aggiuntiva**. Vogliamo determinare la minima informazione strutturale necessaria a ricostruire la $\text{LCS}(X_i, Y_j)$ partendo dalle soluzioni delle sottoistanze del problema. Notiamo che:

- Caso base ($(i = 0) \vee (j = 0)$): non serve nessuna informazione.
- Caso non di base ($(i > 0) \wedge (j > 0)$): ci basta tenere traccia di quale sottoistanza determina $\text{LCS}(X_i, Y_j)$.
 - Se $x_i = y_j$ allora $Z = \text{LCS}(X_{i-1}, Y_{j-1})$ quindi possiamo memorizzare il simbolo $b(i, j) = \nwarrow$.
 - Se $x_i \neq y_j$ allora se $Z = \text{LCS}(X_{i-1}, Y_j)$ allora memorizziamo $b(i, j) = \uparrow$; altrimenti se $Z = \text{LCS}(X_i, Y_{j-1})$ allora memorizziamo $b(i, j) = \leftarrow$;

Il significato dei simboli è legato alla posizione all'interno di una matrice (i è l'indice di riga e j è quello di colonna); successivamente vedremo come sfruttare tale informazione per ricavare la $\text{LCS}(X_i, Y_j)$. Nota che con solamente 3 simboli (rappresentabili con 2 bit)

è possibile ricostruire la LCS, non è necessario memorizzare le LCS delle sottoistanze (che richiederebbero molto più spazio). Riassumendo:

$$b(i, j) = \begin{cases} \swarrow & (i > 0) \wedge (j > 0) \wedge (x_i = y_j) \\ \uparrow & (i > 0) \wedge (j > 0) \wedge (x_i \neq y_j) \wedge Z = \text{LCS}(X_{i-1}, Y_j) \\ \leftarrow & (i > 0) \wedge (j > 0) \wedge (x_i \neq y_j) \wedge Z = \text{LCS}(X_i, Y_{j-1}) \end{cases}$$

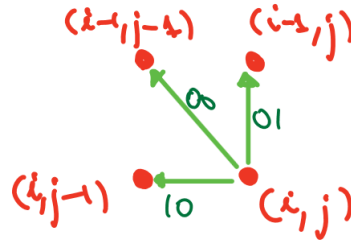


Figura 5.3: Grafico delle dipendenze.

5.7.4 Calcolo dei costi e dell'informazione aggiuntiva (approccio bottom-up)

Sfruttiamo l'approccio bottom-up, più avanti vedremo anche come realizzare la versione con il codice memoizzato. Per il calcolo dei costi utilizziamo una tabella bidimensionale $I[0..m, 0..n]$ (avente m righe ed n colonne) e per il calcolo dell'informazione aggiuntiva $b[0..m, 0..n]$. **Nota che** sfruttando l'approccio bottom-up, per poter calcolare $I[i, j]$ devo aver già calcolato $I[i-1, j-1]$, $I[i-1, j]$ e $I[i, j-1]$. In altre parole, per risolvere la sottoistanza di indici (i, j) può essere risolta solamente dopo aver risolto le sotto-istanze di indici $(i-1, j-1)$, $(i-1, j)$ e $(i, j-1)$. Per capire in che modo scansionare la tabella $I[0..m, 0..n]$ possiamo analizzare il grafico delle dipendenze.

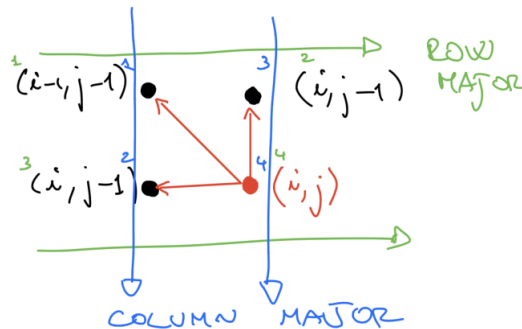


Figura 5.4: Scansionando la tabella dei costi secondo l'ordine row major: calcolo prima la sotto-istanza di indici $(i-1, j-1)$, poi $(i-1, j)$, successivamente $(i, j-1)$ e infine (i, j) . Scansionando la tabella dei costi secondo l'ordine column major: calcolo prima la sotto-istanza di indici $(i-1, j-1)$, poi $(i, j-1)$, successivamente $(i-1, j)$ e infine (i, j) .

Sia sfruttando una scansione row major che column major, andiamo a calcolare la sotto-istanza di indici (i, j) per ultima. Dunque, per il problema che stiamo considerando, possiamo scegliere una qualunque delle due scansioni. Vedremo che non è sempre così, in alcuni casi dobbiamo utilizzare per forza la scansione diagonale.

5.7.5 Codice bottom-up

Il codice si ricava direttamente da ciò che abbiamo detto nelle precedenti sottosezioni.

```

1 LCS(X, Y)
2   m ← |X|
3   n ← |Y|
4
5   // Inizializzo la tabella dei costi con i casi base.
6   for i ← 0 to m do l[i,0] ← 0 // Tutta la prima colonna ho j=0.
7   for j ← 1 to n do l[0,j] ← 0 // Tutta la prima riga ho i=0.
8
9   // Utilizziamo la scansione row major.
10  for i ← 1 to m do
11    for j ← 1 to n do
12      if (xi = yj) then
13        l[i,j] ← 1 + l[i-1,j-1]
14        b[i,j] ← '↖'
15      else if (l[i-1,j] >= l[i,j-1]) then // Caso in cui xi ≠ yj
16        l[i,j] ← l[i-1,j]
17        b[i,j] ← '↑'
18      else
19        l[i,j] ← l[i,j-1]
20        b[i,j] ← '←'
21  return (l[m,n], b) // Torno un intero e una tabella.

```

La **correttezza** dell'algoritmo deriva dalla proprietà di sottostruttura ottima e dal fatto che la realizzazione dell'algoritmo bottom-up è coerente con il grafico delle dipendenze. Per calcolare la complessità consideriamo un modello di costo che considera solamente i confronti tra elementi della tabella. Otteniamo:

$$T(m, n) = \sum_{i=1}^m \sum_{j=1}^n 1 = m \cdot n = \Theta(n \cdot m)$$

ovvero una complessità quadratica.

Supponiamo ora di essere **interessati solamente al costo**. In tal caso possiamo modificare l'algoritmo in maniera tale che occupi meno spazio in memoria. Dal grafico delle dipendenze, notiamo che non ha senso memorizzare l'intera tabella $l[0..m, 0..n]$, ci bastano solamente due righe. Possiamo quindi rimpiazzare la tabella con due vettori $prev[0..n]$ e $curr[0..n]$ che indicano rispettivamente la riga $(i - 1)$ -esima e la riga i -esima. Il codice diventa il seguente:

```

1 LCS(X, Y)
2   m ← |X|
3   n ← |Y|
4
5   for 0 ← 1 to m do prev[j] ← 0
6   curr[0] ← 0
7
8   for i ← 1 to m do
9     for j ← 1 to n do
10      if (xi = yj) then
11        curr[j] ← 1 + prev[j-1]
12      else if (prev[j] >= curr[j-1]) then
13        curr[j] ← prev[j]
14      else

```

```

15         curr[j] ← curr[j-1]
16     prev ← curr
17     return curr[n]
    
```

Infine vediamo come sfruttare l'informazione addizionale per stampare la LCS. L'algoritmo si ricava direttamente da come abbiamo definito $b(i, j)$:

```

1 PRINT_LCS(X, b, i, j)
2   if (i = 0) or (j = 0) then return
3   if (b[i, j] = '↖') then
4       PRINT_LCS(X, b, i-1, j-1) // Prima stampo LSC( $X_{i-1}, Y_{j-1}$ ).
5       print( $x_i$ ) // Poi stampo  $x_i$ .
6   else if (b[i, j] = '←') then
7       PRINT_LCS(X, b, i, j-1) // LSC( $X_i, Y_j$ ) = LSC( $X_i, Y_{j-1}$ ).
8   else
9       PRINT_LCS(X, b, i-1, j) // LSC( $X_i, Y_j$ ) = LSC( $X_{i-1}, Y_j$ ).
    
```

Se consideriamo come modello di costo il numero di invocazioni di `print`, allora la complessità **complessità** è $T(m, n) = |\text{LCS}(X_m, Y_n)| = O(\min\{m, n\})$.

5.7.6 Complessità al caso peggiore utilizzando un approccio D&C

Nelle sottosezioni precedenti abbiamo visto come calcolare la LCS di due stringhe utilizzando un approccio DP. Se avessimo utilizzato l'approccio D&C, sarebbe cambiato qualcosa? Che complessità si otteneva? L'algoritmo si ricava direttamente dalla ricorrenza sui costi (della sottosezione 5.7.3). (Consideriamo di essere interessati solamente ai costi).

```

1 LCS_2(X, Y, i, j)
2   if (i = 0) or (j = 0) then return 0
3   if ( $x_i = y_j$ ) then return 1 + LSC_2(X, Y, i-1, j-1)
4   a = LSC_2(X, Y, i, j-1)
5   b = LSC_2(X, Y, i-1, j)
6   if (a >= b) then return a
7   return b
    
```

Al caso peggiore (cioè quando le due stringhe hanno tutti simboli diversi), la **complessità**

$$T(m, n) = \begin{cases} 0 & m = 0 \wedge n = 0 \\ T(m, n-1) + T(m-1, n) + 1 & \text{altrimenti} \end{cases}$$

Per semplificare l'analisi, consideriamo $m = n$ e otteniamo la seguente minorazione

$$T(n, n) = T(n, n-1) + T(n-1, n) + 1 \geq T(n-1, n-1) + T(n-1, n-1) + 1 = 2T(n-1, n-1) + 1.$$

Risolviamo la relazione di ricorrenza utilizzando la tecnica dell'unfolding:

$$\begin{aligned}
 T(n, n) &\geq 2T(n-1, n-1) + 1 \\
 &\geq 2(2T(n-2, n-2) + 1) + 1 = 4T(n-2, n-2) + 2 + 1 \\
 &\vdots \\
 &\geq 2^i T(n-i, n-i) + \sum_{j=0}^{i-1} 2^j.
 \end{aligned}$$


```

25     l[i,j] ← l[i-1, j] // l[i-1, j] viene calcolato dalla chiamata
        REC_LCS(X, Y, i-1, j).
26     else
27         l[i,j] ← l[i, j-1] // l[i, j-1] viene calcolato dalla chiamata
        REC_LCS(X, Y, i, j-1).
28     return l[i,j]

```

La **correttezza** deriva direttamente dalla proprietà di sottostruttura e dalla scelta del valore di default. La **complessità**: al caso peggiore (quando le due stringhe non hanno nessun elemento in comune) vengono risolte tutte le $n \cdot m$ sotto-istanze non di base 1 sola volta come nodi interni. Dunque $T(m, n) \leq m \cdot n$.

Con l'approccio top-down offerto dall'algoritmo memoizzato, esistono casi in cui il numero di sotto-istanze risolte è inferiore a quello che risolve l'algoritmo iterativo bottom-up. Per esempio, consideriamo le due stringhe

$$X = \langle A, A, B, B, C \rangle, \quad Y = \langle B, B, C \rangle$$

di lunghezza 5 e 3. Le sotto-istanze risolte dall'algoritmo top down sono le seguenti

$$(5, 3) \rightarrow (4, 2) \rightarrow (3, 1) \rightarrow (2, 0)$$

mentre l'algoritmo bottom-up ne richiede $5 \cdot 3 = 15$! Questo perché, quando $x_i = y_i$, l'algoritmo top-down non va a risolvere anche le sotto-istanze $(i-1, j)$ e $(i, j-1)$ (come invece fa l'algoritmo bottom-up). Siamo quindi nel caso in cui l'approccio bottom-up risolve più sotto-istanze di quelle strettamente necessarie alla risoluzione dell'istanza principale. Come abbiamo detto all'inizio del capitolo, l'approccio ricorsivo permette di evitare questa problematica.

5.8 Matrix Chain Multiplication (MCM)

Studiamo un altro problema in cui l'approccio della DP permette di migliorare significativamente le prestazioni che si ottengono con un approccio *D&C* (il quale ottiene prestazioni pessime a causa dell'elevato numero di sotto-istanze ripetute).

5.8.1 Premesse

Consideriamo due matrici rettangolari $A \in {}_p\mathbb{C}_q$ e $B \in {}_q\mathbb{C}_r$, queste si dicono **compatibili** se per esse è definita la matrice $C = A \times B$ ottenuta utilizzando il prodotto righe per colonna. Si ha che ${}_p\mathbb{C}_r$ con

$$c_{ij} = \sum_{k=1}^q a_{ik} b_{kj}$$

Ricaviamo l'algoritmo che calcola C direttamente dalla definizione:

```

1  RECT_MAT(A,B) { columns(A) = rows(B) }
2      p ← rows(A)
3      q ← columns(A)
4      r ← columns(B)
5      for i ← 1 to p do
6          for j ← 1 to r do
7              cij ← ai1 · b1j
8              for k ← 2 to q do
9                  cij ← cij + aik · bkj
10     return C

```


Considerando con costo unitario solamente il prodotto tra scalari (non le somme) si ottiene come **complessità** $T(p, q, r) = p \cdot q \cdot r$.

Supponiamo ora di avere una catena di matrici $\langle A_1, \dots, A_n \rangle$ compatibili per il prodotto. Per essere compatibili deve essere vero che:

$$\text{columns}(A_i) = \text{rows}(A_{i+1}) \quad 1 \leq i < n$$

ovvero le matrici devono essere compatibili a coppie adiacenti. Possiamo definire un vettore $\vec{p}$ che contiene tutte le dimensioni distinte delle matrici appartenenti alla catena. Si nota che il numero di dimensioni distinte sono in tutto $n + 1$ (nel nostro esempio precedente avevamo due matrici e si aveva $n + 1 = 2 + 1 = 3$ dimensioni distinte). In particolare si ha che:

$$p_0 = \text{rows}(A_1), \quad p_n = \text{columns}(A_n), \quad p_i = \text{columns}(A_i) = \text{rows}(A_{i+1}) \quad 1 \leq i \leq n - 1$$

$$\vec{p} \in (\mathbb{N}^+)^{n+1} \quad \text{vettore delle dimensioni.}$$

L'obiettivo che ci poniamo è quello di calcolare il prodotto di matrici $A_1 \times A_2 \times \dots \times A_n \in {}_{p_0}\mathbb{C}_{p_n}$ nel modo più efficiente possibile. Ogni **parentesizzazione completa**¹ della catena, indica un ordine diverso di esecuzione del prodotto. Esiste un ordine migliore degli altri? Consideriamo un esempio con $n = 3$:

$$A_1 \in {}_{10}\mathbb{C}_{100}, \quad A_2 \in {}_{100}\mathbb{C}_5, \quad A_3 \in {}_5\mathbb{C}_{50},$$

$$\vec{p} = (10, 100, 5, 50), \quad C = A_1 \times A_2 \times A_3 \in {}_{10}\mathbb{C}_{50}$$

le possibili parentesizzazioni sono soltanto due:

$$C = ((A_1 \times A_2) \times A_3), \quad C = (A_1 \times (A_2 \times A_3))$$

sfruttando RECT_MAT possiamo ricavare i costi associati alle due parentesizzazioni:

$$((A_1 \times A_2) \times A_3) : 10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 7500$$

$$(A_1 \times (A_2 \times A_3)) : 100 \cdot 5 \cdot 50 + 100 \cdot 50 \cdot 10 = 75000$$

Notiamo quindi che la parentesizzazione influisce sui costi.

Osservazione 5.2. Ogni parentesizzazione può essere rappresentata come un **albero binario pieno**, in cui le foglie rappresentano gli operandi e i nodi interi rappresentano i singoli prodotti (ricordando che se gli operandi sono n allora i singoli prodotti sono $n - 1$).

Il numero di parentesizzazioni possibili coincide con il numero di possibili alberi pieni aventi n foglie. D'ora in avanti identificheremo una parentesizzazione con l'albero binario completo T che la rappresenta.

5.8.2 Descrizione del problema

Dato un vettore $\vec{p} = (p_0, \dots, p_n)$, delle dimensioni di n matrici rettangolari compatibili, determinare la parentesizzazione T^* associata al costo minimo per l'esecuzione del prodotto a catena. Questa è la descrizione del problema *Matrix Chain Multiplication*².

L'**approccio banale** prevede di considerare tutte le possibili parentesizzazioni. Ma come cresce il numero totale rispetto alla dimensione della catena considerata? Per ogni catena di lunghezza n si ha che il numero $P(n)$ di parentesizzazioni è dato da:

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k) \cdot P(n-k) & n > 1 \end{cases}$$

¹La parentesizzazione sfrutta la proprietà associativa del prodotto.

²Nota che non servono le matrici ma solo le loro dimensioni.

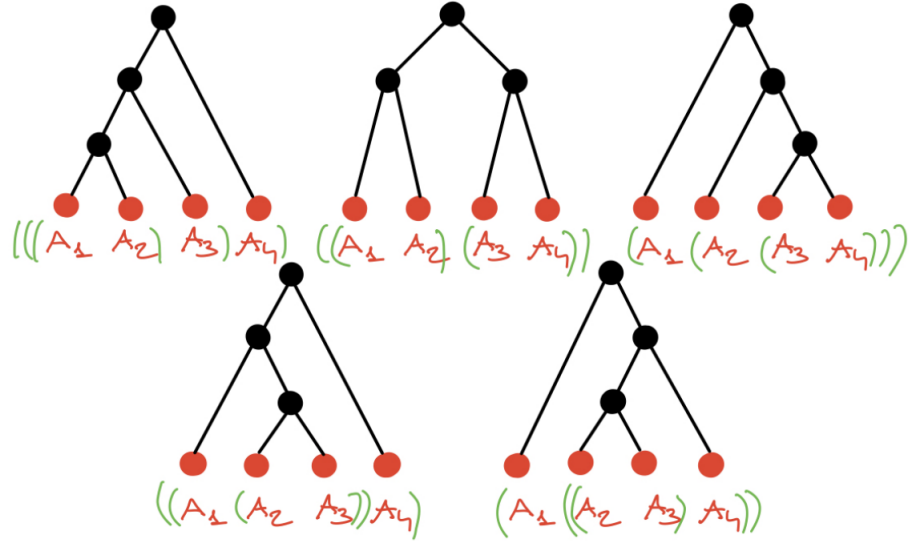


Figura 5.6: Con $n = 4$ operandi si ottengono 5 parentesizzazioni (alberi binari) possibili.

dove $P(k)$ sono le $P(n-k)$ forniscono le sottoparentesizzazioni (sottoalberi) rispettivamente con k e $n-k$ foglie. Si dimostra che:

$$P(n) = \frac{1}{n} \binom{2(n-1)}{n-1} = \Theta\left(\frac{4^n}{n^{3/2}}\right).$$

Dunque l'approccio banale non è applicabile, conviene invece trovare una proprietà di sottostruttura.

5.8.3 Proprietà di sottostruttura ottima

Osservazione 5.3. Una parentesizzazione di una catena contiene al suo interno sottoparentesizzazioni di sottocatene.

La precedente osservazione deriva dal fatto che una parentesizzazione può essere vista come un albero binario pieno e un sottoalbero di un albero binario pieno è ancora un albero binario pieno.

Si tratta ora di capire come determinare la parentesizzazione ottima $T_{i..j}^*$ per la sottocatena $A_{i..j} = \langle A_i, \dots, A_j \rangle$. Ragioniamo quindi sull'istanza generica ricordando che siamo interessati a $T_{1..n}^*$. Consideriamo la risoluzione del caso di base e del caso non di base:

- **Caso di base:** abbiamo $i = j$ quindi $A_{i..j} = A_{i..i} = \langle A_i \rangle$. L'unica parentesizzazione è $T_{i..i}^* = \bullet$: un albero pieno composto da un solo nodo radice/foglia.
- **Casi non di base:** sia data una soluzione ottima $T_{i..j}^*$ per $i < j$. Possiamo considerare i due sottoalberi della radice individuati a partire da un certo valore $\bar{k}$ tale che $i \leq \bar{k} < j$:

$$T_{i..\bar{k}}^*, \quad T_{\bar{k}+1..j}^*.$$

Questi sono associati rispettivamente alla parentesizzazione delle sottocatene $\langle A_i, \dots, A_{\bar{k}} \rangle$ e $\langle A_{\bar{k}+1}, \dots, A_j \rangle$. In questo modo si ha che $T_{i..j}^*$ calcola $A_{i..j}$ come $A_{i..\bar{k}} \times A_{\bar{k}+1..j}$.

Si tratta di dimostrare che, per il caso non di base, i due sottoalberi individuati a partire dal valore di $\bar{k}$ tale $i \leq \bar{k} < j$, siano effettivamente ottimi. Per farlo, consideriamo la relazione sui costi per $T_{i..j}^*$:

$$c(T_{i..j}^*) = c(T_{i..\bar{k}}) + c(T_{\bar{k}+1..j}) + p_{i-1} \cdot p_{\bar{k}} \cdot p_j$$

dove l'ultimo termine deriva dal fatto che $A_{i..\bar{k}} \in \mathbb{C}_{p_{i-1}}$, $A_{\bar{k}+1..j} \in \mathbb{C}_{p_{\bar{k}}}$ e $A_{i..\bar{k}} \times A_{\bar{k}+1..j} = A_{i..j} \in \mathbb{C}_{p_j}$; quindi utilizzando l'algoritmo RECT_MAT si ottiene come costo (numero di prodotti) $p_{i-1} \cdot p_{\bar{k}} \cdot p_j$. Ora, se avessimo che $T_{i..\bar{k}} \neq T_{i..\bar{k}}^*$ oppure che $T_{\bar{k}+1..j} \neq T_{\bar{k}+1..j}^*$ (ovvero se i due sottoalberi non fossero parentesizzazioni ottime), allora si potrebbe sostituire il sottoalbero non ottimo con quello ottimo e ottenere una nuova nuova parentesizzazione $\hat{T}_{i..j}$ migliore di $T_{i..j}^*$. Questo è un **assurdo** visto che stiamo supponendo che $T_{i..j}^*$ sia ottimo. Ancora una volta abbiamo sfruttato la tecnica del **cut-and-paste** per arrivare all'assurdo.

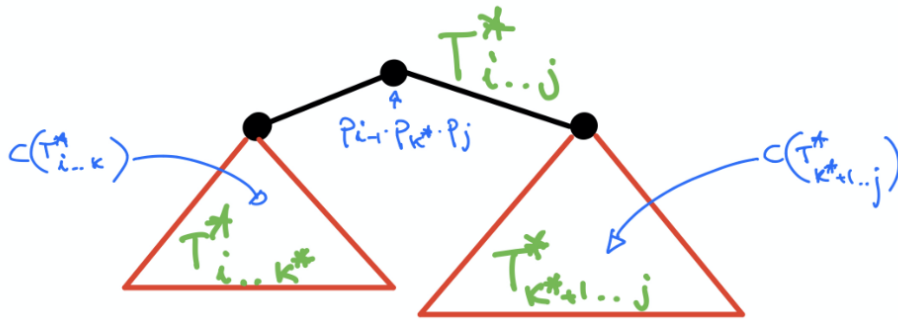


Figura 5.7: Caso non di base: la parentesizzazione ottima (albero binario pieno) $T_{i..j}^*$ è formata da due sottoparentesizzazioni ottime (sottoalberi binari pieni).

Abbiamo dimostrato che una parentesizzazione ottima contiene al suo interno sottoparentesizzazioni ottime. Questo non basta per dimostrare la proprietà di sottostruttura ottima, ci serve definire meglio $\bar{k}$, $i \leq \bar{k} < j$. Il valore di $\bar{k}$ in $T_{i..j}^*$ deve essere quello corrispondente alla composizione di costo minimo:

$$\bar{k} = \operatorname{argmin} \{ c(T_{i..k}) + c(T_{k+1..j}) + p_{i-1} \cdot p_k \cdot p_j : i \leq k < j \}.$$

Altrimenti, per assurdo, se così non fosse, allora si potrebbe ottenere una parentesizzazione migliore per $A_{i..j}$ spezzando rispetto ad un qualche $k \neq \bar{k}$. Questo di nuovo sarebbe un assurdo in quanto abbiamo supposto che $T_{i..j}^*$ è la parentesizzazione ottima. Abbiamo dimostrato la correttezza della proprietà di sottostruttura ottima.

5.8.4 Ricorrenza sui costi e informazione aggiuntiva

La **ricorrenza sui costi** si ricava direttamente dalla proprietà di sottostruttura ottima. Poniamo

$$c(i, j) = c(T_{i..j}^*) \quad 1 \leq i \leq j \leq n$$

allora la ricorrenza è data

$$c(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ c(i, k) + c(k+1, j) + p_{i-1} \cdot p_k \cdot p_j \} & 1 \leq i < j \leq n \end{cases}$$

L'**informazione aggiuntiva** è data da $s(i, j) = \bar{k}$.

5.8.5 Approccio DP bottom-up

Prima di analizzare l'approccio bottom-up, ci chiediamo se veramente necessitiamo di DP, oppure possiamo sfruttare un approccio D&C. La domanda che ci poniamo è: esiste in questo caso il problema delle sottoistanze ripetute?

```

1 R_MCM( $\vec{p}$ , i, j)
2   if (i = j) then return 0
3   m  $\leftarrow$   $+\infty$ 
4   for k  $\leftarrow$  i to j-1 do
5     q  $\leftarrow$  R_MCM( $\vec{p}$ , i, k) + R_MCM( $\vec{p}$ , k+1, j) +  $p_{i-1} \cdot p_k \cdot p_j$ 
6     if (q < m) then m  $\leftarrow$  q // Salvo il nuovo minimo.
7   return m

```

La ricorrenza associata alla chiamata $R_MCM(\vec{p}, 1, n)$ è:

$$T(n) = \begin{cases} 0 & n = 1 \\ \sum_{k=1}^{n-1} (T(k) + T(n-k) + 2) = 2 \sum_{k=1}^{n-1} T(k) + 2(n-1) & n > 1 \end{cases}$$

Si dimostra (per induzione) che $T(n) > 2^n - 2$. Tale complessità esponenziale è dovuta alla presenza di sottoistanze ripetute. Dunque sì, conviene utilizzare un algoritmo DP.

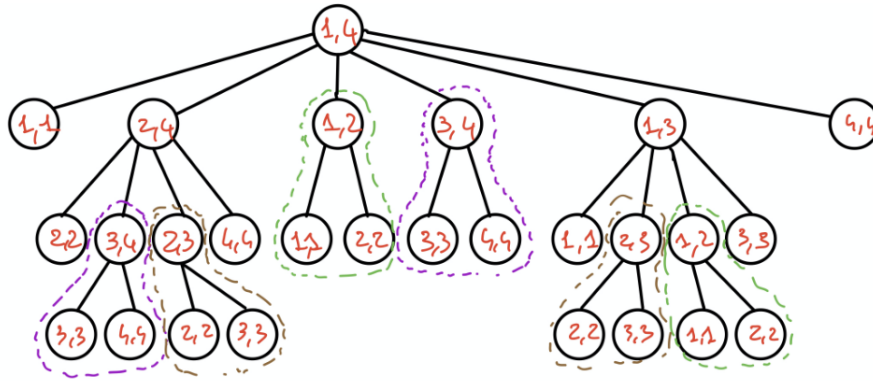


Figura 5.8: Albero delle chiamate di R_MCM per $n = 4$.

Per implementare l'approccio iterativo bottom-up, definiamo le due tabelle

$$c[i..n, 1..n], \quad s[1..n, 1..n]$$

rispettivamente per i costi associati all'istanza (i, j) e per l'informazione aggiuntiva. Si tratta ora di organizzare la computazione in modo che al momento di calcolare $c(i, j)$ si abbiano a disposizione i valori di $c(i, k)$ e $c(k+1, j)$ per $i \leq k < j$.

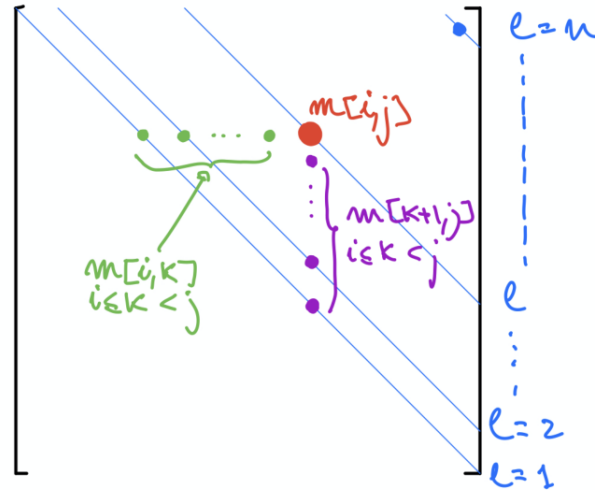


Figura 5.9: Grafico delle dipendenze per MCM bottom-up ($m[i,j]$ corrisponde a $c[i,j]$). Nota che nel triangolo inferiore della tabella abbiamo valori che non utilizziamo in quanto in quei casi abbiamo che $i > j$.

Dal grafico delle dipendenze notiamo che non è possibile utilizzare né la scansione row major né la scansione column major. Si deve utilizzare la **scansione per diagonali parallele a quella principale**. Infatti in entrambe, $c[i,j]$ (in rosso) viene scansionato prima di tutti i valori $c[k+1,j]$ per $i \leq k < j$ (in viola). Notiamo che i valori necessari all'istanza (i,j) sono relativi a sottocatene più corte. Inoltre gli elementi relativi a catene di lunghezza l si trovano sulla l -esima diagonale parallela alla diagonale principale. L'equazione per identificare una diagonale è $l = j - i + 1$. Notiamo infine che i costi di base $c[i,j] = 0$ ($l = 1$) si trovano sulla diagonale principale.

Dunque: troviamo sulla diagonale $l = 1$ i casi di base; per $l = 2, \dots, n$, fissato l'indice i , si ha che $l = j - i + 1 \Rightarrow j = i + l - 1$. L'indice i varia da 1 a $n - l + 1$ (dove ricordiamo che n è la lunghezza della catena di matrici $\langle A_1, \dots, A_n \rangle$).

```

1 MCM( $\vec{p}$ )
2    $n \leftarrow \vec{p}.\text{size} - 1$  // Lunghezza catena di matrici da moltiplicare.
3   for  $i \leftarrow 1$  to  $n$  do
4      $c[i,i] \leftarrow 0$  // Inizializzo casi di base.
5   for  $l \leftarrow 2$  to  $n$  do
6     for  $i \leftarrow 1$  to  $n-l+1$  do
7        $j \leftarrow i + l - 1$ 
8        $c[i,j] \leftarrow +\infty$ 
9       for  $k \leftarrow i$  to  $j-1$  do
10         $q \leftarrow c[i,k] + c[i+1,j] + p_{i-1} \cdot p_k \cdot p_j$ 
11        if ( $q < c[i,j]$ ) then
12           $c[i,j] \leftarrow q$  // Salvo il nuovo minimo.
13           $s[i,j] \leftarrow k$ 
14   return  $c[1,n]$ ,  $s$  // Costo associato all'istanza  $(1,n)$  e tabella con
    l'informazione aggiuntiva.

```

La **correttezza** dell'algoritmo deriva dalla proprietà di sottostruttura individuata e dalla scansione diagonale. Ricaviamo la **complessità** considerando come modello di costo solamente i

prodotti (non somme o assegnamenti):

$$\begin{aligned}
 T(n) &= \sum_{l=2}^n \sum_{i=1}^{n-l+1} \sum_{k=i}^{i+l-2} 2 \\
 &= 2 \sum_{l=2}^n \sum_{i=1}^{n-l+1} (i + l - 2 - i + 1) \\
 &= 2 \sum_{l=2}^n \sum_{i=1}^{n-l+1} (l - 1) \\
 &= 2 \sum_{l=2}^n (l - 1)(n - l + 1 - 1 + 1) \\
 &= 2 \sum_{l=2}^n (l - 1)(n - (l - 1)) \quad \text{pongo } l' = l - 1 \\
 &= 2 \sum_{l'=1}^n l'(n - l') \\
 &= 2n \sum_{l'=1}^{n-1} l' + 2 \sum_{l'=1}^{n-1} (l')^2 \\
 &= 2n \frac{n(n-1)}{2} - 2 \frac{(n-1)n(2n-1)}{6} \\
 &= \frac{n^3 - n}{3}
 \end{aligned}$$

5.8.6 Algoritmo bottom-up con diverse scansioni della matrice

Dal grafico delle dipendenze di Figura 5.9 notiamo che anche sono valide anche le scansioni reverse rows major e reverse column major. Vediamo come cambia l'algoritmo utilizzando la scansione reverse rows major:

```

1 MCM_REV_ROW( $\vec{p}$ )
2    $n \leftarrow \vec{p}.\text{size} - 1$ 
3   for  $i \leftarrow 1$  to  $n$  do
4      $c[i,i] \leftarrow 0$ 
5   for  $i \leftarrow n-1$  down to 1 do
6     for  $j \leftarrow i+1$  to  $n$  do
7        $c[i,j] \leftarrow +\infty$ 
8       for  $k \leftarrow i$  to  $j-1$  do
9          $q \leftarrow c[i,k] + c[k+1,j] + p_{i-1} \cdot p_k \cdot p_j$ 
10        if ( $q < c[i,j]$ ) then
11           $c[i,j] \leftarrow q$ 
12           $s[i,j] \leftarrow k$ 
13   return  $c[1,n], s$ 

```

La variante reverse column major:

```

1 MCM_REV_COL( $\vec{p}$ )
2    $n \leftarrow \vec{p}.size - 1$ 
3   for  $i \leftarrow 1$  to  $n$  do
4      $c[i,i] \leftarrow 0$ 
5   for  $j \leftarrow n$  down to 2 do
6     for  $i \leftarrow j-1$  down to 1 do
7        $c[i,j] \leftarrow +\infty$ 
8       for  $k \leftarrow i$  to  $j-1$  do
9          $q \leftarrow c[i,k] + c[k+1,j] + p_{i-1} \cdot p_k \cdot p_j$ 
10        if ( $q < c[i,j]$ ) then
11           $c[i,j] \leftarrow q$ 
12           $s[i,j] \leftarrow k$ 
13   return  $c[1,n], s$ 

```

5.8.7 Algoritmo memoizzato

Sempre sfruttando la scaletta vista nella sezione 5.3, possiamo ricavare direttamente il codice dell'algoritmo memoizzato. Calcoliamo solo i costi e non anche l'informazione aggiuntiva.

```

1 INIT_MCM( $\vec{p}$ )
2    $n \leftarrow \vec{p}.size - 1$  // Lunghezza catena di matrici da moltiplicare.
3   if ( $n = 1$ ) then return 0
4   for  $i \leftarrow 1$  to  $n$  do
5      $c[i,i] \leftarrow 0$  // Caso base.
6     for  $j \leftarrow i+1$  to  $n$  do // Inizializzo casi non di base (solo triangolo
7       // superiore della tabella)
8        $c[i,j] \leftarrow +\infty$ 
9   return REC_MCM( $\vec{p}, 1, n$ )
10 REC_MCM( $\vec{p}, i, j$ )
11   if ( $c[i,j] = +\infty$ ) then
12     for  $k \leftarrow i$  to  $j-1$  do
13        $q \leftarrow \text{REC\_MCM}(\vec{p}, i, k) + \text{REC\_MCM}(\vec{p}, k+1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
14       if ( $q < c[i,j]$ ) then  $c[i,j] \leftarrow q$  // Salvo io nuovo minimo.
15   return  $c[i,j]$ 

```

Per quanto riguarda la **complessità**, si nota che ogni sottoistanza non di base (i, j) è associata ad un nodo interno una sola volta ed il lavoro ad esso associato (dato dal for) è $2(j-1-i+1) = 2(j-i)$. Si trova dunque:

$$T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2(j-i) = \frac{n^3 - n}{3}$$

5.8.8 Stampa della parentesizzazione di costo minimo

```

1 PRINT_MCM( $s, i, j$ )
2   if ( $i = j$ ) then printf("A_{%c}",  $i$ )
3   printf("(")
4   PRINT_MCM( $s, i, s[i,j]$ ) //  $s[i,j] = k$ 
5   PRINT_MCM( $s, s[i,j]+1, j$ )
6   printf(")")

```

Considerando con costo unitario il numero di caratteri stampati, si ha che la **complessità** è:

$$T(n) = \underbrace{n}_{\text{foglie}} + 2(\underbrace{n-1}_{\text{nodi interni}}) = 3n - 2$$

Capitolo 6

Il paradigma Greedy

Il paradigma Greedy¹, come la programmazione DP, si applica ai problemi di ottimizzazione.

6.1 Critica alla programmazione dinamica

La programmazione dinamica applica un approccio che l'opposto rispetto al paradigma Greedy, la prima infatti procede in maniera molto più cauta. Abbiamo visto nel capitolo precedente che l'approccio DP costruisce la soluzione ottima come una composizione di scelte: attraverso l'informazione addizionale potevamo codificare queste scelte e ricostruire la soluzione ottima.

Per esempio, nel calcolo dell'algoritmo *Matrix Chain Multiplication*, dovevamo decidere il valore di $\bar{k}$. Tale valore permetteva di dividere in due l'albero binario associato alle sotto-catene di matrici. Ricordiamo che il calcolo del valore di $\bar{k}$ ottimo richiedeva la risoluzione di diverse sottoistanze. Un altro esempio: nel calcolo della LCS abbiamo visto che c'era il caso in cui $LCS(X_m, Y_n) = LCS(X_{m-1}, Y_{n-1})$ se $X_m = Y_n$ che era associato alla scelta ↖, oppure $LCS(X_m, Y_n) = LCS(X_m, Y_{n-1})$ associato alla scelta ← e infine si poteva avere $LCS(X_m, Y_n) = LCS(X_{m-1}, Y_n)$ associato alla scelta ↑. In particolare, negli ultimi due casi, la decisione dipendeva dal calcolo di diverse sottoistanze: molteplici sottoistanze dovevano essere risolte prima di poter prendere una decisione.

Riassumendo: il problema principale del paradigma DP è che le scelte possono essere effettuate solamente dopo aver risolto numerose sottoistanze (procede in maniera cauta).

6.2 Aspetti generali dell'approccio Greedy

In generale l'approccio Greedy non è applicabile per qualsiasi tipologia di problema, per esempio non è possibile applicarlo per il calcolo della LCS e del MCM. Esiste però una classe di problemi che invece trae vantaggio dall'approccio Greedy. Vediamo ad alto livello quelli che sono le due caratteristiche principali dell'approccio Greedy:

1. **Scelta Greedy** (scelta "incauta"). Si effettua immediatamente una scelta, senza risolvere sottoistanze. La scelta da effettuare è quella che sembra essere **localmente** la migliore.
2. **Individuazione della sottoistanza residua**. Dopo aver effettuato la scelta si ripulisce l'istanza dagli elementi non compatibili con la scelta effettuata. Questa fase (chiamata di **clean-up**) permette di individuare una singola sottoistanza (detta **sottoistanza residua**) dello stesso problema che viene poi risolta utilizzando lo stesso procedimento (ritornando al punto (1) considerando questa nuova sottoistanza residua come input).

¹Greedy sta per incauto o imprudente.

Ricaviamo le seguenti osservazioni:

1. Ancora una volta, l'approccio è basato su una **proprietà di sottostruttura ottima**. Infatti, la soluzione di una istanza si ottiene componendo le scelte Greedy con la soluzione (ottima) della sottoistanza residua. Notiamo che il paradigma Greedy mette in relazione ogni istanza (non di base con una singola sottoistanza (la sottoistanza residua)). Ciò comporta che lo spazio delle sottoistanze è lineare.
2. La ricorsione presenta una sola sottoistanza per livello dell'albero delle chiamate (catena). Dunque non possono esistere sottoistanze ripetute.
3. Siccome l'albero delle chiamate è una catena (si parla di *ricorsione di coda*): l'unica chiamata ricorsiva viene effettuata alla fine della fase di clean-up. In questo caso la ricorsione si può sostituire facilmente con un approccio iterativo in cui si alternano le fasi di scelta greedy e di clean-up.

Come abbiamo già anticipato in precedenza, l'approccio greedy non è applicabile a tutte le classi di problemi, precisiamo ora che la classe di problemi che sfruttano questo approccio è ristretta. Ciò è dovuto al fatto che la costruzione della soluzione per scelte locali non sempre converge alla soluzione ottima. Un altro svantaggio dell'approccio greedy è la non immediatezza delle dimostrazioni di correttezza che però viene compensata da un codice molto semplice e immediato.

6.3 Analisi di un algoritmo Greedy

Vanno dimostrate due proprietà della strategia risolutiva:

1. **Proprietà di scelta greedy (greedy choice, GCP)**: bisogna dimostrare che esiste sempre una soluzione ottima componibile a partire dalla prima scelta greedy. In altre parole si tratta di dimostrare che la scelta incauta non compromette la presenza di una soluzione ottima.
2. **Proprietà di sottostruttura ottima (optimal substructure, OSP)**: si tratta di dimostrare che esiste una soluzione ottima che, oltre alla scelta greedy, si compone dalla scelta associata alla soluzione della sottoistanza residua.

6.4 Il problema della selezione delle attività

Consideriamo un primo caso di studio che ci permette di esaminare tutti gli aspetti del paradigma Greedy. Il problema generico che consideriamo è lo *scheduling*.

Immaginiamo di avere una risorsa condivisa (es: un aula del DEI) e un insieme di attività $S = \{a_1, \dots, a_n\}$ (es: le lezioni). Poniamo che ogni attività a_i sia associata ad un intervallo temporale $[s_i, f_i)$ di utilizzo della risorsa. Diciamo che due attività a_i, a_j sono **compatibili** se

$$[s_i, f_i) \cap [s_j, f_j) = \emptyset$$

o equivalentemente

$$(f_i \leq s_j) \vee (f_j \leq s_i).$$

Due attività compatibili possono utilizzare la risorsa senza creare dei conflitti. Diciamo che un sottoinsieme $A \subseteq S$ è composto da attività mutuamente compatibili se $\forall a_i, a_j \in A$ t.c. $a_i \neq a_j$ sono compatibili. Le attività di un insieme di attività mutuamente compatibili possono usare la risorsa senza entrare in conflitto.

Il problema della selezione delle attività (**Activity Selection Problem**), dato $S = \{a_1, \dots, a_n\}$ e $[s_i, f_i)$, consiste nel trovare un sottoinsieme $A \subseteq S$ di attività mutuamente compatibili di cardinalità massima (funzione obiettivo da massimizzare).

6.4.1 Algoritmo Greedy

Sviluppiamo una strategia greedy per risolvere il problema. Supponiamo che le attività ci vengano fornite in maniera tale che i tempi di fine f_i siano ordinati in ordine non crescente:

$$f_1 \leq f_2 \leq \dots \leq f_n.$$

Se così non fosse allora ci basta applicare un algoritmo di ordinamento (con un costo $\Theta(n \log n)$)².

Si tratta ora di trovare una **scelta greedy**. Una possibile scelta è la seguente: selezionare l'attività che finisce prima³. Così facendo lasciamo più "spazio" per selezionare ulteriori attività. Chiamiamo tale attività a_1 . La fase di **clean-up** che consideriamo è la seguente: elimino tutte le attività che seguono a_1 nella sequenza fino a che non si incontra la prima attività compatibile con a_1 oppure si eliminano tutte le attività. Avendo definito la scelta greedy e la fase di clean-up, si riesce a ricavare direttamente un algoritmo top down che esegue la scelta greedy, esegue il clean-up e invoca ricorsivamente se stesso sulla sottoistanza residua.

```

1 REC_GREEDY_SEL(s, f, i) {  $f_1 \leq f_2 \leq \dots \leq f_n$  }
2    $n \leftarrow s.size$ 
3   if ( $i > n$ ) then return  $\emptyset$ 
4    $G \leftarrow \{i\}$  // Scelta greedy.
5    $j \leftarrow i + 1$ 
6   // Clean-up: elimino le attivita' incompatibili.
7   while (( $s_j < f_i$ ) and ( $j \leq n$ )) do
8      $j \leftarrow j + 1$ 
9   return  $G \cup \text{REC\_GREEDY\_SEL}(s, f, j)$ 

```

L'algoritmo viene invocato inizialmente ponendo $i = 1$.

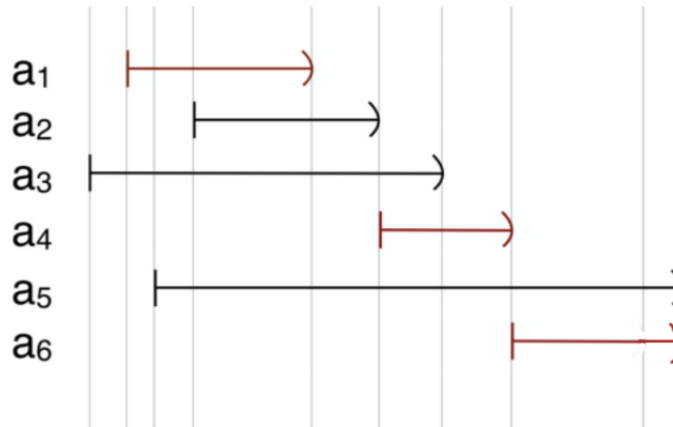


Figura 6.1: Esempio di esecuzione dell'algoritmo REC_GREEDY_SEL: le attività aventi le frecce rosse corrispondono alla soluzione proposta dall'algoritmo. Notiamo che la soluzione ottima trovata non è l'unica, infatti, per esempio, anche il sottoinsieme $A = \{a_2, a_4, a_6\}$ contiene attività mutuamente compatibili e ha sempre dimensione 3.

²Nota che avere i tempi f_i ordinati in modo non crescente non implica che lo siano anche i tempi di inizio s_i .

³Potrebbe essere più intuitiva la scelta "selezionare l'attività che dura meno". Tuttavia vedremo che tale scelta non porta sempre all'individuazione di una soluzione ottima (come invece fa la scelta greedy che abbiamo posto noi)

L'algoritmo può essere riscritto facilmente in modo da eliminare la ricorsione.

```

1 IT_GREEDY_SEL(s, f)
2   n ← s.size
3   A ← {1}
4   g ← 1
5   for i ← 2 to n do
6       if (si ≥ fg) then
7           A ← A ∪ {i}
8           g ← i
9   return A
    
```

L'indice g rappresenta la scelta greedy corrente (all'inizio la prima attività), mentre il ciclo aggiunge come prossima attività a_i , ovvero la prima che risulta compatibile con a_g (aggiornando successivamente g per riflettere la nuova scelta greedy corrente). La **complessità** è lineare: $\Theta(n)$.

6.4.2 Correttezza

Proprietà di scelta greedy Cominciamo con il dimostrare che esiste sempre una soluzione ottima componibile a partire da una scelta greedy. Nel nostro caso si vuole dimostrare che:

$$\exists A^* \text{ soluzione ottima, con } a_1 \in A^*$$

dove a_1 è appunto la nostra scelta greedy. L'idea è quella di partire da una soluzione ottima generica e di manipolarla (con il cut and paste) in modo da far rientrare la scelta greedy nella soluzione.

Consideriamo quindi una soluzione ottima $\hat{A}$ arbitraria. Se $a_1 \in \hat{A} \Rightarrow \hat{A} = A^*$ e quindi non c'è nulla da dimostrare. Nel caso in cui $a_1 \notin \hat{A}$, allora individuiamo

$$\hat{i} = \min \{i : a_i \in \hat{A}\} > 1$$

l'indice relativo all'attività appartenente ad $\hat{A}$ che termina prima. Ma allora si può ottenere

$$A^* = \hat{A} - \underbrace{\{a_{\hat{i}}\}}_{\text{cut}} \cup \underbrace{\{a_1\}}_{\text{paste}}$$

ne segue che $|A^*| = |\hat{A}|$ ovvero hanno la stessa cardinalità. Ci rimane da dimostrare che A^* è una soluzione ammissibile, per farlo ci basta dimostrare che a_1 è compatibile con ogni attività $a_j \in \hat{A} - \{a_{\hat{i}}\}$ (le altre sono già compatibili tra di loro). Siccome

$$\forall a_j \in \hat{A} - \{a_{\hat{i}}\} : f_1 \leq f_{\hat{i}} \leq s_j$$

dato che a_1 è l'attività che termina prima di tutte, allora si ha che

$$f_1 \leq s_j \Rightarrow a_1 \text{ e } a_j \text{ sono compatibili.}$$

Proprietà di sottostruttura ottima Vogliamo dimostrare che

$$A^* - \{a_1\} \text{ è soluzione ottima della sottoistanza residua } S_r.$$

Dopo la fase di clean-up, la sottoistanza residua:

1. Può essere vuota se non esistono attività compatibili con a_1 , quindi $S_r = \emptyset$.

2. Altrimenti possiamo porre $g = \min \{i : f_1 \leq s_i\}$ e dunque $S_r = \{a_g, a_{g+1}, \dots, a_n\}$.

Consideriamo $A^* - \{a_1\}$. Abbiamo due casi:

1. $A^* - \{a_1\} = \emptyset$ significa che non esiste nessuna attività compatibile con a_1 (altrimenti A^* non sarebbe ottima) e quindi

$$\forall j > 1 : s_j < f_1 \Rightarrow S_r = \emptyset$$

e dunque $A^* - \{a_1\} = \emptyset$ è soluzione ottima di $S_r = \emptyset$.

2. $A^* - \{a_1\} \neq \emptyset$ allora $\exists i > 1$ tale che a_i è compatibile con a_1 e dunque

$$S_r = \{a_g, \dots, a_n\}, \quad g = \min \{i : f_1 \leq s_i\}.$$

Notiamo che in questo caso $A^* - \{a_1\} \subseteq S_r$ visto che in A^* ho solamente attività compatibili ed entrambi gli insiemi non hanno a_1 . Di conseguenza $A^* - \{a_1\}$ è soluzione ammissibile per S_r ; ci rimane da dimostrare che è anche ottima. Per farlo utilizziamo l'assurdo e il cut and paste. Poniamo per assurdo che $A^* - \{a_1\}$ non sia ottima per S_r . Consideriamo $\hat{A} \subseteq S_r$ ottima per S_r che contiene la scelta greedy a_g per S_r (un tale $\hat{A}$ esiste visto che abbiamo dimostrato la proprietà di scelta greedy). Si ha che

$$|\hat{A}| > |A^* - \{a_1\}| = |A^*| - 1$$

in quanto stiamo supponendo che $A^* - \{a_1\}$ non sia l'ottimo. Possiamo dimostrare che a_1 è compatibile con ogni attività di $\hat{A}$:

- (a) a_1 è compatibile con a_g per come abbiamo definito g .
 (b) per quanto riguarda le rimanenti attività di $\hat{A}$ vale che

$$\forall i : a_i \in \hat{A} - \{a_g\} : f_1 \leq s_g \leq f_g \leq s_i \Rightarrow f_1 \leq s_i \Rightarrow a_1 \text{ compatibile con } a_i$$

dove abbiamo sfruttato il fatto che a_g è compatibile con a_i .

Dunque possiamo (paste) aggiungere $\{a_1\} \cup \hat{A}$ ed ottenere una soluzione per l'istanza di partenza S (non quella ridotta) tale che

$$|\{a_1 \cup \hat{A}\}| > |A^*|$$

e questo è un assurdo in quanto abbiamo trovato una soluzione migliore di A^* , cioè la soluzione supposta ottima. In questo modo abbiamo dimostrato che $A^* - \{a_1\}$ è soluzione ottima di S_r .

Nota che nella dimostrazione del punto 2, quando abbiamo posto per assurdo che $\hat{A}$ fosse la soluzione ottima di S_r , abbiamo posto che $\hat{A}$ contenesse la scelta greedy. Questo passaggio è stato necessario in quanto ci ha permesso facilmente di dimostrare che non poteva esistere in $\hat{A}$ un'attività non compatibile con a_1 .

6.4.3 Scelte Greedy alternative

Come facciamo a capire se una scelta greedy che ci sembra ragionevole sia corretta? Si usano i controesempi. Per risolvere il problema della selezione delle attività, abbiamo posto come scelta greedy *scelgo l'attività che finisce prima*. Un'altra scelta che poteva sembrare ragionevole era *scelgo l'attività che ha durata più breve*. Per dimostrare che questa scelta greedy alternativa non porta ad una soluzione ottima, ci basta trovare un esempio in cui si verifica ciò che vogliamo dimostrare.

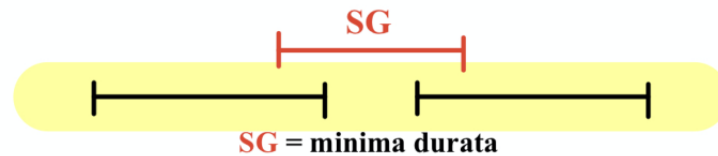


Figura 6.2: Notiamo come in questo esempio la soluzione ottima ha dimensione 2, mentre con la scelta greedy *scelgo l'attività che ha la durata più breve* troviamo un insieme di dimensione uno.

6.5 Compressione

Dato un file $F \in C^*$ (dove C^* sono sequenze di caratteri dell'alfabeto C) e le frequenze relative $\{f(c) : c \in C\}$ vogliamo determinare un **prefix code** efficiente

$$e : C \rightarrow \{0, 1\}^*$$

(dove $\{0, 1\}^*$ sono sequenze di zeri e uni) per la compressione di F . Con prefix code si intende una codifica di carattere a lunghezza variabile, in cui nessuna **codeword** $e(c)$ è il prefisso di un'altra codeword $e(c')$ con $c \neq c'$.

L'obiettivo di questa sottosezione è quello di realizzare un algoritmo (l'**algoritmo di Huffman**) greedy che permette di costruire un prefix code.

6.5.1 Rappresentazione di un prefix code

Per rappresentare una codifica lunghezza variabile, è preferibile utilizzare una struttura ad albero chiamata **trie**. Un trie è un albero binario avente delle etichette (bit) sugli archi:

- archi verso il *figlio sinistro* hanno etichetta 0;
- archi verso il *figlio destro* hanno etichetta 1.

Ogni nodo è associato alla stringa ottenuta come concatenazione delle etichette sul cammino dalla radice al nodo.

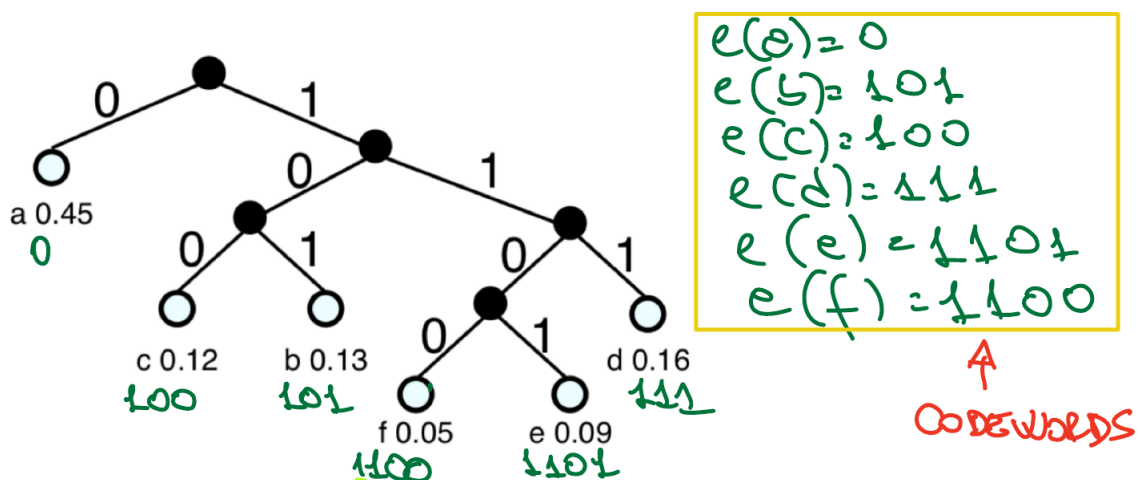


Figura 6.3: Esempio di un trie associato ad un prefix code per il file F .

Notiamo che in un trie di un prefix code, le codeword sono associate sempre alle foglie; i nodi interni sono associati a prefissi delle stringhe dei loro discendenti e dunque non possono essere delle codeword.

6.5.2 Metrica di costo

Sia T il trie associato ad un prefix code. Indichiamo con $d_T(c)$ la profondità in T dell'etichetta contenente il carattere $c \in C$. Si ha che $d_T(c) = |e(c)|$. Il **numero di occorrenze** del carattere c nel file F è dato da $f(c) \cdot |F|$, dove $f(c)$ è la frequenza con cui il carattere compare. Lo spazio occupato da F dopo la compressione è dato da:

$$|e(F)| = \sum_{c \in C} \underbrace{|e(c)|}_{\text{num bit } e(c)} \underbrace{(f(c) \cdot |F|)}_{\text{num occorrenze di } c \text{ in } F} = \left(\sum_{c \in C} d_T(c) f(c) \right) \cdot |F|.$$

Possiamo definire

$$B(T) = \sum_{c \in C} d_T(c) f(c)$$

allora

$$|e(F)| = B(T) \cdot |F|.$$

La codifica migliore per il file F è quindi quella associata al trie T^* che minimizza $B(T)$.

6.5.3 Problema della codifica di carattere ottima

Possiamo definire formalmente il problema della codifica di carattere ottima. Dato un alfabeto C e un insieme di frequenze $f : C \rightarrow \mathbb{Q}^+$, si vuole determinare il prefix code e associato al trie T^* (avente $|C|$ foglie) che minimizza $B(T)$.

Nota che non è necessario avere a disposizione il file F ma solo l'alfabeto e le frequenze dei caratteri. Nella successiva fase di compressione e decompressione sarà ovviamente necessario il file F . Inoltre, oltre a $e(F)$, è necessario memorizzare anche il trie T .

6.5.4 Proprietà di un trie ottimo

Si dimostra che un trie ottimo deve essere un albero binario pieno⁴. Dimostriamolo per assurdo:

- Poniamo che per assurdo esista un albero T^* ottimo non pieno.
- Ciò significa che esiste almeno un nodo interno u avente un unico figlio v (supponiamo che sia un figlio sinistro).
- Chiamiamo T' il sottoalbero di radice v (potrebbe anche essere composto solamente dal nodo v). Tale sottoalbero contiene almeno una foglia c' .
- Possiamo costruire $\hat{T}$ da T^* contraendo u a v in un solo nodo. In questo modo u diventa la radice di T' e, come risultato della contrazione, si ha che la lunghezza di tutte le codeword associate alle foglie di T' si accorcia di 1 bit (scompare il bit 0 associato all'arco (u, v)). Considerando $\hat{T}$ e T^* , possiamo dire che:

$$d_{\hat{T}}(c) \leq d_{T^*}(c) \quad \forall c \in C \setminus \{c'\}, \quad d_{\hat{T}}(c') = d_{T^*}(c') - 1$$

visto che la foglia c' "sale" di un livello.

⁴Ogni nodo interno deve avere due figli.

- Abbiamo ottenuto che:

$$\begin{aligned}
 B(\hat{T}) &= \sum_{c \in C} f(c) d_{\hat{T}}(c) = f(c') d_{\hat{T}}(c') + \sum_{c \in C \setminus \{c'\}} f(c) d_{\hat{T}}(c) \\
 &\leq f(c') (d_{T^*}(c') - 1) + \sum_{c \in C \setminus \{c'\}} f(c) d_{T^*}(c) \\
 &= f(c') d_{T^*}(c') - f(c') + \sum_{c \in C \setminus \{c'\}} f(c) d_{T^*}(c) \\
 &= -f(c') + \sum_{c \in C \setminus \{c'\}} f(c) d_{T^*}(c) + f(c') d_{T^*}(c') \\
 &= -f(c') + \sum_{c \in C} f(c) d_{T^*}(c) \\
 &= B(T') - f(c) \\
 &< B(T')
 \end{aligned}$$

dato che $f(c) > 0$. Siamo giunti all'assurdo: abbiamo ottenuto che $B(\hat{T}) < B(T^*)$ ma avevamo supposto T^* trie ottimo.

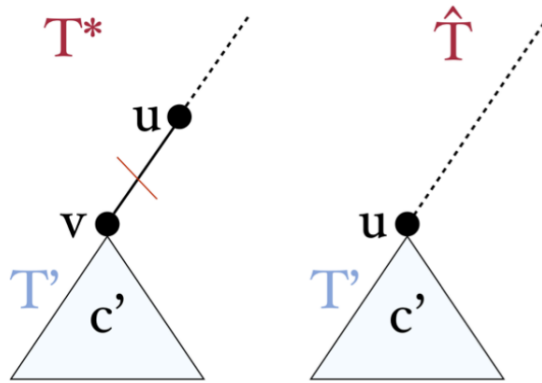


Figura 6.4

Osservazione 6.1. Se $|C| = n$, allora un trie pieno T ha sempre n foglie e $n - 1$ nodi interni.

6.5.5 Codifica di Huffman: intuizione

Intuizione Partiamo da alcune considerazioni:

- Vogliamo organizzare il processo di costruzione del trie ottimo T^* in passi, ciascuno associato a una scelta.
- Date le n foglie associate ai caratteri di C (e alle loro frequenze) bisogna costruire il trie creando $n - 1$ nodi interni.
- Ad ogni passo si ha una foresta di alberi parziali: all'inizio si parte da una foresta composta da n foglie.
- La scelta consiste nell'unione di due alberi parziali con una nuova radice (in questo modo si viene a formare un nodo interno). I due alberi parziali diventano il sottoalbero sinistro e destro della nuova radice (operazione di **merge**).

- Dopo esattamente $n - 1$ operazioni di merge, si ottiene una foresta composta da un unico albero.

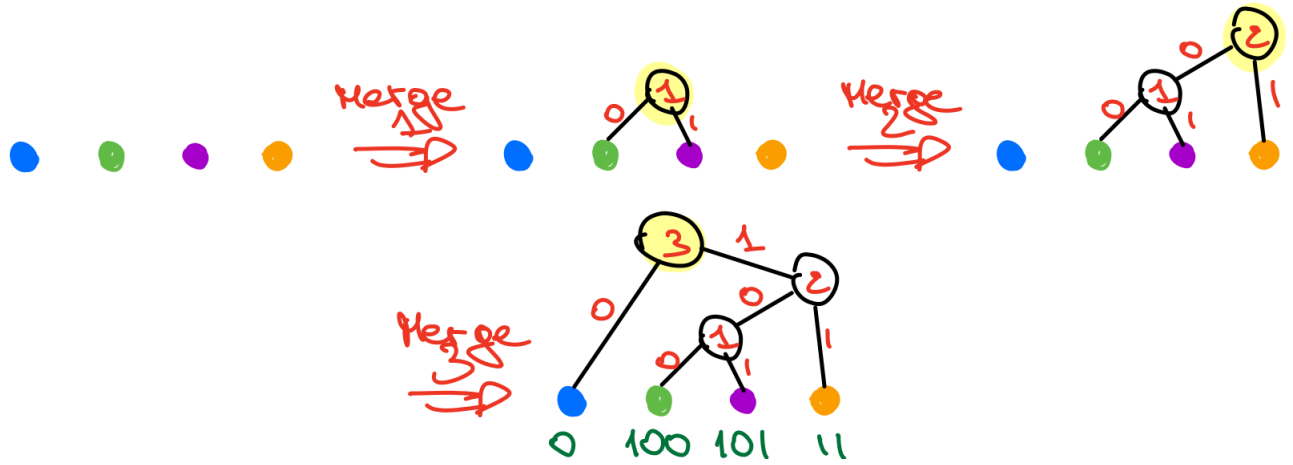


Figura 6.5

Possiamo associare ad ogni nodo interno la frequenza cumulativa delle foglie di cui è radice. In questo modo si ha che la radice del trie ha frequenza cumulativa unitaria (per le proprietà della probabilità).

Scelta greedy e fase di clean-up Si tratta ora di individuare una **scelta greedy**. Notiamo che un'operazione di merge fa aumentare di uno la profondità delle foglie coinvolte e ricordiamoci che la profondità è legata alla lunghezza della codifica del carattere. Ricordiamoci inoltre che l'obiettivo è quello di ottenere delle codifiche più lunghe per i caratteri associati ad una frequenza minore e più corte per quelli che hanno una frequenza maggiore. In conseguenza a queste considerazioni, sembra ragionevole la seguente scelta greedy:

Faccio un merge delle foglie che hanno associata una frequenza minore.

La fase di **clean-up** in questo caso è molto semplice: la foresta viene aggiornata sostituendo ai due sottoalberi *merged* il nuovo sottoalbero con frequenza cumulativa pari alla somma delle frequenze cumulative. Si ripetono queste due fasi ($n - 1$ volte in tutto) finché non si crea l'albero.

Notiamo che unire due foglie implica che le codewords dei due caratteri associati differiscono solo nell'ultimo bit. Notiamo anche che dopo il merge, la **sottoistanza residua** consiste nella codifica dei restanti $n - 2$ caratteri, più la codifica del prefisso comune dei due caratteri coinvolti nel merge.

Algoritmo Greedy L'algoritmo deve creare nuovi sottoalberi selezionando, ad ogni iterazione, i due sottoalberi aventi frequenza cumulativa minore. Possiamo utilizzare una coda prioritaria per memorizzare la foresta corrente: ogni sottoalbero ha come chiave la sua frequenza. Introduciamo un tipo di dato astratto chiamato **node**. Un **node** z ha come campi: $z.left$, $z.right$ che resituiscono rispettivamente il figlio sinistro e destro del nodo; $z.f \in \mathbb{Q}^+$ che restituisce la frequenza cumulativa del nodo; $z.char \in C$ che restituisce il carattere associato al nodo (solo se è una foglia). Una foglia ha $z.left = z.right = nil$. Quindi la coda prioritaria Q contiene elementi di tipo **node** ordinati rispetto $z.f$. I metodi della coda prioritaria sono: $Q.insert(z)$; $Q.extractmin()$. La coda prioritaria viene implementata con un *minheap*, quindi le prestazioni dei due metodi sono $O(\log |Q|)$ (logaritmici nella dimensione).

```

1 HUFFMAN(C, f)
2   n ← |C|
3   Q ← ∅
4   // Inizializzo la foresta con le n foglie.
5   for c ∈ C do
6     z ← newnode()
7     z.f ← f(c)
8     z.char ← c
9     z.left ← nil
10    z.right ← nil
11    Q.insert(z)
12  // Scelta greedy.
13  for i ← 1 to n-1 do
14    x ← Q.extractmin()
15    y ← Q.extractmin()
16    z ← newnode()
17    z.left ← x
18    z.right ← y
19    z.f ← x.f + y.f
20    Q.insert(z) // Clean-up.
21  return Q.extractmin() // Restituisco l'unico albero presente.

```

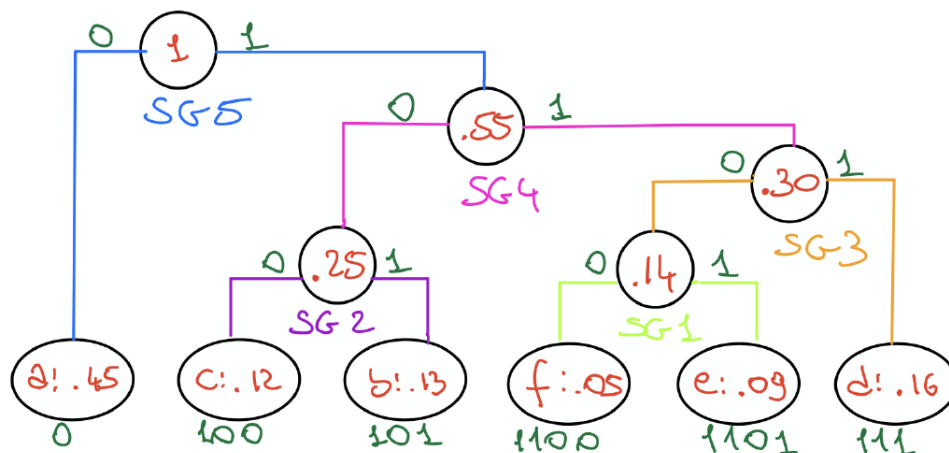


Figura 6.6: Esempio di esecuzione dell'algoritmo di Huffman.

Notiamo che per come è costruito l'algoritmo, il sottoalbero sinistro è sempre associato al figlio con frequenza cumulativa minore. L'algoritmo che abbiamo sviluppato ritorna sempre una soluzione ammissibile, infatti ritorna un trie pieno avente n foglie. Considerando $n = |C|$ si ha che $T_H(n) = \Theta(n \log n)$ in quanto vengono effettuate $\Theta(n)$ operazioni sulla coda Q .

Ci rimane solamente da dimostrare l'ottimalità del trie ritornato.

Correttezza Cominciamo dalla **proprietà di scelta greedy**: vogliamo dimostrare che esiste sempre una soluzione ottima componibile a partire da una scelta greedy. Nel nostro caso la scelta greedy è il primo merge che coinvolge le foglie associate ai due caratteri $x, y \in C$ di frequenza minore. Definiamo formalmente la proprietà di scelta greedy come segue:

Siano x e y i caratteri dell'alfabeto C associati alle frequenze più piccole, con $f(x) \leq f(y)$. Allora esiste un trie ottimo T^* in cui i nodi associati a x e y sono figli dello stesso nodo padre.

Dimostriamo la correttezza sfruttando il cut-and-paste:

- Consideriamo un generico trie ottimo T^* .
- Se x e y sono figli dello stesso padre in T^* , allora la dimostrazione termina.
- Consideriamo ora il caso in cui x , y non sono figli dello stesso padre. Individuiamo w e z due caratteri associati a nodi foglia che sono figli dello stesso padre in T^* e aventi profondità massima. Possiamo porre che $f(w) \leq f(z)$ senza perdere di generalità e supponiamo anche che $f(x) \leq f(y)$.
- Costruiamo l'albero $\hat{T}$ partendo da T^* in questo modo: scambiamo il nodo w con x e y con z (come mostrato in Figura 6.7). Nota che non è escluso che $x = w$ oppure $y = z$, in tal caso non si effettua uno swap.

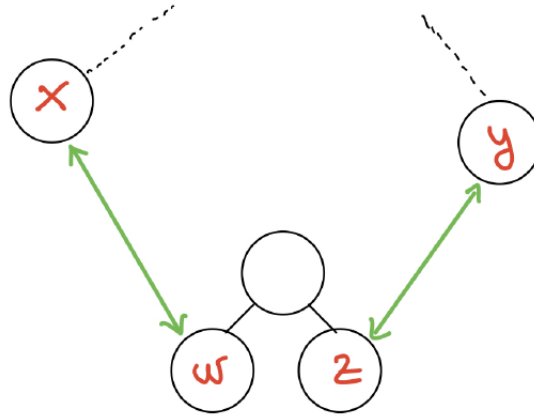


Figura 6.7: Operazione di swap dei nodi.

- Studiamo $B(\hat{T})$. In particolare consideriamo la variazione di costo tra i due trie:

$$B(T^*) - B(\hat{T})$$

possiamo subito dire che molti termini delle sommatorie si annullano, più precisamente si annullano tutti i termini tranne quelli che coinvolgono i nodi che sono stati scambiati:

$$\begin{aligned}
 B(T^*) - B(\hat{T}) &= f(w)d_{T^*}(w) + f(x)d_{T^*}(x) + f(z)d_{T^*}(z) + f(y)d_{T^*}(y) \\
 &\quad - f(w)d_{\hat{T}}(w) - f(x)d_{\hat{T}}(x) - f(z)d_{\hat{T}}(z) - f(y)d_{\hat{T}}(y) \\
 &= f(w)d_{T^*}(w) + f(x)d_{T^*}(x) + f(z)d_{T^*}(z) + f(y)d_{T^*}(y) \\
 &\quad - f(w)d_{T^*}(x) - f(x)d_{T^*}(w) - f(z)d_{T^*}(x) - f(y)d_{T^*}(z) \\
 &= (f(w) - f(x))d_{T^*}(w) - (f(w) - f(x))d_{T^*}(x) \\
 &\quad + (f(z) - f(y))d_{T^*}(z) - (f(z) - f(y))d_{T^*}(y) \\
 &= \underbrace{(f(w) - f(x))(d_{T^*}(w) - d_{T^*}(x))}_{\geq 0} + \underbrace{(f(z) - f(y))(d_{T^*}(z) - d_{T^*}(y))}_{\geq 0}
 \end{aligned}$$

ne deriva che

$$B(T^*) - B(\hat{T}) \geq 0$$

e siccome T^* è soluzione ottima si deve avere che

$$B(T^*) - B(\hat{T}) = 0 \Rightarrow B(T^*) = B(\hat{T})$$

altrimenti sarebbe un assurdo. In questo modo abbiamo ottenuto un trie ottimo $\hat{T}$ che contiene la scelta greedy.

Ci rimane da provare la **proprietà di sottostruttura ottima**. Dobbiamo dimostrare che il trie ottenuto $\hat{T}$ che contiene la scelta greedy, contiene anche un trie ottimo per la sottoistanza residua. La sottoistanza residua è il nuovo alfabeto $C_R = C \setminus \{x, y\} \cup \{z\}$ in cui $f(z) = f(x) + f(y)$. Consideriamo $\hat{T}$ ed eliminiamo x e y . Così facendo otteniamo un trie T' ammissibile per C_R . Ci rimane da dimostrare che non solo è ammissibile ma è ottimo.

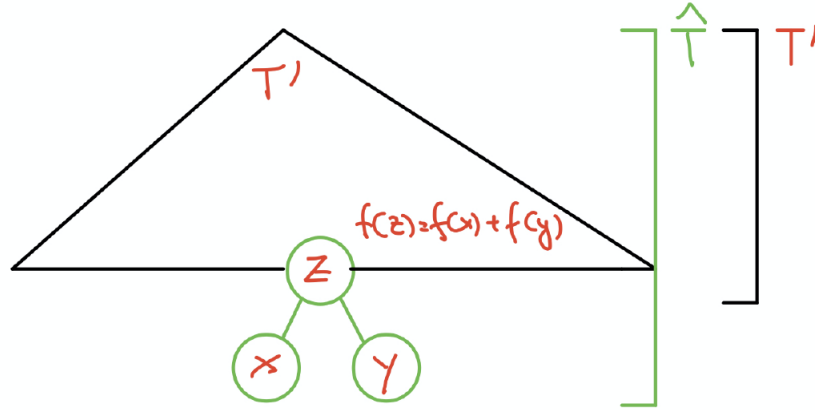


Figura 6.8

Per dimostrare che T' è soluzione ottima della sottoistanza residua C_R , mettiamo in relazione $B(\hat{T})$ con $B(T')$.

$$\begin{aligned}
 B(\hat{T}) &= \sum_{c \in C} f(c) d_{\hat{T}}(c) \\
 &= f(x) d_{\hat{T}}(x) + f(y) d_{\hat{T}}(y) + \sum_{c \in C \setminus \{x, y\}} f(c) d_{\hat{T}}(c) \\
 &= (f(x) + f(y)) d_{\hat{T}}(x) + \sum_{c \in C \setminus \{x, y\}} f(c) d_{\hat{T}}(c) \quad (x, y \text{ hanno stessa profondità}) \\
 &= (f(x) + f(y)) d_{\hat{T}}(x) + \sum_{c \in C_R \setminus \{z\}} f(c) d_{T'}(c) \quad (\text{gli altri nodi hanno stessa profondità}) \\
 &= f(z) d_{T'}(z) + f(x) + f(y) + \sum_{c \in C_R \setminus \{z\}} f(c) d_{T'}(c) \\
 &= f(x) + f(y) + \sum_{c \in C_R} f(c) d_{T'}(c) \\
 &= f(x) + f(y) + B(T')
 \end{aligned}$$

Abbiamo ricavato che $B(\hat{T}) = B(T') + f(x) + f(y)$. Sfruttiamo il cut-and-paste. Se T' non fosse ottimo per C_R , potrei considerare T'' , ottimo per C_R tale che $B(T'') < B(T')$. Da T'' potrei espandere la foglia z aggiungendogli due nodi figli x e y per ottenere un $\tilde{T}$ per C avente costo

$$B(\tilde{T}) = B(T'') + f(x) + f(y) < B(T') + f(x) + f(y) = B(\hat{T})$$

quindi abbiamo ricavato $B(\tilde{T}) < B(\hat{T})$ il che è un assurdo in quanto abbiamo supposto $\hat{T}$ soluzione ottima.